

LIMS System Documentation (Laboratory Information Management System)

Dokumen ini menyajikan prasyarat perangkat lunak, arsitektur teknis lengkap, alur akses pengguna, struktur direktori, mekanisme partisi data, pemetaan modul dengan API serta basis data, konfigurasi paket dinamis, sistem penilaian (*scoring*) LIMS, dan panduan deployment produksi Nginx terlengkap.

Daftar Isi

- [Executive Summary](#)
- [Project Objectives](#)
- [Stakeholders & User Roles](#)
- [Non-Functional Requirements](#)
- [Ringkasan Modul & Sub-Modul LIMS](#)
- [1. Prasyarat Perangkat Lunak \(Software Prerequisites\)](#)
- [2. Arsitektur Sistem & Alur Akses Pengguna \(System Architecture & Access Flow\)](#)
 - [Diagram Arsitektur Sistem & Alur Akses](#)
 - [Alur Akses Detil Pengguna](#)
- [3. Struktur Direktori Proyek \(Project Directory Structure\)](#)
 - [A. Struktur Direktori Backend \(Go\)](#)
 - [B. Struktur Direktori Frontend \(React\)](#)
- [4. Alur Kerja & Fragmentasi Data \(Workflow & Data Partitioning\)](#)
 - [A. Tahapan Alur Kerja Detil LIMS](#)
 - [B. Fragmentasi & Partisi Database \(Database Partitioning\)](#)
- [5. Modul Aplikasi, API, dan Basis Data \(Application Modules, APIs, & Database Schema\)](#)
 - [1. Modul Otentikasi & RBAC \(Auth & Access Control\)](#)
 - [2. Modul Registrasi & Pengajuan \(Submission\)](#)
 - [3. Modul Perencanaan & Pelaksanaan Uji](#)
 - [4. Modul Analisa Data & Scoring \(Scoring Engine\)](#)
 - [5. Modul Keuangan & Perjalanan \(Finance & Travel\)](#)
 - [6. Modul Manajemen Aset Peralatan Client \(Asset Tracking\)](#)
 - [7. Modul Manajemen Testing Tools \(Testing Tools Tracking\)](#)
 - [8. Modul Kecerdasan Buatan \(AI\)](#)
 - [9. Modul Integrasi Simulator IoT](#)
 - [10. Modul Monitoring](#)
 - [11. Modul Reporting](#)
 - [12. Modul Pemeliharaan Sistem](#)
- [6. Arsitektur & Konfigurasi Paket Layanan \(Testing Packages\)](#)
- [7. Tabel Basis Data \(Database Tables Directory\)](#)
- [8. Panduan Deployment Produksi LIMS \(Multi-Direktori Backend & Frontend\)](#)
 - [A. Arsitektur Produksi \(Multi-Direktori vs Multi-VM\)](#)
 - [B. Prasyarat Sistem & Manajemen Pengguna Linux](#)
 - [C. Port Mapping & Manajemen Konflik Port](#)
 - [D. Deployment & Konfigurasi Frontend \(PM2\) di VPS](#)
 - [E. Deployment & Konfigurasi Backend \(Multi-Direktori\) di VPS](#)
 - [F. Konfigurasi NGINX di VPS](#)

- [G. Rotasi Log Harian \(Logrotate\) & Penjadwalan di VPS](#)
 - [H. Real-Time Analytics dengan GoAccess](#)
 - [I. Pemantauan & Pemeliharaan \(Checklist\)](#)
 - [J. Pelatihan Model AI \(PQC\) & Penjadwalan Otomatis \(Crontab\)](#)
 - [K. Administrasi & Pemeliharaan Modul Pendukung \(PostgreSQL, MinIO, Camunda\)](#)
 - [L. Akses LIMS dari Internet \(Ngrok Dev & Produksi Asli\)](#)
 - [M. Ringkasan Lokasi Berkas Log LIMS](#)
 - [N. Ringkasan Perintah Manajemen Modul LIMS \(Start, Stop, Status\)](#)
 - [O. Strategi Manajemen Versi Aplikasi Android LIMS](#)
 - [P. Panduan Pembuatan APK Android \(Berbagai Skema Jaringan & Target\)](#)
 - [Q. Monitoring Infrastruktur LIMS \(Grafana & Prometheus\)](#)
-

Executive Summary

LIMS (*Laboratory Information Management System*) merupakan sistem terintegrasi yang dirancang untuk mendigitalkan dan mengotomatisasi seluruh siklus operasional pengujian laboratorium, mulai dari pendaftaran klien, pelacakan aset, penjadwalan uji, hingga penagihan dan pelaporan (SHP).

Project Objectives

- Mendigitalkan alur kerja pengujian secara menyeluruh (*paperless*) untuk mempercepat SLA layanan kepada klien.
- Mengintegrasikan manajemen aset, ketersediaan alat uji, sistem keuangan, dan data hasil uji ke dalam satu platform terpusat.
- Meningkatkan akurasi analisis dan evaluasi melalui integrasi kecerdasan buatan (RAG & LLM) serta pengambilan data otomatis dari IoT/Simulator.

Stakeholders & User Roles

Sistem ini melayani berbagai pemangku kepentingan dengan kontrol akses berbasis peran (RBAC), antara lain:

- **Klien / Mitra:** Pihak eksternal yang mengajukan pendaftaran uji dan menerima laporan hasil akhir.
- **Admin & Frontdesk:** Staf operasional yang menangani verifikasi registrasi, penagihan awal, dan serah terima aset fisik.
- **Analisis & Tester:** Tim teknis lapangan/lab yang mengeksekusi pengujian dan memasukkan parameter ukur.
- **Manajemen / Kepala Lab:** Pihak manajerial yang memvalidasi persetujuan SPD, uang muka, dan mengesahkan SHP.
- **Keuangan (Finance):** Staf yang mengelola pencairan dana operasional, penerimaan pembayaran, dan *settlement reimbursement*.

Non-Functional Requirements

- **Data Integrity:** Standardisasi ID 64-bit untuk memastikan kompatibilitas dengan dataset relasional skala besar (menggunakan PostgreSQL BIGINT).
- **Security:** Kontrol akses sistem secara berlapis menggunakan *Role-Based Access Control* (RBAC) dan otentikasi aman berbasis JWT (*JSON Web Tokens*).

- **Performance:** Partisi basis data (*Database Partitioning*) otomatis untuk tabel log aplikasi dan hasil uji guna memastikan waktu eksekusi kueri di bawah satu detik (*sub-second query times*).
- **Interoperability:** Integrasi mulus dengan perangkat keras (Simulator Hardware) melalui perantara protokol *RESTful API*, *HTTP Request*, maupun *MQTT*.

Ringkasan Modul & Sub-Modul LIMS

Berikut adalah ringkasan (*summary table*) dari seluruh modul dan sub-modul yang beroperasi di dalam ekosistem LIMS beserta fungsi utamanya:

Modul	Fitur	Fungsi & Kapabilitas	Backend (Golang)	Frontend (React)
1. Otentikasi & RBAC	Manajemen Pengguna & Sesi	Menangani login, pendaftaran token JWT, dan validasi sesi aktif.	controllers/auth_controller.go, middleware/auth.go, lims.user_sessions	AppContext.jsx, Header.jsx
	Kontrol Akses Berbasis Peran	Membatasi dan merender menu sidebar secara dinamis sesuai jabatan (<i>role</i>) pengguna.	middleware/role_check.go, lims.roles, lims.menus, lims.role_menus	Sidebar.jsx, MainContent.jsx
2. Registrasi & Pengajuan	Manajemen Klien (Mitra)	Mengelola data perusahaan/mitra yang mendaftarkan pengujian.	controllers/application_controller.go, lims.partners	Submission.jsx
	Pendaftaran Aplikasi Uji	Pendataan dokumen persyaratan, pengajuan uji, serta pendaftaran awal aset klien.	controllers/application_controller.go, services/camunda_service.go, services/minio_service.go, lims.testing_applications	Submission.jsx
3. Perencanaan & Pelaksanaan	Plotting Jadwal & Tim	Menjadwalkan pengujian lapangan/lab dan menugaskan tim tester tersertifikasi.	controllers/application_controller.go, lims.testing_plans, lims.tester_applications, lims.master_testers	PlanningForm.jsx
	Perekaman Hasil Uji	Form input untuk merekam nilai fisik lapangan	controllers/application_controller.go,	AppDetail.jsx (Form Input)

Modul	Fitur	Fungsi & Kapabilitas	Backend (Golang)	Frontend (React)
		berdasarkan hierarki parameter pengujian.	lims.testing_results	
4. Analisa Data & Scoring	Kalkulasi <i>Scoring Engine</i>	Menghitung persentase bobot aspek secara hierarkis (Tipe Uji → Metodologi → Grup → Aspek → Sub-Aspek).	services/scoring_service.go, lims.testing_results, lims.testing_aspect_scores, lims.testing_applications	AppDetail.jsx (View Score)
	Penentuan Kelulusan	Membandingkan nilai akhir dengan ambang batas (<i>threshold</i>) global untuk label kelulusan.	services/scoring_service.go, lims.global_parameters	AppDetail.jsx
5. Keuangan & Perjalanan	Penagihan (<i>Billing/Invoice</i>)	Pembuatan tagihan otomatis saat registrasi.	controllers/billing_controller.go, lims.invoices	Submission.jsx
	Pembayaran Invoice	Validasi dan pencatatan pembayaran invoice dari klien.	controllers/billing_controller.go, lims.payments	Submission.jsx
	Surat Perjalanan Dinas (SPD)	Pengajuan perjalanan petugas lapangan beserta rincian anggarannya.	controllers/travel_controller.go (implicit), lims.travel_requests, lims.travel_request_counters	Travel request UI
	<i>Cash Advance & Reimbursement</i>	Proses permohonan uang muka dan klaim biaya aktual (<i>upload</i> nota/struk digital).	controllers/travel_controller.go (implicit), services/minio_service.go, lims.cash_advances, lims.reimbursements, lims.travel_request_counters	Reimbursement UI

Modul	Fitur	Fungsi & Kapabilitas	Backend (Golang)	Frontend (React)
6. Manajemen Aset Peralatan Client	Pelacakan Aset Klien (<i>Moving</i>)	Pemantauan lokasi barang klien (<i>Moving Asset</i>) di dalam atau luar laboratorium.	controllers/asset_controller.go, lims.testing equipments, lims.master_asset_statuses	AssetManagementPage.jsx
	Serah Terima Aset (<i>Handover</i>)	Pembuatan dan pencetakan Berita Acara Serah Terima barang kembali kepada klien.	controllers/asset_controller.go, lims.asset_handovers	AssetManagementPage.jsx
	Movement Logs	Mencatat log perpindahan lokasi (<i>moving</i>) alat uji secara <i>real-time</i> .	POST /api/assets/movement, lims.asset_activity_logs	AssetManagementPage.jsx
	Activity History (Logs)	Audit trail lengkap pencatatan riwayat log aktivitas aset tertentu.	GET /api/asset-logs, lims.asset_activity_logs	AssetManagementPage.jsx
	Asset Labeling	Pembuatan label QR Code otomatis untuk memudahkan identifikasi fisik.	lims.testing equipments	AssetManagementPage.jsx (qrcode.react)
	Scan Integration	Pemindaian barcode/QR via kamera (web/mobile) untuk akses cepat ke data aset.	GET /api/assets, lims.testing equipments	AssetManagementPage.jsx (html5-qrcode)
7. Manajemen Testing Tools	Inventaris & Reservasi Alat	Pemantauan stok alat, jadwal kalibrasi, serta sistem pemesanan (<i>booking</i>) alat uji.	controllers/tool_controller.go, lims.testing_tools, lims.testing_tool_reservations, lims.testing_tool_availabilitys	ToolAvailabilityPage.jsx

Modul	Fitur	Fungsi & Kapabilitas	Backend (Golang)	Frontend (React)
	Monitor Pemakaian Alat	Melacak riwayat <i>check-in/check-out</i> serta intensitas penggunaan setiap alat saat pengujian.	controllers/tool_controller.go, lims.sting_tool_transactions	ToolAvailabilityPage.jsx
8. AI & Otomatisasi IoT	<i>Chatbot RAG Assistant</i>	Agen virtual cerdas untuk tanya-jawab referensi manual/SOP lab secara <i>omnichannel</i> .	controllers/sop_controller.go, controllers/ai_controller.go, services/rag_service.go, database/rag_schema.go, chat_sch.documents, chat_sch.document_chunks	AsistenLab.jsx
	↳ Document Ingestion (Pengolahan Dokumen)	Unggah & ekstraksi teks PDF SOP halaman-per-halaman, pemecahan teks (<i>chunking</i> sliding window) dan penyimpanan metadata.	controllers/sop_controller.go, services/rag_service.go (IngestPDF), chat_sch.documents	AsistenLab.jsx (upload UI)
	↳ Embedding & Vector Storage (Representasi Vektor)	Konversi potongan teks menjadi vektor 768-dimensi (nomic-embed-text) dan penyimpanan ke PostgreSQL pgvector.	services/rag_service.go (GetEmbedding), chat_sch.document_chunks	—
	↳ Retrieval & Prompt Engineering (Pencarian)	Pencarian kemiripan semantik (<i>cosine similarity</i>) terhadap vektor chunks dan perakitan <i>prompt</i> k	services/rag_service.go, chat_sch.document_chunks	AsistenLab.jsx

Modul	Fitur	Fungsi & Kapabilitas	Backend (Golang)	Frontend (React)
	n & Sintesis)	onteks untuk LLM.		
	↳ Generation (Pembuatan Jawaban)	Generasi jawaban natural berbasis konteks SOP oleh LLM lokal (Ollama) atau cloud (Groq), lengkap dengan sumber nama file & nomor halaman.	controllers/ai_controller.go, Ollama / Groq API	AsistenLab.jsx
	Rekomendasi Laporan (LLM)	Penghasil draf otomatis analisis kesimpulan dan rekomendasi perbaikan dari AI.	controllers/ai_controller.go, lims.testing_applications (analysis_conclusion)	AppDetail.jsx
	Predictive Quality Control (PQC)	Deteksi anomali statistik multivariate (<i>Isolation Forest</i>) secara <i>real-time</i> saat operator menginput skor.	Go native inference (onnxruntime_go), Python train.py, lims.ai_model_registry, lims.ai_anomaly_logs	UI form eksekusi (modal warning/supervisor override)
	↳ Retraining	Pelatihan ulang model PQC secara terjadwal menggunakan data historis terbaru untuk menjaga akurasi deteksi anomali.	Python train.py, lims.ai_model_registry	UI Maintenance Page
	↳ Deteksi Anomali Statistik (Outlier)	Identifikasi skor parameter yang menyimpang secara statistik dari distribusi normal data historis menggunakan	Go native inference (onnxruntime_go), lims.testing_pqc_ai_anomalies	UI form eksekusi (modal warning)

Modul	Fitur	Fungsi & Kapabilitas	Backend (Golang)	Frontend (React)
		model <i>Isolation Forest</i> .		
	↳ Validasi Batas Toleransi	Pengecekan otomatis apakah nilai skor yang diinput masih berada dalam rentang toleransi batas minimum–maksimum parameter pengujian.	services/scoring_service.go, lims.scoring_sub_aspects	AppDetail.jsx (Form Input)
	↳ Mekanisme Blokir & Override	Pemblokiran otomatis input skor anomali disertai notifikasi peringatan, dengan opsi <i>supervisor override</i> berbasis otorisasi peran.	controllers/ai_controller.go, lims.ai_anomaly_logs	UI form eksekusi (modal supervisor override)
	Integrasi Simulator IoT	Penerimaan data log otomatis secara <i>real-time</i> langsung dari perangkat uji fisik/eksternal.	controllers/machine_controller.go (webhook), flows.json (Node-RED HTTP & MQTT bridge), mecs.simulator_data_logs	hardware_simulator.html (statis/test, hosted di /simulator), test_http_scoring.ps1 (HTTP), test_mqtt_scoring.ps1 (MQTT), test_quick.ps1 (Quick)
9. Monitoring	<i>Dashboard & Tracking</i>	Pemantauan progres tahapan pengujian (aplikasi) dan SLA layanan harian.	NGINX Load Balancer, GoAccess WebSocket (port 7890)	WelcomePage.jsx, report.html (GoAccess static)
10. Reporting	Sertifikat Hasil Pengujian (SHP)	Penghasil (<i>generator</i>) laporan kelulusan dan laporan teknis akhir untuk	utils/print.js (printing helper)	AppDetail.jsx (trigger print SHP)

Modul	Fitur	Fungsi & Kapabilitas	Backend (Golang)	Frontend (React)
		dicetak/didistribusikan.		
	Laporan Summary Testing	Ringkasan statistik seluruh hasil pengujian per periode/status.	controllers/report_controller.go, lims.testing_applications	Report Page
	Laporan Detil Testing	Rincian lengkap parameter, skor, dan hasil uji per aplikasi.	controllers/report_controller.go, lims.testing_results, lims.testing_aspect_scores	Report Page
	Laporan Daftar Aset	Inventarisasi seluruh aset klien beserta status dan lokasinya.	controllers/report_controller.go, lims.testing equipments	Report Page
	Laporan Serah Terima	Berita acara serah terima aset kepada klien setelah pengujian selesai.	controllers/report_controller.go, lims.asset_handovers	Report Page
	Laporan Tagihan	Daftar seluruh invoice dan status pembayarannya per periode.	controllers/report_controller.go, lims.invoices	Report Page
	Laporan Pembayaran	Rekap transaksi pembayaran invoice yang telah diterima.	controllers/report_controller.go, lims.payments	Report Page
	Laporan SPD	Rekapitulasi Surat Perjalanan Dinas beserta status persetujuannya.	controllers/report_controller.go, lims.travel_requests	Report Page
	Laporan Cash Advance	Rincian pengajuan dan pencairan uang muka operasional dinas.	controllers/report_controller.go, lims.cash_advances	Report Page
	Laporan Reimbursement	Rekap klaim penggantian biaya	controllers/report_controller.go, lims.reimbursements	Report Page

Modul	Fitur	Fungsi & Kapabilitas	Backend (Golang)	Frontend (React)
		aktual beserta bukti struk/nota.		
11. Pemeliharaan Sistem	Manajemen Partisi Database	Penjadwalan <i>partitioning</i> tabel otomatis untuk menjaga performa data historis.	models/schema_manager.go	UI Maintenance Page
	Analisa <i>Bloat & Vacuum</i>	Fitur untuk mengidentifikasi fragmentasi data (<i>dead tuples</i>) dan melakukan <i>vacuuming</i> mandiri.	models/schema_manager.go	UI Maintenance Page
	Analisa Struktur & Partisi	Pemeriksaan kondisi struktur tabel dan status partisi aktif di database untuk verifikasi integritas.	models/schema_manager.go	UI Maintenance Page
	Backup & Restore Database	Pengamanan dan pemulihan data aplikasi secara <i>native</i> langsung dari UI.	models/schema_manager.go	UI Maintenance Page

1. Prasyarat Perangkat Lunak (Software Prerequisites)

Sebelum melakukan deployment LIMS, sistem server harus terpasang perangkat lunak berikut:

Komponen	Perangkat Lunak	Versi Minimal	Keterangan
Sistem Operasi	Linux (Ubuntu Server / Debian) atau WSL2	Ubuntu 20.04 LTS	Lingkungan dasar deployment
Bahasa Pemrograman	Go (Golang)	1.21+ (Rekomendasi 1.23)	Runtime backend
Runtime JS & PM2	Node.js & npm	Node 18+ & npm 9+	Menjalankan build frontend & PM2 manager

Komponen	Perangkat Lunak	Versi Minimal	Keterangan
Basis Data	PostgreSQL dengan pgvector	Postgres 14+	Penyimpanan utama & AI embedding
Workflow Engine	Camunda BPM Community Edition	Platform 7.x	Otomatisasi alur pendaftaran
Object Storage	MinIO Server	Latest Stable	Penyimpanan berkas digital
Utilitas Backup MinIO	MinIO Client (mc)	Latest Stable	Alat untuk backup sinkronisasi file (inkremental)
Container Engine	Docker & Docker Compose	v20+ / v2+	Menjalankan infrastruktur pendukung (Postgres, MinIO, Camunda)
Web Server	NGINX	1.18+	Reverse proxy & load balancing
OCR Engine	Python & PaddleOCR	Python 3.8+	Pengenalan dokumen/hasil uji
AI LLM Runner	Ollama	Latest Stable	Runner model AI lokal (jika tidak via Groq Cloud)
Broker Pesan (MQTT)	Mosquitto Broker	2.0+	Broker pesan untuk telemetri sensor IoT
Middleware Integrasi	Node-RED	3.0+	Penerjemah protokol & data pipeline peralatan
Utilitas PDF Parser	poppler-utils (pdftotext)	Latest Stable	Mengekstrak teks & halaman PDF untuk modul RAG Chatbot

Panduan Instalasi Paket Perangkat Lunak di VPS (Langkah-demi-Langkah)

Berikut adalah daftar perintah terpadu untuk memasang seluruh perangkat lunak prasyarat di server VPS Ubuntu Anda secara langsung:

1. Update Paket Sistem & Utilitas Dasar

sudo apt update && sudo apt upgrade -y

```
sudo apt install curl wget git tmux unzip build-essential -y
```

2. Pasang Runtime Go (Golang 1.23)

```
wget https://go.dev/dl/go1.23.0.linux-amd64.tar.gz
```

```
sudo rm -rf /usr/local/go && sudo tar -C /usr/local -xzf go1.23.0.linux-amd64.tar.gz
```

```
echo 'export PATH=$PATH:/usr/local/go/bin' >> ~/.profile
```

```
source ~/.profile
```

3. Pasang Node.js, NPM, & PM2 (Frontend & Service Manager)

```
curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -
```

```
sudo apt install -y nodejs
```

```
sudo npm install -g pm2 pm2-logrotate
```

4. Pasang PostgreSQL dengan Ekstensi pgvector

```
sudo apt install postgresql postgresql-contrib -y
```

```
sudo apt install postgresql-$(pgcode=$(psql --version | egrep -o '[0-9]+' | head -n1); echo $pgcode)-pgvector -y
```

5. Pasang Utilitas PDF Parser (Untuk RAG Chatbot)

```
sudo apt install poppler-utils -y
```

6. Pasang Docker & Docker Compose (Untuk MinIO & Camunda)

```
sudo apt install docker.io docker-compose -y
```

```
sudo systemctl enable docker
```

```
sudo systemctl start docker
```

```
sudo usermod -aG docker $USER
```

7. Pasang MinIO Client (mc) (Untuk Sinkronisasi Backup)

```
wget https://dl.min.io/client/mc/release/linux-amd64/mc
```

```
chmod +x mc
```

```
sudo mv mc /usr/local/bin/
```

8. Pasang Python Virtual Environment & Library AI Retraining

```
sudo apt install python3-venv python3-pip -y
```

```
python3 -m venv /home/lims/lims-ai-env
```

```
/home/lims/lims-ai-env/bin/pip install --upgrade pip
```

```
/home/lims/lims-ai-env/bin/pip install pandas numpy scikit-learn==1.4.1.post1
```

```
sqlalchemy skl2onnx onnx cryptography psycpg2-binary ml_dtypes
```

7. Pasang MinIO (Object Storage)

```
wget https://dl.min.io/server/minio/release/linux-amd64/minio
```

```
chmod +x minio
```

```
sudo mv minio /usr/local/bin/
```

8. Pasang Nginx Web Server & GoAccess (Monitoring)

```
sudo apt install nginx -y
```

```
wget -O - https://deb.goaccess.io/gnupg.key | gpg --dearmor | sudo tee
```

```
/usr/share/keyrings/goaccess.gpg >/dev/null
```

```
echo "deb [signed-by=/usr/share/keyrings/goaccess.gpg] https://deb.goaccess.io/"
```

```
$(lsb_release -cs) main" | sudo tee /etc/apt/sources.list.d/goaccess.list
```

```
sudo apt update
```

```
sudo apt install goaccess -y
```

9. Pasang MQTT Broker (Mosquitto) & Node-RED

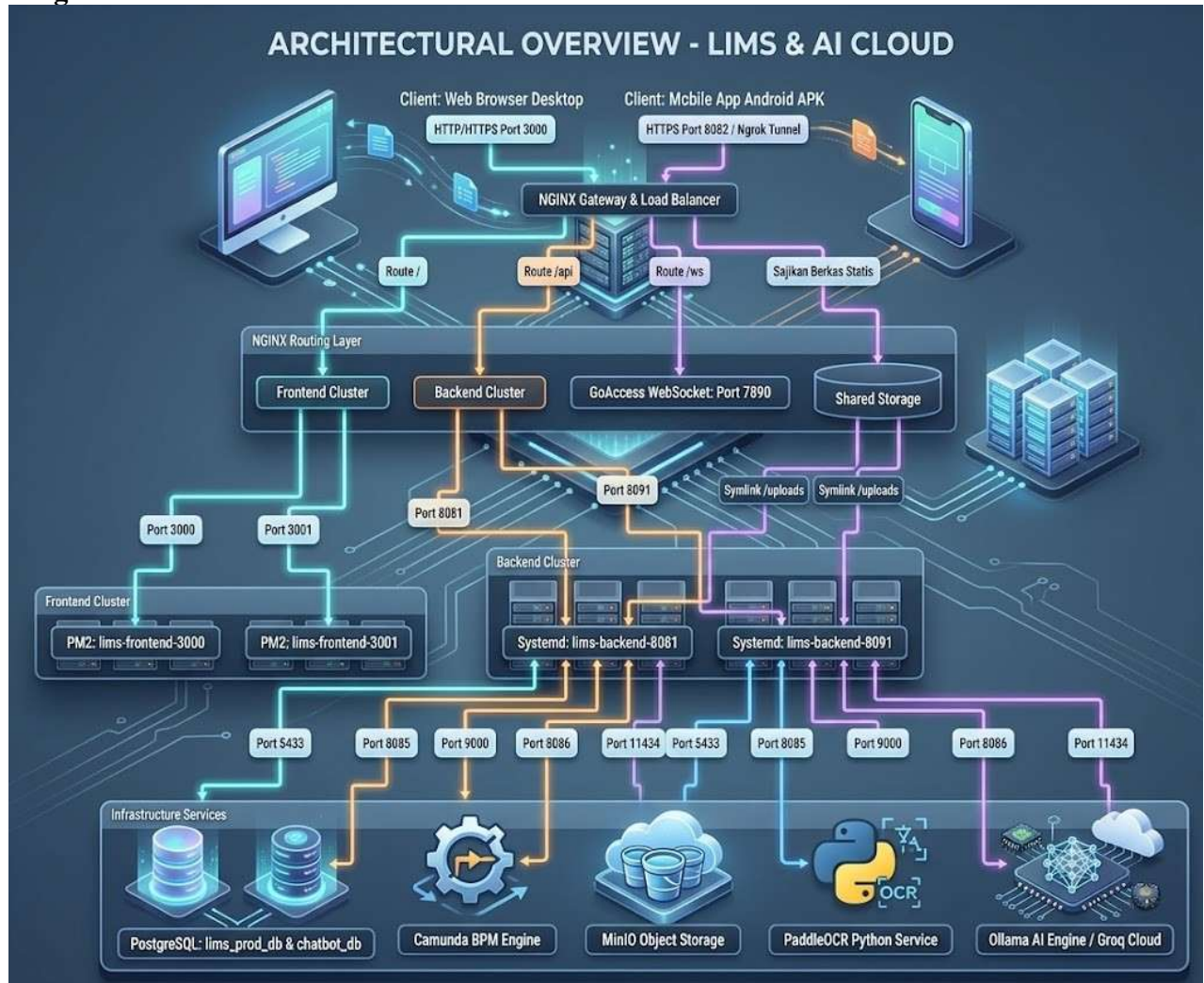
```
sudo apt install mosquitto mosquitto-clients -y
```

```
sudo npm install -g --unsafe-perm node-red
```

2. Arsitektur Sistem & Alur Akses Pengguna (System Architecture & Access Flow)

LIMS diakses melalui dua tipe klien utama: **Web Application (Desktop/Admin)** dan **Mobile Application (Android APK via Capacitor)**. Semua permintaan dirutekan melalui NGINX Load Balancer.

Diagram Arsitektur Sistem & Alur Akses



Alur Akses Detil Pengguna

1. Akses via Web (Desktop) - Bebas Port:

- Pengguna mengakses menggunakan domain standar tanpa port: `https://lims.local` (HTTPS Port 443) atau `http://lims.local` (HTTP Port 80).
- NGINX bertindak sebagai gerbang tunggal yang mendengarkan Port 80/443, mengeliminasi kebutuhan pengguna mengetik port di browser.
- NGINX merutekan permintaan frontend ke port internal 3000 & 3001 (PM2) dan permintaan API /api ke port internal 8081 & 8091 (Go Backend).

2. Akses via Mobile App (Android APK):

- **Mode Produksi:** Aplikasi Android memanggil API LIMS secara langsung melalui URL DNS Publik resmi (Port 443) dengan sertifikat SSL terpercaya (misal Let's Encrypt).
- **Mode Pengembangan (Development Only):** URL API diarahkan menggunakan bantuan terowongan aman **Ngrok** (https://*.ngrok-free.dev) untuk menjembatani server lokal dengan perangkat ponsel uji secara instan.

3. Penyajian Laporan & Analitik (GoAccess):

- Akses /report.html untuk memantau log lalu lintas secara real-time. NGINX menyajikan berkas HTML ini secara statis dan membuka koneksi WebSocket di /ws ke server GoAccess (port 7890).

3. Struktur Direktori Proyek (Project Directory Structure)

Proyek LIMS terbagi menjadi dua bagian besar: backend (Go) dan frontend (React + Vite + Capacitor).

A. Struktur Direktori Backend (Go)

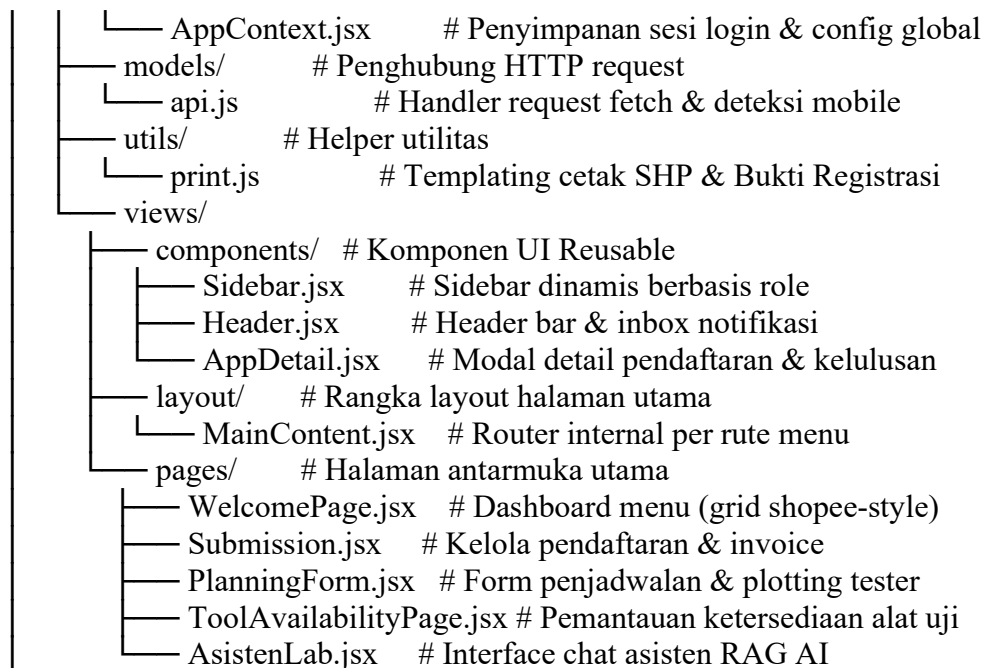
```

backend/
├── controllers/           # Logika pengendali HTTP REST API
│   ├── ai_controller.go   # Penanganan RAG & AI report generator
│   ├── application_controller.go # Alur utama transaksi pengujian
│   ├── auth_controller.go # Otentikasi sesi & pengambilan menu
│   ├── billing_controller.go # Manajemen invoice & pembayaran
│   ├── asset_controller.go # Pelacakan pergerakan aset & handover
│   └── tool_controller.go  # Pemakaian alat uji & reservasi
├── database/             # Inisialisasi pool koneksi database
│   └── db.go              # Konfigurasi GORM ke Postgres & Chatbot DB
├── middleware/           # Interseptor HTTP Request
│   ├── auth.go            # Validasi otentikasi JWT Token
│   ├── rate_limit.go      # Pembatasan laju request per menit
│   └── role_check.go       # Pemeriksaan hak akses menu peran
├── models/               # Definisi skema tabel GORM & logic data
│   ├── models.go          # Struct utama (User, Application, dll.)
│   └── schema_manager.go   # Manajemen partisi tabel dinamis
├── routes/               # Pemetaan endpoint URI ke controller
│   └── routes.go           # Konfigurasi rute Gin Router
└── services/             # Integrasi layanan pihak ketiga
    ├── camunda_service.go  # Pengiriman sinyal ke Camunda BPM
    ├── minio_service.go    # Operasi berkas ke Object Storage MinIO
    ├── scoring_service.go   # Algoritma perhitungan skor aspek uji
    └── email_service.go     # Pengiriman email notifikasi otomatis
  
```

B. Struktur Direktori Frontend (React)

```

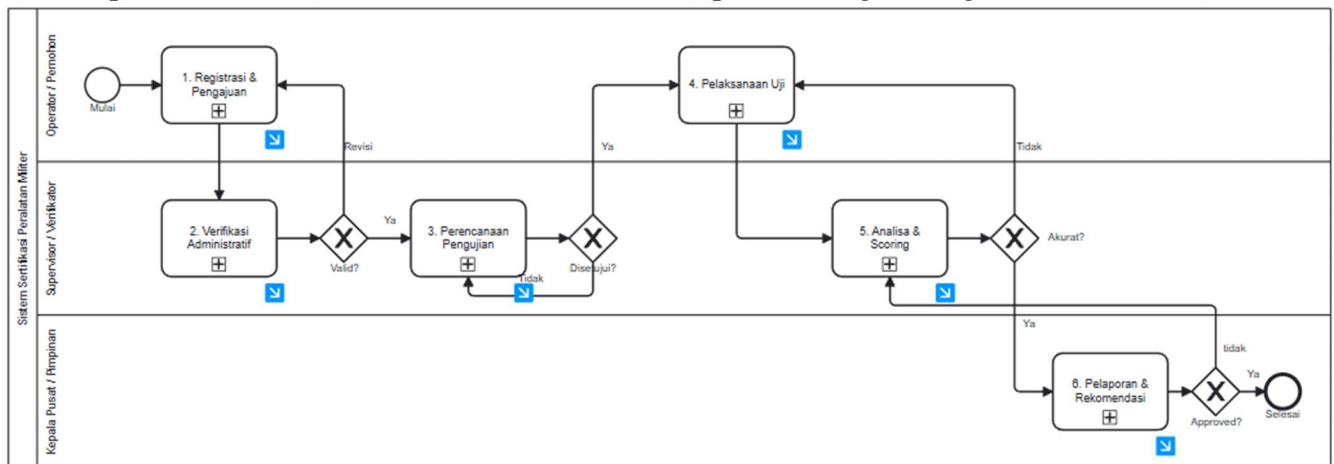
frontend/
├── src/
│   ├── constants/        # Konfigurasi global & pemetaan rute
│   │   └── routes.jsx     # Rute workflow & pemetaan label tombol aksi
│   └── context/           # State management React
  
```



4. Alur Kerja & Fragmentasi Data (Workflow & Data Partitioning)

A. Tahapan Alur Kerja Detil LIMS

Setiap pengajuan pengujian terikat pada daur hidup (*workflow*) yang dikendalikan oleh instansi proses Camunda BPM. Berikut detail aktivitas pada masing-masing status:



1. REGISTERED (Registrasi & Pengajuan):

- o Pemohon menginput data peralatan, data pemohon, dan mengunggah dokumen administrasi ke MinIO.
- o Sistem secara otomatis menggenerasi Nomor Registrasi unik (format: YYYY-000XX) dan data Invoice (status UNPAID) jika pemohon memilih paket pengujian.

2. VERIFIED (Verifikasi Administratif):

- o Petugas memeriksa kesesuaian berkas. Jika ada kekurangan dokumen, status diubah menjadi **REVISI** agar pemohon mengunggah ulang. Jika lengkap, status naik menjadi VERIFIED.

3. APPROVED (Persetujuan Pimpinan):

- Kepala Lab memvalidasi pengajuan. Jika disetujui, workflow melangkah ke perencanaan teknis.
- 4. **PLANNED (Perencanaan Pengujian):**
 - Petugas memetakan parameter pengujian dinamis (menurut paket/metodologi), penjadwalkan tanggal uji lab/lapangan, dan menunjuk tim penguji (tester_applications).
- 5. **EXECUTED (Pelaksanaan Pengujian):**
 - Tim penguji mengisi hasil pengujian fisik parameter uji (skor 0-100) ke dalam tabel testing_results. Data juga dapat masuk secara otomatis dari log IoT/Simulator.
- 6. **ANALYZED (Pengolahan & Analisa Data):**
 - Sistem menghitung persentase bobot aspek secara otomatis untuk memperoleh skor akhir.
 - Analis dapat memicu AI untuk menghasilkan rekomendasi dan draf kesimpulan evaluasi.
- 7. **CERTIFIED / FINALIZED (Pelaporan & Rekomendasi):**
 - Penerbitan Sertifikat Hasil Pengujian (SHP) dengan nomor seri sertifikat otomatis (certificate_num) dan masa berlaku (expiry_date), serta pencetakan Laporan Teknis.
- 8. **CLOSED (Arsip Pengujian):**
 - Daur hidup transaksi selesai, data aplikasi dikunci untuk kepentingan audit.

B. Fragmentasi & Partisi Database (Database Partitioning)

Untuk mencegah penurunan performa kueri akibat volume transaksi yang besar dari waktu ke waktu, tabel testing_applications dan testing_results di-partisi secara bulanan di PostgreSQL.

Partisi Bulanan Juni 2026

Partisi Bulanan Juli 2026

Fallback Default

Tabel Induk: lims.testing_applications

lims.testing_applications_202606

lims.testing_applications_202607

lims.testing_applications_default

- **Mekanisme Partisi Dinamis:**
 - Partisi dibuat otomatis berdasarkan kolom tanggal pembuatan data (created_at).
 - Parameter DATA_RETENTION_MONTHS di berkas .env (default: 3) menentukan rentang bulan aktif yang dimuat sistem untuk mempercepat pembacaan data.
 - Skema partisi diatur langsung di tingkat database menggunakan fitur PostgreSQL Native Partitioning:
 - CREATE TABLE lims.testing_applications (
 - id BIGINT NOT NULL,
 - created_at TIMESTAMP NOT NULL,
 - ...

-) PARTITION BY RANGE (created_at);
- Untuk setiap bulan baru, database membuat tabel partisi turunan seperti lims.testing_applications_YYYYMM secara dinamis.

5. Modul Aplikasi, API, dan Basis Data (Application Modules, APIs, & Database Schema)

Berikut adalah detail teknis dari masing-masing modul utama yang digunakan di LIMS. Secara umum, berbagai modul bergantung pada tabel lims.global_parameters untuk mengambil konfigurasi dinamis yang memengaruhi alur kerja, di antaranya:

- **Modul Registrasi & Sistem:** Menggunakan parameter global (misal: APP_VERSION, kode integrasi eksternal) untuk identitas aplikasi.
- **Modul Penilaian & Scoring Engine:** Membaca parameter SCORE_THRESHOLD_PASS dan SCORE_THRESHOLD_NOTE sebagai ambang batas absolut penentuan kelulusan akhir pengujian.
- **Modul Database & Arsip:** Menggunakan konfigurasi seperti DATA_RETENTION_MONTHS untuk menentukan kapan data dipartisi atau diarsipkan.
- **Modul AI & Chatbot:** Membaca kredensial rahasia (API Keys) atau pengaturan LLM model dari tabel parameter global.

1. Modul Otentikasi & RBAC (Auth & Access Control)

- **Deskripsi:** Penanganan otentikasi login pengguna, pencatatan sesi aktif, manajemen status keaktifan user, logout brute-force, pembatasan single-session, OTP takeover, serta pembuatan struktur menu sampingan dinamis.
- **API Endpoints:**
 - POST /api/login - Otentikasi kredensial pengguna, pengecekan brute-force, status aktif, single-session, dan pemicu OTP pengambilalihan.
 - POST /api/verify-otp - Verifikasi kode OTP 6-digit (via Telegram/WhatsApp) untuk takeover sesi.
 - GET /api/verify-session - Memvalidasi token sesi HttpOnly cookie dan sidik jari klien secara berkala/saat bootstrap.
 - GET /api/menus - Mengambil menu sidebar aktif sesuai role_id pengguna.
 - POST /api/logout - Menghapus sesi aktif dari database.
- **Tabel**

Terkait: lims.users, lims.hist_users, lims.roles, lims.menus, lims.role_menus, lims.user_sessions, lims.otp_codes.

Mekanisme Pengamanan Sesi & Hardening LIMS

Untuk mencegah penyalahgunaan sesi dan mengamankan LIMS dari ancaman luar (seperti token sharing, hijack via Postman, atau brute-force tebak password), sistem mengimplementasikan mekanisme berikut:

1. **Secure HttpOnly Cookie, Otentikasi Otomatis, & Anti-XSS:**
 - **Penyimpanan Ganda (Hybrid Storage):** Secara arsitektural, API backend secara otomatis mengatur Cookie browser dengan flag HttpOnly, Secure (untuk HTTPS), dan SameSite=Lax sebagai lapis perlindungan utama agar token terisolasi dari peretasan XSS. **Namun demikian**, pada antarmuka web LIMS, token juga secara eksplisit disalin oleh React ke dalam localStorage (sebagai auth_token). Keputusan desain

ini (API-First) sengaja diambil untuk memastikan **Kompatibilitas Multi-Platform**: Kode web LIMS siap dikompilasi menjadi aplikasi *Mobile* (Capacitor) atau *Desktop* (Electron) yang kerap memiliki keterbatasan sistem *Cookie*. Frontend bisa dengan fleksibel membaca token dari *localStorage* dan menyisipkannya ke *HTTP Header* standar (*Authorization: Bearer <TOKEN>*), sekaligus memudahkan manajemen *state* status pengguna (seperti mendeteksi *is_logged_in* tanpa perlu selalu bertanya ke backend).

- **Otentikasi Otomatis (Akses Web):** Browser memiliki mekanisme internal untuk selalu menyertakan (*attach*) cookie *HttpOnly* ini secara otomatis setiap kali antarmuka web LIMS (yang dikonfigurasi dengan *credentials: 'include'*) mengirimkan request ke backend API. Oleh karena itu, pengguna tetap masuk (*logged in*) dan tidak perlu menginputkan username dan password berulang kali selama sesi token belum kedaluwarsa.
- **Integrasi Pihak Ketiga (M2M / Mobile):** Meskipun pengguna web dilindungi penuh oleh cookie, backend LIMS tetap menerapkan arsitektur otentikasi *hybrid*. Saat login berhasil, backend juga memberikan string token di dalam struktur *response payload JSON*. Sistem pihak ketiga (seperti aplikasi Android/iOS, integrasi ERP eksternal, atau Postman) dapat membaca token tersebut, menyimpannya pada memori internal mereka (seperti Android *Keystore*), dan mengakses API selanjutnya dengan menyisipkan token pada *HTTP Header* standar, yaitu *Authorization: Bearer <TOKEN>*. Backend didesain untuk memprioritaskan pengecekan token dari *header Authorization* terlebih dahulu, sebelum *fallback* memeriksa *Cookie*.

2. Sidik Jari Klien (Client Fingerprinting - IP & User-Agent Binding):

- Setiap kali user membuat sesi baru, database mencatat *ip_address* dan *user_agent* perangkat.
- Pada middleware otentikasi, setiap request akan diverifikasi ulang kesesuaian IP & User-Agent-nya. Jika terdeteksi ketidakcocokan (misalnya token *di-copy-paste* ke Postman di komputer lain atau perangkat berbeda), middleware otomatis **menghapus sesi tersebut secara permanen dari database** dan mengembalikan status **401 Unauthorized** (pesan: "Akses ditolak. Deteksi perubahan perangkat atau IP ilegal. Sesi Anda telah ditutup.").
- **Catatan Penting:** Proteksi ini bersifat mutlak dan tidak bergantung pada konfigurasi *SINGLE_SESSION_MODE*. Meskipun *SINGLE_SESSION_MODE* bernilai *false*, upaya pencurian/penggunaan token di lingkungan yang berbeda akan tetap ditolak dan sesi korban akan otomatis diputuskan demi keamanan.

3. Batas Waktu Idle Dinamis (Dynamic Idle Timeout):

- Sesi akan kedaluwarsa secara otomatis jika pengguna tidak aktif (tidak ada aktivitas terdeteksi) selama kurun waktu tertentu.
- Waktu kedaluwarsa bersifat dinamis per-user (kolom *idle_timeout_minutes* di *lims.users*), dengan *fallback* ke nilai

parameter global DEFAULT_IDLE_TIMEOUT_MINUTES (default 30 menit).

- Database diperbarui secara cerdas (*throttling*) hanya jika selisih waktu aktivitas terakhir (last_activity_at) lebih dari 1 menit untuk menghindari overloading database query.

4. **Brute Force Lockout:**

- Percobaan login gagal berturut-turut dibatasi maksimal sesuai parameter LOGIN_MAX_ATTEMPTS (default 5 kali) dalam rentang waktu LOGIN_LOCKOUT_MINUTES (default 15 menit).
- Jika batas terlampaui, akun pengguna otomatis terkunci sementara, mencegah serangan bot tebak password.

5. **Kebijakan Kata Sandi Kuat (Strong Password Policy):**

- LIMS mengadopsi standar keamanan yang sangat ketat untuk kata sandi, yang dieksekusi baik di sisi antarmuka maupun validasi backend.
- Kata sandi **wajib** memiliki **minimal 9 karakter**.
- Kata sandi **wajib** mengandung setidaknya kombinasi dari: **1 huruf kapital (Uppercase), 1 huruf kecil (Lowercase), dan 1 karakter spesial/symbol (@, #, \$, dsb)**.
- Jika kriteria ini tidak terpenuhi saat pembuatan atau penggantian sandi, sistem akan menolak permintaan tersebut secara langsung.

6. **Kendali Single Session & Takeover dengan OTP:**

- LIMS mendukung pembatasan login tunggal per akun via parameter global SINGLE_SESSION_MODE (true/false).
- Parameter ini khusus mengatur apakah pengguna diizinkan melakukan **login resmi berulang** untuk menghasilkan sesi-sesi baru yang berbeda (misal: login di HP dan Laptop secara sah bersamaan).
- Jika bernilai true dan pengguna mencoba login baru ketika sesi sebelumnya di perangkat lain masih aktif, sistem akan menolak login tersebut dengan status 409 Conflict (pesan: "Session Anda masih aktif").
- Pengguna diberikan opsi untuk melakukan pengambilalihan sesi (**Session Takeover**). Jika dipilih, sistem akan mengirimkan kode OTP 6-digit ke WhatsApp/Telegram terdaftar dan menyimpan data sementara ke tabel lims.otp_codes. Pengguna wajib melakukan verifikasi OTP di endpoint /api/verify-otp untuk menghancurkan sesi lama dan mengaktifkan sesi baru.

7. **Skrip Migrasi Basis Data (Database Migration SQL):**

- Berikut adalah skrip DDL/DML lengkap yang harus dijalankan secara manual pada PostgreSQL (dengan skema lims) untuk mendukung seluruh fitur hardening sesi di atas:

8. -- 1. Tambah kolom user_agent dan last_activity_at di tabel lims.user_sessions

9. ALTER TABLE lims.user_sessions ADD COLUMN IF NOT EXISTS
user_agent VARCHAR(255) DEFAULT NULL;

10. ALTER TABLE lims.user_sessions ADD COLUMN IF NOT EXISTS
last_activity_at TIMESTAMP WITHOUT TIME ZONE DEFAULT NOW();

11.

12. -- 2. Tambah kolom *is_active* dan *idle_timeout_minutes* di tabel *lims.users* & *lims.hist_users*
13. ALTER TABLE *lims.users* ADD COLUMN IF NOT EXISTS *is_active* BOOLEAN NOT NULL DEFAULT TRUE;
14. ALTER TABLE *lims.users* ADD COLUMN IF NOT EXISTS *idle_timeout_minutes* INT DEFAULT NULL;
- 15.
16. ALTER TABLE *lims.hist_users* ADD COLUMN IF NOT EXISTS *is_active* BOOLEAN NOT NULL DEFAULT TRUE;
17. ALTER TABLE *lims.hist_users* ADD COLUMN IF NOT EXISTS *idle_timeout_minutes* INT DEFAULT NULL;
- 18.
19. -- 3. Membuat tabel penyimpanan OTP sementara di schema *lims*
20. CREATE TABLE IF NOT EXISTS *lims.otp_codes* (
21. *id* SERIAL PRIMARY KEY,
22. *user_id* INT NOT NULL REFERENCES *lims.users*(*id*) ON DELETE CASCADE,
23. *code* VARCHAR(10) NOT NULL,
24. *expires_at* TIMESTAMP WITHOUT TIME ZONE NOT NULL,
25. *created_at* TIMESTAMP WITHOUT TIME ZONE DEFAULT NOW()
26.);
- 27.
28. -- Index untuk mempercepat query pencocokan OTP
29. CREATE INDEX IF NOT EXISTS *idx_otp_codes_user_code* ON *lims.otp_codes*(*user_id*, *code*);
- 30.
31. -- 4. Pengisian Parameter Global (DML) ke tabel *lims.global_parameters*
32. -- A. Parameter untuk mengaktifkan Single-Session (*true/false*)
33. INSERT INTO *lims.global_parameters* (*param_key*, *param_value*, *description*, *created_at*, *updated_at*, *created_user*, *updated_user*)
34. VALUES (
35. 'SINGLE_SESSION_MODE',
36. 'false',
37. 'Menentukan apakah user hanya boleh memiliki satu sesi aktif (*true*) atau multi sesi (*false*).',
38. NOW(), NOW(), 'SYSTEM', 'SYSTEM'
39.) ON CONFLICT (*param_key*) DO UPDATE SET *description* = EXCLUDED.*description*;
- 40.
41. -- B. Parameter untuk mengizinkan pengambilalihan sesi/*force login* (*true/false*)
42. INSERT INTO *lims.global_parameters* (*param_key*, *param_value*, *description*, *created_at*, *updated_at*, *created_user*, *updated_user*)
43. VALUES (
44. 'ALLOW_SESSION_TAKEOVER',
45. 'true',

```

46. 'Mengizinkan user mengambil alih sesi aktif lainnya saat login kembali jika
    bernilai true.',
47. NOW(), NOW(), 'SYSTEM', 'SYSTEM'
48. ) ON CONFLICT (param_key) DO UPDATE SET description =
    EXCLUDED.description;
49.
50. -- C. Parameter untuk waktu idle global default (dalam menit)
51. INSERT INTO lims.global_parameters (param_key, param_value, description,
    created_at, updated_at, created_user, updated_user)
52. VALUES (
53. 'DEFAULT_IDLE_TIMEOUT_MINUTES',
54. '30',
55. 'Batas waktu idle sistem default dalam menit sebelum sesi ditutup otomatis.',
56. NOW(), NOW(), 'SYSTEM', 'SYSTEM'
57. ) ON CONFLICT (param_key) DO UPDATE SET description =
    EXCLUDED.description;
58.
59. -- D. Parameter Batas Maksimal Percobaan Login Salah
60. INSERT INTO lims.global_parameters (param_key, param_value, description,
    created_at, updated_at, created_user, updated_user)
61. VALUES (
62. 'LOGIN_MAX_ATTEMPTS',
63. '5',
64. 'Jumlah maksimal percobaan login salah sebelum akun dikunci sementara.',
65. NOW(), NOW(), 'SYSTEM', 'SYSTEM'
66. ) ON CONFLICT (param_key) DO UPDATE SET description =
    EXCLUDED.description;
67.
68. -- E. Parameter Durasi Waktu Akun Dikunci (Menit)
69. INSERT INTO lims.global_parameters (param_key, param_value, description,
    created_at, updated_at, created_user, updated_user)
70. VALUES (
71. 'LOGIN_LOCKOUT_MINUTES',
72. '15',
73. 'Durasi waktu (dalam menit) akun dikunci akibat salah password berturut-
    turut.',
74. NOW(), NOW(), 'SYSTEM', 'SYSTEM'
75. ) ON CONFLICT (param_key) DO UPDATE SET description =
    EXCLUDED.description;

```

2. Modul Registrasi & Pengajuan (Submission)

- **Deskripsi:** Manajemen dokumen persyaratan digital dan pendataan awal registrasi pengajuan pengujian. **Di tahap registrasi ini, sistem juga melakukan pendaftaran Modul Aset (testing equipments), di mana barang fisik klien yang akan diuji didata secara spesifik. Selain itu, begitu registrasi disubmit, LIMS secara otomatis membuat tagihan awal (Invoice Billing) untuk pemohon.**

- **API Endpoints:**
 - GET /api/applications - Mengambil daftar aplikasi dalam periode aktif.
 - POST /api/applications - Membuat data pengajuan baru (memicu Camunda BPM secara asinkron, merekam aset, dan membuat *invoice*).
 - PUT /api/applications/:id - Mengubah data pengisian pengajuan.
- **Tabel**
Terkait: lims.testing_applications, lims.testing equipments, lims.partners, lims.invoices.

3. Modul Perencanaan & Pelaksanaan Uji

- **Deskripsi:** Penjadwalan tanggal uji lapangan/lab, penugasan tim penguji, dan perekaman nilai hasil pengujian parameter. **Kalkulasi nilai dan penentuan kelulusan pada tahap pelaksanaan sepenuhnya dikendalikan oleh *Hierarki Scoring Engine* LIMS (yang terdiri dari Tipe Pengujian > Metodologi > Grup Level > Aspek > Sub-Aspek > Level Kelulusan).**
- **API Endpoints:**
 - POST /api/applications/:id/plan - Menyimpan plot jadwal dan tester.
 - PUT /api/applications/:id/execute - Menyimpan nilai aktual hasil uji per sub-aspek.
- **Tabel**
Terkait: lims.testing_plans, lims.testers_applications, lims.master_testers, lims.testing_results.

4. Modul Analisa Data & Scoring (Scoring Engine)

Mesin penilaian LIMS beroperasi melalui hierarki terstruktur enam level yang terintegrasi secara kumulatif, didukung oleh overlay analisis AI cerdas.

A. Hierarki 6 Level Scoring Engine

Mempengaruhi Hasil Akhir

Level 1: testing_types - Tipe Pengujian

Level 2: methodologies - Metodologi Uji

Level 3: scoring_aspects - Aspek Penilaian

Level 4: scoring_sub_aspects - Parameter Uji

Level 5: scoring_levels - Label Kelulusan Akhir

Level 6: level_groups - Grup Level Kelulusan

1. **Level 1: testing_types (Tipe Pengujian):**
 - Klasifikasi tertinggi dari pengujian (seperti *Uji Laboratorium*, *Uji Lapangan*, atau *Uji Umum*).
2. **Level 2: methodologies (Metodologi Uji):**
 - Prosedur pengujian spesifik yang terikat pada tipe pengujian (via test_type_code).
3. **Level 3: scoring_aspects (Aspek Penilaian):**
 - Kelompok Aspek penilaian utama (misal: *Aspek Manusia*, *Aspek Alat*, *Aspek Lingkungan*) yang berada di bawah metodologi uji dengan bobot persentase tertentu.
4. **Level 4: scoring_sub_aspects (Sub-Aspek / Parameter Uji):**
 - Parameter pengujian fisik terperinci yang dinilai di lapangan/laboratorium yang memiliki bobot parameter internal dalam masing-masing aspeknya.
5. **Level 5: scoring_levels (Level Penilaian Kelulusan):**

- Rubrik atau kategori label hasil akhir pengujian (seperti *Lulus Tinggi, Lulus Sedang, Tidak Lulus*). Level ini berada di bawah payung grup level (level_groups), dan sistem akan mengklasifikasikan pengujian berdasarkan rentang nilai kelulusan (min_score - max_score) di tabel ini.
6. **Level 6: level_groups (Klasifikasi Grup Kelulusan):**
- Tabel ini menampung payung rubrik klasifikasi kelulusan (misalnya: Grup Kelulusan "Sertifikasi Standard SNI" vs "Kalibrasi Dasar"). Metodologi menunjuk ke tabel ini melalui kolom level_group_code.

B. Informasi Tambahan: Lapisan Analisis AI (AI Overlay Engine)

AI bertindak sebagai **lapisan analisis cerdas (overlay)** yang memantau performa kelima level penilaian di atas secara real-time:

- Jika terdapat parameter uji (scoring_sub_aspects) yang gagal memenuhi nilai batas standar atau mengalami deviasi signifikan:
 1. AI mengidentifikasi parameter gagal tersebut secara kuantitatif.
 2. AI melakukan kueri pencarian semantik (Vector Search) menggunakan ekstensi pgvector ke tabel lims.document_chunks untuk menemukan SOP pemecahan masalah (*troubleshooting*) dan panduan teknis yang relevan dari dokumen panduan lab yang diunggah.
 3. AI merumuskan penjelasan analisis naratif yang komprehensif serta rekomendasi teknis perbaikan. Output ini disimpan di kolom analysis_conclusion pada tabel lims.testing_applications dan dicetak langsung pada sertifikat Laporan Teknis SHP.

C. Tabel Basis Data yang Digunakan dalam Penilaian

Perhitungan skor LIMS melintasi beberapa tabel utama berikut:

1. **lims.testing_results:** Menyimpan nilai mentah aktual (actual_value) dan skor terhitung (score) per parameter uji / sub-aspek (Level 4).
2. **lims.testing_aspect_scores:** Menyimpan rata-rata terhitung aspek (score) hasil kalkulasi kumulatif sub-aspek (Level 3).
3. **lims.testing_applications:** Menyimpan skor akhir gabungan (final_score) dan status keputusan kelayakan (final_status) (Level 1, 2, dan 5).
4. **lims.global_parameters:** Menyimpan ambang batas nilai kelulusan sistem (SCORE_THRESHOLD_PASS & SCORE_THRESHOLD_NOTE).

D. Rumus Perhitungan Penilaian

1. **Nilai Aspek ((Score_{Aspect})):** Diambil dari data tabel lims.testing_results yang disaring berdasarkan parameter aktif di bawah aspek terkait:
$$\left[\text{Score}_{\{Aspect\}} = \frac{\sum_{i=1}^n (\text{Score}_{\{SubAspect, i\}} \times \text{Weight}_{\{SubAspect, i\}})}{\sum_{i=1}^n \text{Weight}_{\{SubAspect, i\}}} \right]$$

Catatan: Jika suatu sub-aspek dinonaktifkan (is_disabled = true / disembunyikan lewat konfigurasi paket), maka nilai serta bobotnya dikeluarkan dari rumus perhitungan di atas. Hasil perhitungan ini disimpan di lims.testing_aspect_scores.
2. **Nilai Akhir Gabungan ((Score_{Final})):** Nilai akhir gabungan merupakan kalkulasi terbobot dari seluruh aspek penilaian aktif di lims.testing_aspect_scores:
$$\left[\text{Score}_{\{Final\}} = \sum_{j=1}^m (\text{Score}_{\{Aspect, j\}} \times \text{Weight}_{\{Aspect, j\}}) \right]$$

j})] Hasil akhir ini disimpan di kolom `final_score` pada tabel `lims.testing_applications`.

3. **Status Keputusan Hasil Uji:** Sistem mencocokkan `final_score` dengan `threshold_kelulusan` di `lims.global_parameters`:

- **SCORE_THRESHOLD_PASS** (Default: 75) - Ambang batas kelulusan standar.
- **SCORE_THRESHOLD_NOTE** (Default: 60) - Ambang batas minimal kelayakan bersyarat.

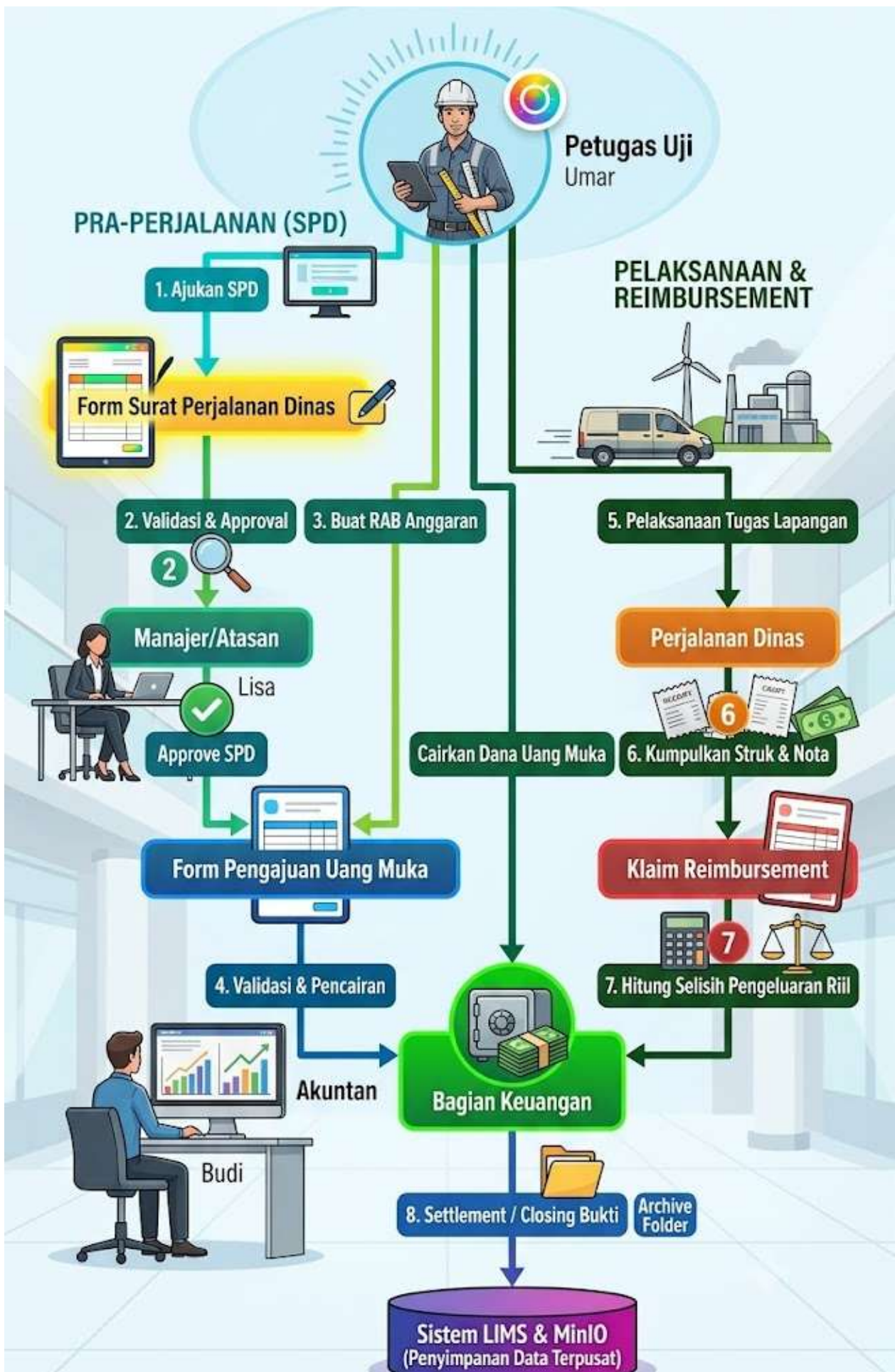
Klasifikasi akhir disimpan di kolom `final_status` pada tabel `lims.testing_applications`:

- **TIDAK LULUS** : Jika nilai akhir gabungan berada di bawah batas minimum ((≤ 75)).

5. Modul Keuangan & Perjalanan (Finance & Travel)

- **Deskripsi:** Modul sentral untuk manajemen kegiatan non-teknis tim penguji yang difokuskan pada penagihan, penganggaran, administrasi finansial lapangan (*cashless & paperless*), serta pencatatan bukti pembayaran. Modul ini terintegrasi erat dengan infrastruktur MinIO untuk penyimpanan bukti struk/nota digital.
- **Detail Proses Keuangan & Perjalanan:**
 1. **Penagihan & Pembayaran:** Pembuatan invoice otomatis berdasarkan tarif paket yang dipilih pemohon saat registrasi. Sistem mencatat pembayaran invoice dan memvalidasinya.
 2. **Pengajuan SPD (Surat Perjalanan Dinas):** Petugas/Tester mengajukan rencana perjalanan terkait suatu pelaksanaan uji lapangan (*Testing Plan*), yang akan divalidasi oleh atasan. Berikut adalah urutan status pada pengajuan SPD (**Tabel: `lims.travel_requests`**):
 - **DRAFT:** Pengajuan baru dibuat oleh pengguna, namun belum disubmit ke sistem (masih bisa diedit secara bebas).
 - **PENDING:** Pengajuan telah disubmit dan masuk ke antrean, menunggu persetujuan (*approval*) dari supervisor atau admin terkait.
 - **APPROVED:** Pengajuan telah disetujui oleh supervisor/admin. Status ini mensahkan perjalanan dinas dan memungkinkan untuk dilanjutkan ke proses pencairan dana (*Cash Advance*).
 - **REJECTED:** Pengajuan ditolak oleh supervisor/admin (umumnya disertai catatan mengenai alasan penolakan).
 - **CANCELED:** Pengajuan dibatalkan (dapat dilakukan oleh pemohon jika terjadi kesalahan sebelum diproses, atau dibatalkan oleh supervisor/admin).
 3. **Pengajuan Cash Advance (Uang Muka):** Berdasarkan SPD yang disetujui, petugas mengajukan pencairan uang muka (untuk tiket, akomodasi, biaya operasional). Bagian Keuangan melakukan verifikasi dan transfer pencairan. Berikut adalah urutan status pada pengajuan Cash Advance (**Tabel: `lims.cash_advances`**):
 - **PENDING:** Pengajuan disubmit dan menunggu persetujuan (*approval*) dari supervisor/admin.

- **APPROVED:** Pengajuan disetujui oleh supervisor/admin, siap untuk diproses pencairannya.
 - **TRANSFERRED:** Dana uang muka telah ditransfer oleh bagian Keuangan kepada pemohon.
 - **REJECTED:** Pengajuan ditolak oleh supervisor/admin.
 - **SETTLED:** Uang muka telah dipertanggungjawabkan atau diselesaikan (biasanya otomatis berstatus *Settled* setelah pengajuan *Reimbursement* terkait telah disetujui/dibayarkan).
4. **Klaim Reimbursement:** Pasca perjalanan, petugas melaporkan seluruh pengeluaran riil (*actual expense*) dan wajib mengunggah nota/struk bukti transaksi. Jika total pengeluaran lebih kecil dari uang muka, sistem akan mencatat kewajiban pengembalian selisih dana (*settlement*). Berikut adalah urutan status pada Reimbursement (**Tabel: lims.reimbursements**):
- **PENDING:** Klaim disubmit dan menunggu persetujuan (*approval*).
 - **APPROVED:** Klaim disetujui oleh supervisor/admin.
 - **PAID:** Klaim telah dibayarkan atau ditransfer oleh bagian Keuangan kepada pemohon.
 - **REJECTED / CANCELED:** Klaim ditolak atau dibatalkan.
 - **CLOSED:** Siklus klaim (termasuk *settlement* jika ada selisih dengan *Cash Advance*) telah ditutup/selesai secara administratif.
- **API Endpoints:**
 - GET /api/invoices - Mengambil daftar tagihan invoice.
 - POST /api/payments - Mencatat entri pembayaran baru.
 - GET /api/travel-requests & POST /api/travel-requests - Pengajuan SPD.
 - PUT /api/travel-requests/:id/approve - Validasi persetujuan SPD.
 - GET /api/reimbursements & POST /api/reimbursements - Pengajuan dan unggah bukti Reimbursement.
 - Serta endpoint terkait persetujuan *Cash Advance*.
 - **Tabel**
Terkait: lims.invoices, lims.payments, lims.travel_requests, lims.cash_advances, lims.reimbursements, lims.travel_request_counters, lims.cash_advance_counters.
- DFD Modul Keuangan & Perjalanan (Finance & Travel)**



6. Modul Manajemen Aset Peralatan Client (Asset Tracking)

- **Deskripsi:** Modul ini melacak seluruh siklus fisik alat klien (*testing equipments*) sejak diserahkan ke fasilitas uji hingga dikembalikan. Prosesnya mencakup pencatatan pergerakan barang secara dinamis (*Moving Asset*) antar laboratorium atau ke lokasi uji lapangan, serta pembuatan Berita Acara Serah Terima (Handover) yang sah kepada klien setelah pengujian selesai.
- **Alur Moving Asset:** Setiap kali aset berpindah titik lokasi atau berpindah penanggung jawab (misal dari Gudang ke Tim Tester), sistem mewajibkan pencatatan *movement* yang menjejaki *timeline* keberadaan aset guna mencegah kehilangan alat.
- **Fitur Asset Tracking & Inventory:**
 - **Unique Identification:** Melacak aset secara akurat menggunakan Nomor Seri / *Serial Numbers* (dengan dukungan integrasi pemindai QR/Barcode).
 - **Movement Logs:** Merekam perubahan lokasi dan status historis aset (seperti *NEW*, *USED*, *MAINTENANCE*, dll).
 - **Disposal Management:** Menangani proses penarikan atau pemusnahan (*decommissioning*) aset yang sudah rusak/tidak layak pakai.
 - **Activity History (Logs):** Audit trail lengkap yang mencatat siapa, kapan, dan apa yang terjadi pada suatu aset.
 - **Asset Labeling:** Pembuatan label QR Code otomatis untuk memudahkan identifikasi fisik.
 - **Scan Integration:** Mendukung pemindaian barcode/QR via kamera (web/mobile) untuk akses cepat ke data aset.
- **Arsitektur Teknis:**
 - **A. Backend (Golang):**
 - **Controller:** BackEnd/controllers/asset_controller.go
 - **Models:** BackEnd/models/models.go
 - **Routes:** Terdaftar di BackEnd/routes/routes.go dengan prefix */api/*.
 - **API Endpoints:**

Method	Endpoint	Fungsi
GET	/api/assets	Mengambil semua daftar aset dan detailnya.
POST	/api/asset-activity	Mencatat aktivitas baru (MOVE, DISPO, CEKIN).
GET	/api/asset-logs	Mengambil riwayat log aktivitas aset tertentu.
GET	/api/asset-statuses	Mengambil daftar status aset yang tersedia.
POST	/api/assets/movement	Mencatat log perpindahan lokasi (<i>moving</i>) alat uji secara <i>real-time</i> .

Method	Endpoint	Fungsi
POST	/api/assets/handover	Menyimpan bukti berita acara serah terima alat ke mitra (<i>Handover</i>).

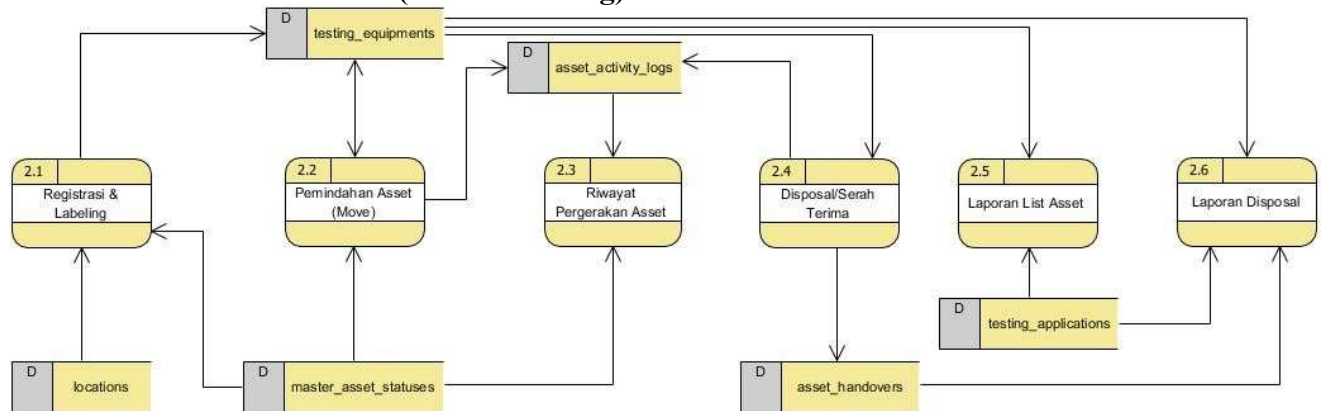
○ **B. Frontend (React):**

- **Page:** FrontEnd/src/views/pages/AssetManagementPage.jsx
- **Scanner:** Menggunakan library html5-qrcode untuk integrasi kamera.
- **Labeling:** Menggunakan qrcode.react untuk merender QR Code.

• **Tabel**

Terkait: lims.asset_activity_logs, lims.asset_handovers (disposal), lims.master_asset_statuses, lims.testing equipments.

DFD Modul Pelacakan Aset (Asset Tracking)



Penjelasan Alur DFD:

1. **Registrasi & Labeling:**

- Aset didaftarkan melalui form pendaftaran peralatan pada tahap registrasi.
- User mencetak label QR Code melalui menu *Asset Tracking*.
- Label ditempelkan secara fisik pada peralatan.

2. **Pemindahan Asset (Move):**

- User membuka menu *Asset Tracking*.
- User dapat memindai QR Code aset, memasukkan asset_id secara manual, atau mencari dari populasi data tabel aset yang ingin dipindahkan.
- Klik tombol **Move**, lalu pilih lokasi tujuan dan status baru.
- Sistem akan memperbarui lokasi aset di tabel utama (testing equipments) dan otomatis membuat entri log pergerakan baru (asset_activity_logs).

3. **Audit Trail (Riwayat):**

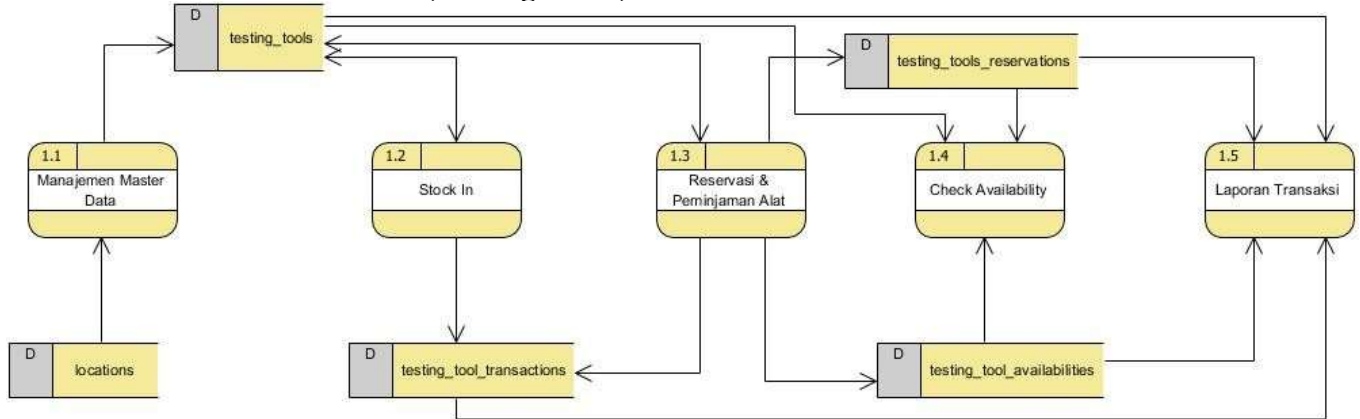
- User mengklik tombol **Logs** pada baris aset.
- Sistem akan menampilkan tabel riwayat yang berisi urutan kronologis seluruh pergerakan dan perubahan status aset tersebut.

7. Modul Manajemen Testing Tools (Testing Tools Tracking)

- **Deskripsi:** Sistem Testing Tools dirancang untuk mengelola inventaris alat pengujian, melacak ketersediaan berdasarkan waktu (reservasi), serta mencatat riwayat transaksi stok secara akurat menggunakan arsitektur database terpartisi.
- **Konsep Dasar Alat (Testing Tools):** Terdapat dua tipe alat dalam sistem:

- **STOCK (Alat Habis Pakai / Kuantitas):** Alat yang berkurang jumlahnya saat digunakan (contoh: Cairan kimia, Baju APD sekali pakai, Amunisi). Pengguna menginput jumlah kuantitas yang dibutuhkan.
 - **USAGE (Alat Pinjam / Durasi):** Alat yang dipinjam untuk durasi waktu tertentu dan dikembalikan (contoh: Osiloskop, Kamera Thermal, Kendaraan Operasional). Pelacakan dilakukan berbasis jadwal jam.
- **Fitur Resource & Tool Reservation:**
 - **Inventory Management:** Mengelola pemeliharaan stok alat-alat pengujian (seperti mesin simulator, sensor, dll).
 - **Availability Matrix:** Melacak matriks ketersediaan alat uji secara spesifik berdasarkan tanggal dan jam (waktu penggunaan).
 - **Reservation System:** Sistem pemesanan otomatis (*automated booking*) yang mencegah terjadinya bentrok atau konflik pemakaian alat oleh tim uji yang berbeda pada waktu bersamaan.
- **Arsitektur Database & Partisi:**
 - Untuk menjaga performa sistem seiring bertambahnya data transaksi, tabel riwayat transaksi alat menggunakan sistem **Monthly Partitioning** (Partisi Bulanan) pada PostgreSQL.
 - **Tabel Induk:** testing_tool_transactions
 - **Tabel Partisi:** testing_tool_transactions_YYYYMM (Contoh: testing_tool_transactions_202604)
 - **Mekanisme Query:**
 - Penambahan stok otomatis menulis ke tabel partisi bulan berjalan.
 - Laporan transaksi diarahkan langsung ke tabel partisi yang sesuai berdasarkan tanggal yang dipilih untuk optimasi kecepatan query.
 - **Aturan:** Pencarian laporan dibatasi maksimal dalam rentang 1 bulan yang sama (tidak boleh lintas bulan) untuk memastikan efisiensi pembacaan tabel partisi.
- **API Endpoints:**
 - GET /api/testing-tools - Mengambil daftar semua alat lab beserta informasi stok dan status.
 - GET /api/testing-tools/availability - Memeriksa status ketersediaan alat pada tanggal tertentu.
 - POST /api/testing-tools/reserve - Membuat pesanan reservasi penggunaan alat lab.
 - GET /api/testing-tools/:id/usage-history - Mengambil riwayat lengkap pemakaian alat lab oleh penguji.
- **Tabel Terkait:**
 - lims.testing_tools - Menyimpan data master alat, tipe (STOCK/USAGE), total stok, stok rusak, dan status.
 - lims.testing_tool_reservations - Menyimpan jadwal booking/reservasi alat oleh tim.
 - lims.testing_tool_availabilities - Matriks ketersediaan harian alat uji.
 - lims.testing_tool_transactions - Tabel induk (terpartisi) untuk mencatat riwayat log check-out/check-in dan transaksi stok.

DFD Modul Pemantauan Alat (Testing Tools)



8. Modul Kecerdasan Buatan (AI)

- **Deskripsi:** Rangkaian fitur kecerdasan buatan untuk mengotomatisasi penilaian pengujian, deteksi anomali, penyusunan draf laporan, dan asisten panduan SOP secara omnichannel.

A. Chatbot RAG Assistant (Lab Assistant)

- **Deskripsi:** Asisten cerdas untuk panduan pengujian menggunakan model Retrieval-Augmented Generation (RAG). **Chatbot (Agen AI) ini dirancang secara omnichannel, sehingga pengguna dan analis dapat berinteraksi mengakses asisten ini dari berbagai channel: Aplikasi Web LIMS, Aplikasi Mobile Android, WhatsApp Bot, maupun Bot Telegram.**
- **Estimasi Kebutuhan Sumber Daya (Offline/On-Premise):**
 - **Hardware (Rekomendasi Spesifikasi Server untuk Ollama Qwen 2.5):**

Ukuran Model	RAM Minimum	Spesifikasi CPU Minimum	Spesifikasi GPU (Sangat Direkomendasikan)	Kecepatan Respon (Token/s)
Qwen 2.5 3B (Default LIMS)	8 GB	4 Cores CPU	CPU-Only (Latensi 1-3s) atau GPU VRAM 4GB+ (Instan)	Tinggi (~20-40 tok/s)
Qwen 2.5 7B (Akurasi Tinggi)	16 GB	8 Cores CPU	GPU dengan VRAM 8GB+ (Nvidia RTX 3060/4060)	Sedang-Tinggi (~15-30 tok/s)
Qwen 2.5 14B (Sangat Akurat)	32 GB	12 Cores CPU	GPU dengan VRAM 12GB+ (Nvidia RTX 3060 12GB/4070)	Sedang (~10-20 tok/s)
Qwen 2.5 32B	64 GB	16 Cores CPU	GPU dengan VRAM 24GB+	Rendah-Sedang

Ukuran Model	RAM Minimum	Spesifikasi CPU Minimum	Spesifikasi GPU (Sangat Direkomendasikan)	Kecepatan Respon (Token/s)
(Enterprise)			(Nvidia RTX 3090/4090/A100)	(~5-15 tok/s)

- [!TIP] Untuk performa produksi yang stabil tanpa GPU, gunakan model **Qwen 2.5 3B** dengan minimal **16 GB RAM** dan **8 Cores CPU** pada VM khusus (Dedicated AI Server).
- **Software & Engine:** PostgreSQL + ekstensi pgvector, LangChain Go / Native Go Vector, dan Ollama (untuk model lokal & *embedding*) atau Groq API SDK (online).
- **Arsitektur Ingestion Pipeline (RAG):**
 - **Schema Database:** Inisialisasi melalui database/rag_schema.go (membuat tabel documents & document_chunks menggunakan tipe data vector). Disediakan juga script SQL manual di database/rag_schema.sql.
 - **RAG Services (services/rag_service.go):**
 - GetEmbedding: Membuat vektor numerik menggunakan model nomic-embed-text dari Ollama secara dinamis (berdasarkan variabel AI_EMBEDDING_API_URL).
 - SplitText: Pemotong teks (*chunker*) dengan ukuran ~1000 karakter dan *overlap* ~200 karakter.
 - IngestPDF: Mengekstrak teks dari PDF menggunakan utilitas Linux pdftotext dan pdftotext (dari paket poppler-utils), lalu mengonversinya menjadi *embeddings* secara asinkron (*background goroutine*).

[!IMPORTANT] **Prasyarat Sistem Linux:** Pastikan paket poppler-utils terpasang di VPS Anda agar backend LIMS dapat menjalankan utilitas pdftotext & pdftotext. Jalankan perintah instalasi berikut di VPS Anda: `sudo apt update && sudo apt install poppler-utils -y`




```

account_id VARCHAR(100) NOT NULL,    -- Chat ID Telegram, nomor HP
WhatsApp, atau ID Teams
created_at TIMESTAMPTZ WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
CONSTRAINT unique_user_channel UNIQUE (user_id, channel_type)
);

```

Panduan SQL Registrasi Sosial Media Pengguna (Manual Binding)

Jika Anda ingin menghubungkan, memeriksa, atau menghapus binding Chat ID Telegram pengguna secara manual langsung dari database pgAdmin:

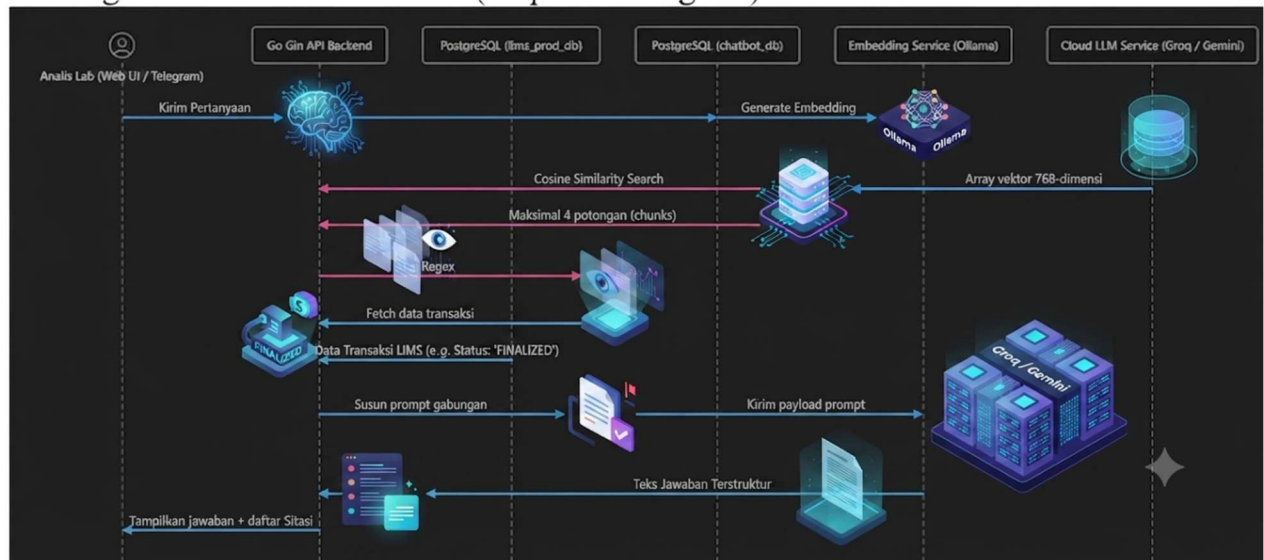
- **Pemeriksaan Status Hubungan Akun Pengguna (Misal User ID = 8):**
- SELECT * FROM chat_sch.user_social_accounts
- WHERE user_id = 8 AND channel_type = 'telegram';
- **Mendaftarkan Hubungan Akun Baru (Insert):**
- INSERT INTO chat_sch.user_social_accounts (user_id, channel_type, account_id)
- VALUES (8, 'telegram', '6310846414');
- **Mengupdate Chat ID Telegram Pengguna (Update):**
- UPDATE chat_sch.user_social_accounts
- SET account_id = '6310846414'
- WHERE user_id = 8 AND channel_type = 'telegram';
- **Memutuskan Hubungan Akun Telegram (Delete):**
- DELETE FROM chat_sch.user_social_accounts
- WHERE user_id = 8 AND channel_type = 'telegram';

Panduan Memeriksa Data RAG di Database PostgreSQL

Anda dapat melakukan audit dan pengecekan keberhasilan pengolahan PDF RAG secara langsung di PostgreSQL menggunakan perintah SQL berikut:

- **Pemeriksaan Status Dokumen PDF SOP yang Diunggah:**
- SELECT id, file_name, file_size, status, uploaded_at
- FROM chat_sch.documents
- ORDER BY uploaded_at DESC;
- **Melihat Jumlah Potongan Vektor (Chunks) yang Berhasil Dibuat (Misal untuk Dokumen ID = 1):**
- SELECT COUNT(*) FROM chat_sch.document_chunks WHERE document_id = 1;
- **Mengintip Cuplikan Teks SOP dan Array Vektor Koordinat:**
- SELECT id, page_number, substring(content from 1 for 60) AS cuplikan_teks, embedding
- FROM chat_sch.document_chunks
- LIMIT 3;
- **Memastikan Ekstensi pgvector Sudah Aktif di Database LIMS:**
- SELECT * FROM pg_extension WHERE extname = 'vector';

Sequence Diagram Query Chatbot



Penjelasan Alur Sequence Diagram:

Tahap 1: Pengunggahan File PDF (Upload SOP)

- Proses dimulai ketika pengguna mengunggah dokumen panduan lab (PDF) melalui antarmuka web LIMS.
- **HTTP Request:** Berkas dikirim melalui metode HTTP POST ke endpoint `/api/sop/upload` dengan enkapsulasi format `multipart/form-data` di bawah kata kunci `form document`.
- **Validasi & Penamaan Unik (sop_controller.go):** Backend memverifikasi tipe berkas harus berakhiran `.pdf`. Kemudian mengubah nama file secara unik menggunakan struktur *nanosecond timestamp* (misal: `1780721019655_nama_asli.pdf`) dan menyimpannya di `./public/uploads/sop`.
- **Penyimpanan Metadata:** Metadata dasar berkas dicatat ke dalam tabel database documents dengan status awal "processing".
- **Pemicu Latar Belakang (Goroutine):** Backend memicu fungsi penyerapan teks di latar belakang: `go services.IngestPDF(filePath, doc.ID)`. Backend langsung mengembalikan HTTP 201 ke frontend agar antarmuka web tetap interaktif.

Tahap 2: Ekstraksi, Pemecahan Teks (Chunking), & Vektorisasi Proses ini berjalan sepenuhnya secara asinkron di dalam fungsi `IngestPDF (rag_service.go)`:

- **Deteksi Halaman:** Sistem menjalankan perintah sistem `pdfinfo` untuk mengambil jumlah halaman yang ada di dalam PDF.
- **Ekstraksi Halaman:** Teks mentah dari tiap halaman diambil menggunakan utilitas sistem `pdftotext` dengan parameter `-layout` guna mempertahankan tata letak kolom asli dokumen.
- **Pembersihan & Pemotongan (Chunking):** Teks dibersihkan dari karakter kosong yang tidak standar. Sistem membaca ukuran potongan secara dinamis dari tabel parameter global database: `AI_CHUNK_SIZE` (Default: 1000 karakter) dan `AI_CHUNK_OVERLAP` (Default: 200 karakter). Teks dipotong menggunakan algoritma *sliding window* untuk menjembatani keterputusan arti.
- **Pembuatan Vektor (Embedding):** Setiap potongan teks dikirim ke endpoint `/api/embeddings` milik server Ollama. Model `nomic-embed-`

text digunakan untuk mengonversi teks mentah menjadi koordinat vektor numerik 768-dimensi.

- **Penyimpanan PostgreSQL:** Potongan teks dan array float32 vektor disimpan ke tabel `document_chunks` di PostgreSQL dengan tipe data khusus vector (ekstensi `pgvector`). Status dokumen diubah menjadi "completed".

Tahap 3: Pencocokan Pertanyaan User (Vector Similarity Search)

- **HTTP Request Query:** Frontend mengirim pesan teks dalam format JSON ke endpoint `/api/chat` (`chat_controller.go`).
- **Embedding Kueri:** Teks pertanyaan user diubah menjadi bentuk vektor 768-dimensi.
- **Pencarian Kemiripan Vektor (Similarity Search):** Backend melakukan kueri pencarian kemiripan kosinus (*Cosine Distance*) menggunakan operator `<=>` terhadap tabel `document_chunks`. Sistem mengambil maksimal 4 potongan terdekat (`AI_SEARCH_LIMIT`, default: 4).
- **Penyaringan Jarak (Threshold):** Hasil dengan *distance* di atas batas parameter `AI_SIMILARITY_THRESHOLD` (default: 0.65) akan diabaikan.

Tahap 4: Sintesis Jawaban oleh LLM & Rendering di UI

- **Penyusunan Prompt Terstruktur:** Potongan teks relevan digabungkan ke dalam instruksi ketat kepada LLM. *System Prompt* menugaskan AI untuk menjawab HANYA berdasarkan konteks dokumen referensi SOP.
- **Kirim Ke LLM:** Payload dikirim via HTTP request ke API LLM (lokal via Ollama `qwen2.5:1.5b` atau cloud via API Groq / Gemini) untuk diproses menjadi sintesis jawaban.
- **HTTP Response:** Backend mengembalikan objek JSON berisi `answer` dan `sources` (daftar nama file asal beserta nomor halamannya).
- **Rendering di UI (AsistenLab.jsx):** Teks jawaban ditampilkan sebagai pesan chat, dan daftar sumber dirender sebagai tautan referensi di bawah pesan, memungkinkan analis laboratorium untuk memverifikasi lembar SOP fisik yang sah secara langsung.

A2. Panduan Pengoperasian Ingestion SOP Chatbot (Upload, Verifikasi, & Detail Teknis)

Untuk melatih asisten cerdas agar dapat menjawab berdasarkan dokumen Standard Operating Procedure (SOP) laboratorium Anda, ikuti panduan operasional berikut:

1. Cara Mengunggah Berkas (Upload SOP)

1. Buka aplikasi web LIMS dan masuk ke menu **Asisten Lab (AI Chatbot)**.
2. Di panel sebelah kiri bawah tab **File SOP**, Anda akan melihat area dropzone bertuliskan "*Pilih atau Tarik File PDF (Maksimal 10MB)*".
3. Klik area tersebut untuk memilih berkas PDF dari laptop Anda, atau seret langsung berkas PDF Anda ke area tersebut.
4. Sistem akan mendeteksi file dan langsung memulai proses pengunggahan ke server.

2. Cara Memeriksa Status Ingestion (Checking Status)

Setelah file dikirim, sistem akan memasukkannya ke dalam daftar antrian pemrosesan di bawah dropzone. Status dokumen ditandai dengan label warna:

- **PROCESSING (Kuning):** Dokumen sedang dibaca, dipotong, dan dikonversi menjadi vektor di latar belakang. Anda dapat menutup halaman ini karena proses berjalan asinkron di server.
- **AKTIF (Hijau):** Proses ekstraksi teks dan vektorisasi selesai dengan sukses. AI Chatbot sekarang sudah memahami isi dokumen tersebut dan siap menjawab kueri Anda berdasarkan konteks dokumen ini.
- **GAGAL (Merah):** Terjadi kesalahan saat memproses dokumen di latar belakang. Jika statusnya adalah **GAGAL**, lakukan pemeriksaan berikut:
 1. **Cek Utilitas OS:** Pastikan package poppler-utils sudah terpasang di VPS (`sudo apt install poppler-utils -y`).
 2. **Cek Ollama:** Pastikan model embedding nomic-embed-text sudah terunduh di Ollama (`ollama pull nomic-embed-text`).
 3. **Hapus & Unggah Ulang:** Hapus berkas yang gagal dengan menekan tombol **Tong Sampah**, kemudian unggah kembali.

3. Detail Proses di Latar Belakang (Detail Ingestion Pipeline)

Ketika PDF diunggah, backend Go menjalankan pipeline pemrosesan data secara asinkron dalam 5 tahapan teknis:

1. **Validasi Format & Penyimpanan:** Memastikan file adalah PDF, mengubah namanya secara unik dengan unix timestamp prefix, dan menyimpannya di direktori `./public/uploads/sop/`.
2. **Penghitungan Halaman:** Backend memanggil perintah Linux `pdftotext` secara native untuk mendeteksi total halaman PDF.
3. **Ekstraksi Teks:** Backend memanggil utilitas Linux `pdftotext` per halaman dengan bendera `-layout` untuk mengambil teks mentah dengan struktur spasial kolom yang rapi.
4. **Pemecahan Teks (Chunking):** Teks dipecah menjadi beberapa bagian kecil (*chunks*) dengan ukuran karakter `AI_CHUNK_SIZE` (default: 1000) dan tumpang-tindih `AI_CHUNK_OVERLAP` (default: 200) untuk menjaga keutuhan konteks antar potongan.
5. **Vektorisasi & Penyimpanan:** Setiap potongan teks dikirim ke model `nomic-embed-text` di Ollama lokal untuk menghasilkan embedding vektor 768-dimensi. Teks beserta vektor koordinat tersebut akhirnya disimpan ke tabel `document_chunks` di PostgreSQL menggunakan ekstensi `pgvector`.

A3. Integrasi Telegram Bot (Dual-Mode Chat & AI)

LIMS memiliki integrasi bawaan dengan Telegram Bot yang berfungsi secara omnichannel untuk interaksi asisten AI (RAG) maupun percakapan langsung (*real-time*) dengan operator Helpdesk LIMS (Human-in-the-Loop).

1. Konfigurasi Awal di Server (VPS)

Untuk mengaktifkan bot Telegram LIMS, lakukan langkah konfigurasi berikut:

Langkah A: Dapatkan Token Bot dari Telegram BotFather

1. Buka aplikasi Telegram, cari akun resmi **@BotFather**.
2. Kirim perintah `/newbot` dan ikuti panduan untuk membuat bot baru (Dapatkan Nama Bot dan HTTP API Token).
3. Salin token API tersebut
(misal: `123456789:ABCdefGhIJKlmNoPQRsTUVwxyz`).

Langkah B: Update Berkas `.env` LIMS di VPS

1. Buka berkas konfigurasi .env instansi Anda di VPS:
2. `sudo nano /home/lims/lims1/backend/.env`
3. `sudo nano /home/lims/lims2/backend/.env`
4. Tambahkan token bot Anda ke variabel lingkungan:
5. `TELEGRAM_BOT_TOKEN=123456789:ABCdefGhIJKlmNoPQRsTUVwxyz`
6. Simpan berkas dan restart backend LIMS:
7. `sudo systemctl restart lims-backend-8081.service`
8. `sudo systemctl restart lims-backend-8091.service`

Langkah C: Daftarkan Webhook URL LIMS ke Telegram

Telegram membatasi port webhook HTTPS hanya pada port **80, 88, 443, dan 8443**.

Karena LIMS default menggunakan port 8082, Anda harus menambahkan port **8443** pada konfigurasi Nginx agar berjalan beriringan.

Untuk mencegah error SSL Handshake karena sertifikat self-signed bawaan (seperti lims.local), ikuti salah satu opsi pendaftaran berikut:

Opsi A: Menggunakan IP Publik VPS (Sertifikat Self-Signed Khusus IP)

Jika Anda mengakses LIMS menggunakan alamat IP (seperti 212.85.24.33), buatlah sertifikat self-signed khusus untuk IP tersebut agar diterima oleh Telegram:

1. **Buat Sertifikat SSL Khusus IP:** Jalankan perintah ini di terminal VPS Anda (ganti 212.85.24.33 dengan IP VPS Anda):
2. `sudo openssl req -x509 -newkey rsa:2048 \`
3. `-keyout /etc/nginx/ssl/telegram.key \`
4. `-out /etc/nginx/ssl/telegram.crt \`
5. `-days 365 -nodes \`
6. `-subj "/CN=212.85.24.33" \`
7. `-addext "subjectAltName = IP:212.85.24.33"`
8. **Konfigurasi Blok Server 8443 di Nginx:** Buka berkas konfigurasi Nginx:
9. `sudo nano /etc/nginx/conf.d/lims.conf`

Tambahkan blok server baru berikut di akhir berkas (pastikan berada di luar blok server lain):

```
server {
    listen 8443 ssl;
    server_name 212.85.24.33;

    ssl_certificate /etc/nginx/ssl/telegram.crt;
    ssl_certificate_key /etc/nginx/ssl/telegram.key;
    ssl_protocols TLSv1.2 TLSv1.3;
    ssl_prefer_server_ciphers on;
    ssl_ciphers HIGH:!aNULL:!MD5;

    location /api {
        proxy_pass http://lims_backend_cluster;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection 'upgrade';
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
```

```

    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto $scheme;
}
}

```

10. Tes Sintaks & Restart Nginx:

11. `sudo nginx -t && sudo systemctl restart nginx`
12. (Jika firewall UFW aktif) Izinkan akses port 8443:
13. `sudo ufw allow 8443/tcp`

14. Daftarkan Webhook & Unggah Sertifikat ke Telegram:

15. `curl -F "url=https://212.85.24.33:8443/api/webhook/telegram" \`
16. `-F "certificate=@/etc/nginx/ssl/telegram.crt" \`
17. `https://api.telegram.org/bot<TOKEN_BOT_ANDA>/setWebhook`

Opsi B: Menggunakan Domain Resmi (Sertifikat SSL Tepercaya / CA-Signed)

Jika Anda menggunakan domain resmi (seperti `lims.perusahaan.com`) dengan SSL tepercaya (seperti Let's Encrypt), Anda cukup mendaftarkan Webhook URL menggunakan domain resmi Anda (tanpa melampirkan file `-F "certificate=@..."`):

```
bash
curl -F "url=https://lims.perusahaan.com/api/webhook/telegram" \
https://api.telegram.org/bot<TOKEN_BOT_ANDA>/setWebhook
```

2. Cara Menghubungkan Akun Telegram Pengguna (Binding Chat ID)

Agar staf/pengguna dapat menerima notifikasi dan membalas pesan Helpdesk via Telegram, akun LIMS mereka harus dihubungkan dengan *Chat ID* Telegram milik mereka. Berikut adalah 2 cara mendapatkan Chat ID:

Cara A: Melalui Bot LIMS (Otomatis)

1. Buka aplikasi Telegram, cari Bot LIMS Anda lalu klik **Start** (atau kirim pesan bebas seperti "Halo").
2. Jika server backend LIMS berjalan normal, bot akan otomatis merespons: *"Halo! Akun Telegram Anda belum terhubung ke sistem LIMS. Chat ID Anda adalah: 123456789..."*
3. Catat angka Chat ID (123456789) tersebut.

Cara B: Menggunakan Bot Pengecek ID (Alternatif Jika Server Mati)

1. Buka Telegram dan cari bot resmi `@userinfobot` atau `@getidsbot`.
2. Klik **Start**.
3. Bot akan membalas dengan profil Anda beserta deretan angka ID. Catat angka tersebut.

Langkah Terakhir: Memasukkan Chat ID ke Sistem LIMS

1. Login ke aplikasi web LIMS menggunakan akun staf yang bersangkutan.
2. Buka halaman **Profil (My Profile)** atau menu **User Management**.
3. Masukkan angka Chat ID yang sudah didapatkan ke dalam kolom **Telegram Chat ID**.
4. Klik **Simpan**. Sekarang akun Telegram Anda sudah terhubung secara dinamis dan siap menerima/mengirim pesan!

3. Cara Menugaskan Staf Sebagai Operator Helpdesk (Penerima Chat)

Sistem LIMS dirancang agar pesan keluhan/bantuan dari pengguna biasa (via Telegram) akan secara otomatis diteruskan (*forwarded*) ke seluruh staf yang bertugas sebagai layanan bantuan (Helpdesk).

Agar seorang staf dapat mulai menerima dan membalas pesan dari pengguna lain, staf tersebut harus diberikan peran (Role) khusus oleh Administrator.

Langkah-langkah Assign Helpdesk:

1. Login ke web LIMS menggunakan akun **Administrator**.
2. Masuk ke menu **System Admin > User Management**.
3. Cari akun staf yang ingin ditugaskan sebagai Helpdesk, lalu klik **Edit**.
4. Pada kolom **Role** (Peran), ubah peran mereka menjadi **HELPDESK** (atau biarkan sebagai ADMIN karena Admin juga secara otomatis menerima pesan).
5. Klik **Simpan**.
6. **Sangat Penting:** Pastikan staf tersebut juga telah mendaftarkan *Telegram Chat ID* mereka ke sistem (menggunakan panduan Binding di atas). Jika Chat ID mereka kosong, sistem tidak akan tahu ke nomor mana pesan harus diteruskan!

Begitu peran diubah dan Chat ID terdaftar, staf tersebut akan langsung menerima notifikasi instan di Telegram mereka setiap kali ada pengguna yang bertanya atau meminta bantuan.

4. Fungsionalitas Dual-Mode (AI Chatbot vs Chat Operator)

Setelah terhubung, bot akan menyaring pesan masuk secara otomatis berdasarkan format pesan:

Mode A: Tanya AI Chatbot (/ai <pertanyaan>)

- **Format:** Kirim pesan diawali dengan kata kunci /ai diikuti pertanyaan Anda.
 - *Contoh:* /ai bagaimana prosedur kalibrasi alat X?
 - *Contoh:* /ai status aplikasi LIMS-2026-00018
- **Proses:** Bot akan memanggil mesin RAG LIMS untuk mencari referensi dokumen SOP di database pgvector dan mencocokkan status pengajuan realtime.
- **Balasan:** Bot membalas pesan di Telegram secara instan berisi jawaban lengkap yang dihasilkan oleh LLM lokal (Ollama Qwen 2.5 3B), disertai daftar referensi file PDF SOP dan nomor halaman aslinya.

Mode B: Chat Operator / Helpdesk (Tanpa Awalan /ai)

- **Format:** Kirim pesan teks biasa tanpa kata kunci khusus.
 - *Contoh:* Halo admin, mohon bantuannya, alat saya mengalami error saat pengujian.
- **Proses:** Bot menganggap ini sebagai pesan keluhan. Pesan tersebut akan disimpan di tabel riwayat chat internal dan **diteruskan secara otomatis (forwarded)** ke seluruh Telegram milik staf **Admin / Helpdesk** LIMS yang aktif.
- **Balasan Operator (Human-in-the-Loop):** Staf Helpdesk/Admin dapat menjawab pesan tersebut langsung dari Telegram mereka sendiri dengan menekan tombol **Reply/Balas** pada pesan forward tersebut. Sistem akan mem-parsing pengirim asli dan merutekan balasan operator kembali ke Telegram pengguna asal secara personal.

[!NOTE] **Catatan Penyimpanan Database:** Chat dalam **Mode B (Chat Operator)** akan disimpan ke dalam tabel **chat_sch.agent_chats** agar operator Helpdesk dapat melihat log riwayat keluhan di dashboard web LIMS. Sedangkan interaksi obrolan asisten AI (**Mode**

A) langsung diproses *on-the-fly* lewat RAG dan tidak dimasukkan ke dalam agent_chats demi menjaga kerapian pelacakan laporan operator.

B. Rekomendasi Laporan AI (Report Generator)

- **Deskripsi Report Generator:** Generator draf kesimpulan kelulusan dan saran teknis dari deviasi parameter uji menggunakan LLM.
- **API Endpoints (Report Generator):**
 - POST /api/applications/:id/generate-report - Memicu LLM untuk membuat draf analisis evaluasi alat.
- **Tabel Terkait:** lims.testing_applications (analysis_conclusion column).

C. Predictive Quality Control (PQC)

Sistem LIMS mengintegrasikan modul Kecerdasan Buatan (AI-ML) untuk melakukan **Predictive Quality Control (PQC)** secara *real-time* pada saat operator menyimpan hasil pengujian parameter produk. Sistem ini dirancang untuk mendeteksi data uji abnormal, mencegah kerusakan fatal pada peralatan uji, serta memprediksi kelayakan peralatan sebelum dilakukan pengujian fisik di lapangan.

1. Latar Belakang & Permasalahan

Laboratorium pengujian LIMS melibatkan pengujian fisik bernilai tinggi, berisiko tinggi, dan memakan biaya besar. Beberapa permasalahan utama yang diselesaikan oleh PQC meliputi:

- **Deteksi Anomali Terlambat:** Ketidaknormalan data sensor sering kali baru terdeteksi setelah pengujian selesai, mengakibatkan pemborosan bahan bakar, energi, dan waktu.
- **Risiko Kerusakan Alat Uji (Fatal Damage):** Menguji peralatan yang sebenarnya secara struktural tidak layak dapat merusak sensor lab atau fasilitas uji yang bernilai tinggi.
- **Keterbatasan Ambang Batas Statis:** Standar kelayakan statis (misal: suhu tidak boleh $> 50^{\circ}\text{C}$) mengabaikan korelasi antar-variabel. Kombinasi parameter (suhu 45°C dengan getaran frekuensi tinggi dan tekanan oli rendah) bisa mengindikasikan anomali kritis meskipun masing-masing parameter masih di bawah batas aman individu.
- **Kesalahan Input Data (Human Error):** Salah ketik desimal atau digit pada lembar kerja uji yang di-input manual oleh operator sering lolos verifikasi awal.

2. Solusi & Pilar Kecerdasan AI PQC

Sistem PQC mengintegrasikan algoritma Machine Learning ke dalam backend LIMS untuk memberikan perlindungan dan rekomendasi prediktif secara *real-time*:

- **Anomaly Detection (Deteksi Anomali):** Mendeteksi deviasi/ketidakwajaran data masukan secara cerdas dengan menganalisis korelasi multi-variabel (suhu, getaran, tekanan, listrik, dll.) secara simultan.
- **Feasibility Prediction (Prediksi Batas Kelayakan):** Menghitung probabilitas keberhasilan atau risiko kegagalan pengujian sebelum pengujian fisik dijalankan (pre-test analysis).

Metode Pengisian & Pengambilan Data:

1. **Metode Semi-Otomatis (Tanpa Integrasi Alat Uji):** Operator memasukkan nilai pengukuran secara manual ke lembar kerja digital (Worksheet). LIMS mengirim data input ke AI Service untuk divalidasi sebelum disimpan ke database. Jika AI

mendeteksi anomali (misal salah ketik), sistem menampilkan notifikasi peringatan seketika kepada operator sebelum data telanjur disimpan.

2. **Metode Otomatis (Terintegrasi):** LIMS membaca langsung data log hasil uji dari IoT/sensor alat uji via API/koneksi serial, kemudian AI memproses data tersebut secara otomatis tanpa intervensi manusia.

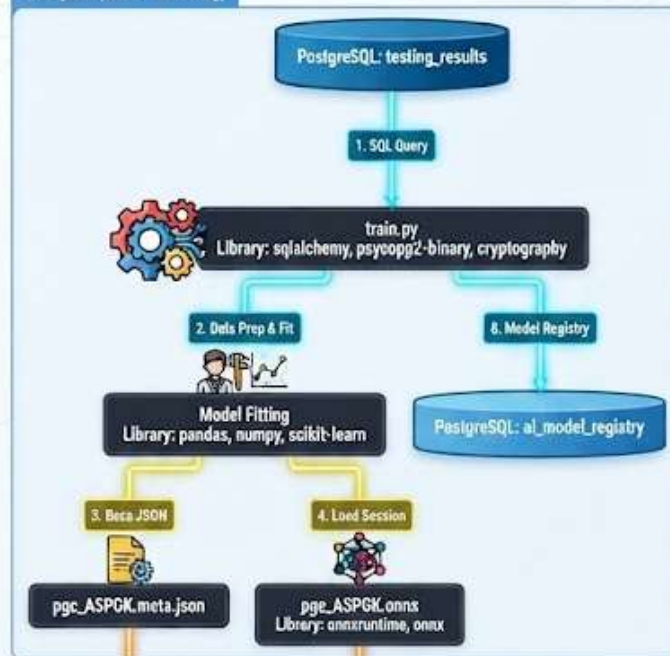
3. Arsitektur Teknis & Alur Proses

LIMS menggunakan arsitektur *microservices* untuk menjalankan model AI ini secara lokal (on-premise) demi menjamin keamanan data pengujian dan operasional sistem.

Arsitektur Hybrid ONNX: Sistem ini telah dimigrasikan dari arsitektur *microservice* Python (FastAPI HTTP port 8086) ke arsitektur Hybrid ONNX. Model dilatih secara offline menggunakan Python, diekspor ke format ONNX (.onnx), lalu dimuat dan dieksekusi secara *native* langsung di dalam Go Backend menggunakan *shared library* C++ ONNX Runtime. Hal ini menghilangkan ketergantungan runtime Python (FastAPI) di server produksi pada saat deteksi real-time.

- **Inference (Deteksi Real-time - Sangat Ringan):** AI hanya menjalankan model yang sudah jadi (Inference) untuk memvalidasi data baru. Proses ini berupa operasi matematika sederhana yang berjalan super cepat (di bawah 10 milidetik) dengan alokasi memori RAM hanya sekitar 50MB - 150MB, sehingga tidak akan menurunkan performa sistem.
- **Training (Pelatihan Ulang Model - Terjadwal):** Pelatihan ulang model (retraining) membutuhkan daya komputasi tinggi dan dijadwalkan secara otomatis pada waktu-waktu tidak sibuk (off-peak hours) sebagai *Background Job* terisolasi di AI Microservice, sehingga sama sekali tidak mengganggu pengguna LIMS yang sedang aktif.

Sisi Python (Offline - Retraining)



Sisi Go Backend (Online - Real-time Inference)



4. Fitur Utama AI PQC

Sistem PQC ini diintegrasikan langsung pada form pengisian lembar kerja uji (testing worksheet):

- **Real-time Input Guard:** AI menganalisis nilai parameter secara *real-time*. Jika mendeteksi ketidakwajaran pola (misal: temperatur naik tapi tekanan drop drastis), form akan menolak penyimpanan otomatis dan meminta operator memverifikasi ulang (notifikasi alert kuning).
- **Pre-Test Feasibility Score:** Sebelum pengujian fisik berjalan, analisis menginput data awal/kondisi lingkungan. AI mengeluarkan nilai persentase kelayakan uji (misal: *Kelayakan Uji: 94% - Aman Dijalankan*) dalam bentuk indikator *speedometer* radial (Merah - Kuning - Hijau).
- **Dynamic Tolerances Checker:** Batas toleransi dihitung secara cerdas berdasarkan merek, varian, umur, dan riwayat kerusakan alat pertahanan, bukan sekadar batas flat seragam.
- **Anomaly Audit Logger:** Halaman log audit khusus untuk supervisor yang menampilkan daftar data anomali yang berhasil diblokir atau di-override oleh operator, lengkap dengan analisis kontribusi variabel yang dicurigai oleh AI (SHAP values).

5. Detail Algoritma AI-ML & Pustaka Pendukung

Sistem LIMS menggunakan berbagai algoritma Machine Learning sesuai kebutuhan:

- **Isolation Forest & One-Class SVM:** Digunakan untuk mendeteksi anomali pada data pengujian multi-dimensi (outliers) tanpa memerlukan banyak data berlabel anomali.
- **Autoencoders (Deep Learning):** Menggunakan neural network sederhana untuk mempelajari pola normal dari parameter uji. Jika rekonstruksi error (reconstruction error) dari input baru tinggi, data ditandai sebagai anomali.
- **XGBoost / Random Forest Classifier:** Digunakan untuk memprediksi kelayakan uji (Feasibility Scoring) berdasarkan fitur-fitur teknis historis peralatan sejenis yang pernah lulus atau gagal.

5.1. Mekanisme Klasifikasi Isolation Forest

- LIMS menggunakan Isolation Forest sebagai model deteksi anomali utama dengan parameter kontaminasi 0.05 dan jumlah pohon 100 (`random_state=42`).
- **Formulasi Skor Anomali:** Skor anomali $s(x)$ untuk sampel masukan x didefinisikan sebagai:

$$s(x) = 2 - E(h(x)) \quad c(n) = 2 - c(n)E(h(x))$$

Di mana:

- $h(x)$: Panjang lintasan sampel x (jumlah percabangan dari root ke ujung daun).
- $E(h(x))$: Rata-rata panjang lintasan $h(x)$ dari seluruh pohon di dalam hutan.
- $c(n)$: Rata-rata panjang lintasan pencarian biner gagal pada dataset berukuran n :

$$c(n) = 2 \ln(n-1) + 0.5772156649 - 2(n-1) \quad nc(n) = 2 \ln(n-1) + 0.5772156649 - n2(n-1)$$

- **Normalisasi Skor:** Pustaka Scikit-Learn Python melaporkan skor sampel mentah (`score_samples`) dalam nilai negatif: $\text{raw_score} = -s(x)$. Go

Backend menormalkan nilai ini menjadi persentase tingkat anomali $[0.0, 1.0]$ $[0.0, 1.0]$:

$$\text{anomalyScore} = \max(0.0, \min(1.0, 0.5 - \text{rawScore}))$$

$$\text{anomalyScore} = \max(0.0, \min(1.0, 0.5 - \text{rawScore}))$$

- **Threshold Sistem:** Jika tingkat anomali $\geq 50\%$ (0.5), data dianggap sebagai **anomali**.

5.2. Batas Toleransi Individual (Secondary Boundary Check) Karena Isolation Forest mengevaluasi pola gabungan (*multivariate*), dilakukan pemeriksaan batas toleransi individu:

Batas = Median $\pm$ (1.5 $\times$ Std) Batas = Median $\pm$ (1.5 $\times$ Std)

- Batas atas dibatasi maksimal 100.0 dan batas bawah minimal 0.0.
- Jika terdapat minimal 1 sub-aspek yang nilainya di luar batas toleransi, sistem secara otomatis menetapkan `is_anomaly = true` dan tingkat anomali minimal 55% (0.55).

5.3. Pseudo-SHAP (Leave-One-Out Permutation) Untuk mengurai kontribusi kesalahan dari masing-masing variabel:

1. Hitung skor anomali dasar dengan semua nilai aktual: `baseScore = session.Run(aktual)`
2. Untuk setiap sub-aspek ii , ganti nilai aktual dengan median historis, hitung ulang skor, lalu hitung selisih:

$\text{contri} = \max(0.0, \text{tempScore}_i - \text{baseScore})$

3. Normalisasi kontribusi menjadi persentase SHAP Value:

$\text{SHAP}_i = (\text{contri} / \sum_j \text{contrib}_j) \times 100\%$

5.4. Daftar Pustaka (Library) & Komponen Teknis Daftar pustaka yang digunakan tercantum pada tabel di bawah ini:

No	Nama Library	Lingkungan	Versi	Fungsi Utama
1	github.com/yalue/onnxruntime_go	Go (Backend)	v1.16.0	Memuat library C++ ONNX Runtime dan mengeksekusi model ONNX secara <i>native</i>
2	scikit-learn	Python (Retrain)	1.4.1.post1	Melatih algoritma <i>Isolation Forest</i>
3	skl2onnx	Python (Retrain)	...	Mengonversi model Scikit-Learn ke format ONNX
4	onnx	Python (Retrain)	...	Serialisasi graf model ke berkas biner .onnx
5	pandas	Python (Retrain)	2.2.1	Memproses data transaksi historis dari PostgreSQL

No	Nama Library	Lingkungan	Versi	Fungsi Utama
6	numpy	Python (Retrain)	1.26.4	Komputasi numerik array dan matriks data
7	sqlalchemy	Python (Retrain)	2.0.28	ORM dan toolkit SQL untuk interaksi basis data
8	psycopg2-binary	Python (Retrain)	2.9.9	Driver PostgreSQL untuk Python
9	cryptography	Python (Retrain)	42.0.5	Mendekripsi password database yang terenkripsi AES-CFB

6. Studi Kasus Perhitungan Matematis AI PQC (Aspek: KEPEN)

Bagian ini menyajikan simulasi perhitungan keputusan anomali dan kontribusi SHAP menggunakan profil data statistik riil dari database LIMS Anda (berdasarkan seeder dataset transaksi).

6.1. Profil Statistik Riil Database (Aspek: KEPEN) Berikut adalah nilai statistik riil untuk 6 sub-aspek KEPEN:

Sub-Aspect Code	Parameter Name	Median (Nilai Tengah)	Standard Deviation (σ)	Rentang Batas Toleransi ($1.5 \times \sigma$ - $1.5 \times \sigma$)
KEDAI	Daya output audio	90.00	18.23	62.65 - 100.0
KELCH	Threshold/Squelch	85.44	19.51	56.17 - 100.0
KERUS	Kerusakan fisik	86.72	11.73	69.12 - 98.31
KESEL	Keselamatan kerja	87.05	18.21	59.73 - 100.0
KESEN	Sensitivitas	86.66	17.40	60.56 - 100.0
KESUA	Kualitas suara	85.73	13.91	64.86 - 100.0

[!NOTE] **Imputasi Nilai Kosong:** Jika operator mengosongkan parameter tertentu saat input (misal KERUS), sistem AI akan otomatis mengisi kolom kosong tersebut dengan nilai median historisnya (86.72) sebelum melakukan evaluasi model.

6.2. Simulasi Kasus 1: Input Data Normal

- Data Input Aktual:**

- o KEDAI = 90.0 (Operator mengetik aktif)
- o KELCH = 85.0 (Operator mengetik aktif)
- o KERUS = kosong (Diisi otomatis oleh AI menggunakan median = 86.72)
- o KESEL = kosong (Diisi otomatis oleh AI menggunakan median = 87.05)
- o KESEN = 85.0 (Operator mengetik aktif)
- o KESUA = kosong (Diisi otomatis oleh AI menggunakan median = 85.73)

- Vektor Input (Urutan Alfabetis):**

Xnormal=[90.0,85.0,86.72,87.05,85.0,85.73] Xnormal=[90.0,85.0,86.72,87.05,85.0,85.73]

- **Kalkulasi Model:**

- Nilai masukan berada di wilayah padat data historis normal, sehingga rata-rata panjang lintasan pohon panjang.
- Model Isolation Forest menghasilkan skor mentah $\text{raw_score} = -0.3232$.
- **Tingkat**
Anomali: $\text{anomalyScore} = -(-0.3232) = 0.3232$
 $\text{anomalyScore} = -(-0.3232) = 0.3232$ (atau 32.32%32.32%).
- **Hasil:** Karena tingkat anomali $32.32\% \leq 50\%$, sistem menyatakan data **NORMAL** (klasifikasi 1) dan transaksi berhasil disimpan.

6.3. Simulasi Kasus 2: Input Data Anomali (Outlier)

- **Data Input Aktual (Salah Ketik/Masalah Sensor):**

- KEDAI = 10.0 (Sangat jauh dari median normal 90.0)
- KELCH = 12.0 (Sangat jauh dari median normal 85.44)
- KERUS = kosong (Median: 86.72)
- KESEL = kosong (Median: 87.05)
- KESEN = 15.0 (Sangat jauh dari median normal 86.66)
- KESUA = kosong (Median: 85.73)

- **Vektor Input:**

$X_{\text{anomaly}} = [10.0, 12.0, 86.72, 87.05, 15.0, 85.73]$
 $X_{\text{anomaly}} = [10.0, 12.0, 86.72, 87.05, 15.0, 85.73]$

- **Kalkulasi Model:**

- Karena nilai 10.0, 12.0, dan 15.0 sangat ekstrem, titik ini terisolasi sangat cepat di percabangan awal pohon.
- Model Isolation Forest menghasilkan skor mentah $\text{raw_score} = -0.6319$.
- **Tingkat**
Anomali: $\text{anomalyScore} = -(-0.6319) = 0.6319$
 $\text{anomalyScore} = -(-0.6319) = 0.6319$ (atau 63.19%63.19%).
- **Hasil:** Karena tingkat anomali $63.19\% > 50\%$, sistem menyatakan data **ANOMALI** (klasifikasi -1). Penyimpanan dibatalkan (rollback), dan status transaksi di-set menjadi **BLOCKED**.

6.4. Perhitungan Kontribusi Anomali (SHAP Leave-One-Out) Untuk mencari tahu parameter mana saja yang paling berkontribusi terhadap status anomali pada Kasus 2:

1. **Baseline Score:** $\text{base_score} = -0.6319$ (skor riil transaksi anomali ini).
2. **Substitusi Satu per Satu (Leave-One-Out):**
 - **Substitusi KEDAI (10.0 → 90.0):**
 - Vektor uji baru: $[90.0, 12.0, 86.72, 87.05, 15.0, 85.73]$
 - Model menghasilkan $\text{new_raw_score} = -0.5609$.
 - Selisih kontribusi: $\Delta \text{score}_{\text{KEDAI}} = -0.5609 - (-0.6319) = 0.0710$
 - $\Delta \text{score}_{\text{KEDAI}} = -0.5609 - (-0.6319) = 0.0710$ (tingkat keanehan membaik sebesar 0.0710 poin).
 - **Substitusi KELCH (12.0 → 85.44):**

- Vektor uji baru: [10.0,85.44,86.72,87.05,15.0,85.73][10.0,85.44,86.72,87.05,15.0,85.73]
 - Model menghasilkan new_raw_score = -0.5586.
 - Selisih kontribusi: $\Delta\text{score}_{\text{KELCH}} = -0.5586 - (-0.6319) = 0.0733$ $\Delta\text{score}_{\text{KELCH}} = -0.5586 - (-0.6319) = 0.0733$ (tingkat keanehan membaik sebesar 0.0733 poin).
 - **Substitusi KESEN (15.0 → 86.66):**
 - Vektor uji baru: [10.0,12.0,86.72,87.05,86.66,85.73][10.0,12.0,86.72,87.05,86.66,85.73]
 - Model menghasilkan new_raw_score = -0.5482.
 - Selisih kontribusi: $\Delta\text{score}_{\text{KESEN}} = -0.5482 - (-0.6319) = 0.0837$ $\Delta\text{score}_{\text{KESEN}} = -0.5482 - (-0.6319) = 0.0837$ (tingkat keanehan membaik sebesar 0.0837 poin).
 - **Substitusi Parameter Normal (KERUS, KESEL, KESUA):**
 - Karena nilainya sudah sama dengan median historis, selisih kontribusinya: $\Delta\text{score} = 0.0$ $\Delta\text{score} = 0.0$.
- 3. Persentase Kontribusi:**
- Total selisih kontribusi: $\sum \Delta\text{score} = 0.0710 + 0.0733 + 0.0837 = 0.2280$ $\sum \Delta\text{score} = 0.0710 + 0.0733 + 0.0837 = 0.2280$
 - **KESEN:** $0.0837 / 0.2280 \times 100\% = 36.70\%$ $0.0837 / 0.2280 \times 100\% = 36.70\%$
 - **KELCH:** $0.0733 / 0.2280 \times 100\% = 32.16\%$ $0.0733 / 0.2280 \times 100\% = 32.16\%$
 - **KEDAI:** $0.0710 / 0.2280 \times 100\% = 31.14\%$ $0.0710 / 0.2280 \times 100\% = 31.14\%$
 - Hasil persentase kontribusi ini dikirim ke frontend untuk ditampilkan sebagai grafik kontributor kesalahan.

7. Skema Basis Data & Integrasi Otorisasi Keamanan

7.1. Tabel lims.ai_model_registry Tabel registri mencatat setiap versi model yang sukses dilatih secara offline oleh skrip train.py:

```
CREATE TABLE IF NOT EXISTS lims.ai_model_registry (
  id SERIAL PRIMARY KEY,
  model_name VARCHAR(100) NOT NULL,
  version VARCHAR(20) NOT NULL,
  accuracy_score NUMERIC(5, 4),
  fl_score NUMERIC(5, 4),
  trained_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP NOT NULL,
  model_path VARCHAR(255) NOT NULL,
  status VARCHAR(20) DEFAULT 'ACTIVE'
);
```

```
CREATE INDEX idx_ai_model_registry_model_name ON lims.ai_model_registry (model_name);
```

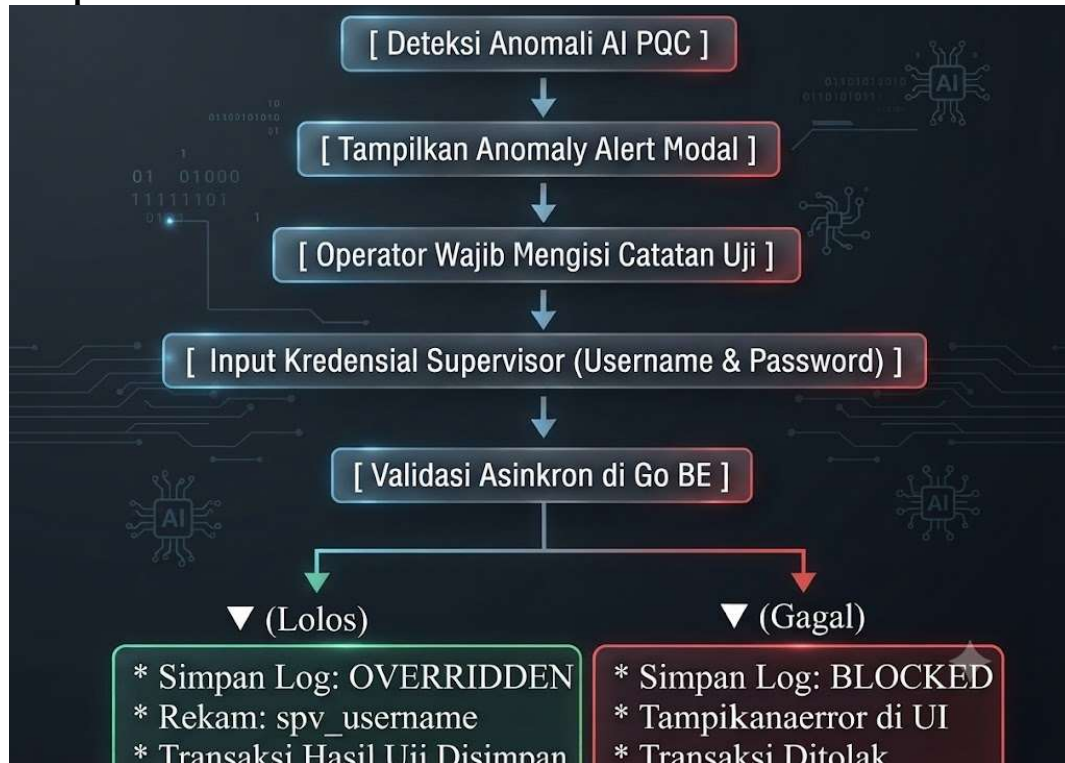


```
CREATE INDEX idx_ai_model_registry_status ON lims.ai_model_registry (status);
```

7.2. Tabel lims.ai_anomaly_logs Tabel terpartisi bulanan untuk merekam log kejadian anomali:

```
CREATE TABLE IF NOT EXISTS lims.ai_anomaly_logs (  
  id BIGSERIAL,  
  application_id BIGINT,  
  operator_username VARCHAR(30),  
  parameters_data JSONB,  
  anomaly_score NUMERIC(5, 4),  
  shap_values JSONB,  
  status VARCHAR(20) DEFAULT 'BLOCKED', -- 'BLOCKED', 'OVERRIDDEN'  
  override_reason VARCHAR(500),  
  created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP  
  NOT NULL,  
  PRIMARY KEY (id, created_at)  
) PARTITION BY RANGE (created_at);
```

Tampilan UI Mekanisme Otorisasi & Override:



7.3. Validasi Keamanan Otorisasi Override (Supervisor Bypass) Jika data uji ditandai anomali oleh model, operator dapat melakukan *override* dengan persetujuan supervisor di tempat (*on-site override*):

1. **Form Input Override:** Modal anomali menampilkan kolom Username Supervisor dan Password Supervisor.
2. **Otorisasi Satu Kali (One-Off Verification):** Backend Go memverifikasi spv_username dan spv_password secara mandiri dari sesi operator yang sedang berjalan.

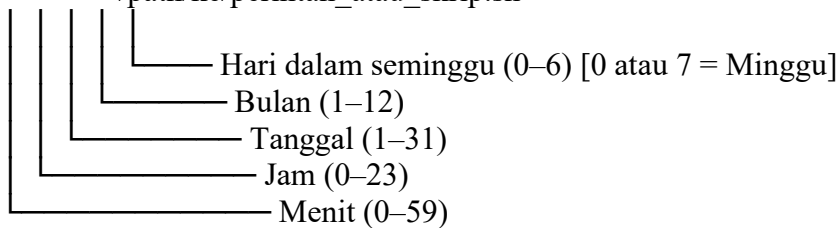
3. **Hak Akses Spesifik:** Akun supervisor yang melakukan otorisasi wajib memiliki role **SUPERVISOR_SCORE** atau **ADMIN**. Peran supervisor departemen lain (seperti **SUPERVISOR_LABORATORY**, **SUPERVISOR_FIELD**, dsb.) **TIDAK diizinkan** melakukan override demi menjaga akuntabilitas penilaian hasil pengujian.
4. **Audit Trail:** Jika otorisasi valid, status dicatat sebagai **OVERRIDDEN** dan kolom `override_reason` diisi dengan format: `[Override by: supervisor_username]` alasan_override. Jika gagal, status tetap **BLOCKED**.

8. Penjadwalan Retraining Otomatis (Crontab)

Format Perintah Crontab

Format perintah crontab (*cron job*) digunakan untuk menjalankan skrip atau perintah secara otomatis pada waktu yang telah ditentukan di sistem operasi Linux/Unix.

`* * * * * /path/ke/perintah_atau_skrip.sh`



Nilai yang Diizinkan per Bidang:

Bidang	Kegunaan	Nilai
Menit	Menentukan menit jalannya perintah	0–59
Jam	Menentukan jam jalannya perintah	0–23 (Format 24 jam)
Tanggal	Menentukan hari dalam bulan	1–31
Bulan	Menentukan bulan	1–12 (atau jan, feb, dst.)
Hari	Menentukan hari dalam seminggu	0–6 (0 = Minggu, 1 = Senin, dst., atau sun, mon)

Operator Khusus (Wildcard):

- ***** (Asterisk): Berarti "setiap". Misalnya * di bidang menit berarti "jalankan setiap menit".
- **,** (Koma): Daftar nilai. Contoh: 1,3,5 di bidang jam = berjalan pada jam 1, 3, dan 5.
- **-** (Tanda Hubung): Rentang nilai. Contoh: 1-5 di bidang hari = Senin sampai Jumat.
- **/** (Garis Miring): Interval/langkah. Contoh: */15 di bidang menit = setiap 15 menit.

Contoh Perintah Crontab:

`# Jalankan setiap hari Senin jam 2 dini hari`

`0 2 * * 1 /home/user/maintenance.sh`

`# Jalankan setiap 15 menit`

```
*/15 * * * * /home/user/check_server.sh
```

```
# Jalankan dua kali sehari (jam 5 pagi dan jam 5 sore)
```

```
0 5,17 * * * /home/user/sync.sh
```

```
# Contoh konfigurasi cron job retraining model AI LIMS
```

```
0 2 * * 0 /home/nurlim/train.sh >> /home/nurlim/lims-ai-train-$(date +%Y%m%d).log  
2>&1
```

[!IMPORTANT] Penggunaan variabel penanggalan `$(date +%Y%m%d)` pada nama file log memastikan berkas log baru terbuat setiap hari (misalnya: `lims-ai-train-20260612.log`) sehingga riwayat log pelatihan hari-hari sebelumnya tidak terhapus (*overwritten*). Pastikan Anda memberikan backslash `\` pada karakter persen `%` agar dievaluasi dengan benar oleh cron daemon.

Monitoring & Analisis Log Retraining

Untuk memeriksa riwayat atau memantau jalannya retraining secara *real-time*, gunakan perintah pemantauan berkas log harian:

```
tail -f /home/nurlim/lims-ai-train-$(date +%Y%m%d).log
```

D. Parameter Global AI (Konfigurasi Dinamis & Env)

Modul Kecerdasan Buatan (AI) di LIMS (PQC, RAG Chatbot, dan Report Generator) dikonfigurasi melalui kombinasi parameter dinamis pada basis data (tabel `lims.global_parameters`) serta variabel lingkungan (*environment variables* pada berkas `.env`).

1. Parameter Global di Database (`lims.global_parameters`)

Parameter ini dapat diubah secara dinamis melalui database tanpa perlu merestart atau mengompilasi ulang aplikasi backend LIMS:

No	Parameter Key	Tipe / Format	Nilai Default	Keterangan & Fungsi
1	AI_PQC_ENABLED	Boolean (true/false)	true	Mengaktifkan/menonaktifkan modul deteksi anomali <i>Predictive Quality Control</i> secara global. Jika false, semua validasi anomali dilewati (<i>soft-bypass</i>).
2	AI_METADATA_FOLDER	String / Path	/home/lims/lims1/backend/ai_service/models	Path absolut folder tempat Go Backend dan <code>train.py</code> membaca/menulis berkas model <code>.onnx</code> dan metadata statistik <code>.json</code> .
3	AI_SIMILARITY_THRESHOLD	Float (0.0 - 1.0)	0.65	Batas jarak kemiripan maksimum (Cosine Distance) untuk matching chunks pada chatbot RAG. Hasil dengan jarak di atas batas ini akan diabaikan.

No	Parameter Key	Tipe / Format	Nilai Default	Keterangan & Fungsi
4	AI_SEARCH_LIMIT	Integer	4	Batas maksimum jumlah potongan dokumen (<i>chunks</i>) terdekat yang diambil dari document_chunks sebagai basis konteks prompt LLM.
5	AI_CHUNK_SIZE	Integer	1000	Batas maksimum panjang karakter per potongan (<i>chunk</i>) saat melakukan penyerapan (<i>ingestion</i>) dokumen PDF SOP.
6	AI_CHUNK_OVERLAP	Integer	200	Jumlah panjang karakter tumpang tindih (<i>overlap</i>) antar potongan dokumen yang berdekatan untuk menjaga keutuhan arti kalimat.
7	AI_MAX_TOKENS	Integer	1000	Batas maksimal panjang token respons yang dihasilkan oleh LLM (Chatbot & Report Generator).
8	AI_TIMEOUT	Integer	120	Batas waktu (timeout) dalam satuan detik saat backend Go melakukan HTTP request API panggilan ke LLM.
9	AI_OCR_ENABLED	Boolean (true/false)	true	Mengaktifkan/menonaktifkan parsing teks berbasis OCR untuk lampiran file PDF atau gambar dokumen pengujian.
10	AI_INTERVAL_CHAT	Integer	12000	Batas jeda waktu minimum (rate limit) dalam milidetik antar request chat untuk mencegah pemborosan kuota API LLM.

2. Variabel Lingkungan di Berkas .env

Variabel lingkungan ini bersifat statis dan dimuat saat aplikasi *startup* (membutuhkan restart service LIMS jika nilainya diubah):

No	Environment Variable	Nilai Default / Contoh	Keterangan & Fungsi
1	AI_API_URL	https://api.groq.com/openai/v1	URL endpoint API dasar LLM eksternal (Groq,

No	Environment Variable	Nilai Default / Contoh	Keterangan & Fungsi
			OpenAI, atau Ollama Local).
2	AI_API_KEY	gsk_hAVuGExM9...	API Key/kunci otentikasi rahasia untuk mengakses LLM eksternal (gunakan none jika memakai Ollama lokal).
3	AI_MODEL	llama-3.1-8b-instant	Nama model LLM yang digunakan untuk generasi teks asisten chatbot dan rekomendasi laporan.
4	AI_EMBEDDING_API_URL	http://127.0.0.1:11434	URL API endpoint server Ollama lokal untuk pembuatan embedding vektor.
5	AI_EMBEDDING_MODEL	nomic-embed-text	Nama model embedding lokal yang digunakan untuk mengonversi dokumen mentah menjadi representasi koordinat vektor.

9. Modul Integrasi Simulator IoT

- **Deskripsi:** Perekaman data otomatis dari mesin simulator/peralatan luar secara langsung melalui REST API. Modul ini memungkinkan skor hasil pengujian fisik dikirim secara otomatis dari peralatan testing atau simulator hardware ke LIMS, tanpa input manual oleh operator. Data masuk ke tabel *staging* `simulator_data_logs` sebelum dikonsumsi oleh proses Pelaksanaan Pengujian.
- **API Endpoints:**
 - POST `/api/machine-integration/results` - Menerima data payload skor pengujian dari sistem luar dengan validasi header `X-Simulator-Key`.
- **Tabel Terkait:** `mecs.simulator_data_logs`, `mecs.testing_results`.

A. Arsitektur Integrasi & Protokol

Protokol yang Didukung:

Kondisi Peralatan	Protokol	Keterangan
Peralatan bisa kirim HTTP + custom header	HTTP/REST (Webhook)	Langsung ke LIMS tanpa middleware

Kondisi Peralatan	Protokol	Keterangan
Peralatan hanya kirim data mentah (Serial/RS232)	Node-RED + HTTP	Node-RED sebagai penerjemah & <i>transformer</i>
Perlu agregasi data dari banyak sensor	Node-RED Aggregator	Node-RED sebagai <i>buffer</i> dan agregator
Peralatan hanya support Modbus/OPC-UA	Node-RED + Modbus/OPC-UA	Node-RED punya node driver siap pakai
Sensor lapangan dengan koneksi putus-nyambung	MQTT + Broker	Protokol ringan untuk perangkat lapangan
Peralatan hanya support socket raw	TCP/UDP Sockets (Raw Stream)	Node-RED / Custom middleware sebagai Socket listener & forwarder

Opsi 1: Langsung ke LIMS (Tanpa Node-RED) ✓ (Direkomendasikan)

Jika peralatan sudah dapat mengirim HTTP POST dengan *custom header*, tidak perlu middleware tambahan:

database MEC tanpa intervensi manusia.

ID Registrasi (Application ID Database)

37

Mesin Penguji (Terminal ID)

Uji Balistik Utama (MESIN-BALISTIK-01)

Parameter Pengujian (Kode Parameter)

ACC

Hasil / Skor Terukur (Angka)

98,5

Catatan Telemetri Mesin

Kondisi normal. Kecepatan angin 10 knots.

 **KIRIM DATA TELEMETRI KE MEC**

```
{
  "code": "",
  "name": "",
  "methodology_code": "",
  "methodology": {
    "code": "",
    "name": "",
    "test_type_code": "",
    "test_type": {
      "code": "",
      "name": "",
      "created_at": "0001-01-01T00:00:00Z",
      "updated_at": "0001-01-01T00:00:00Z",
      "created_user": "",
      "updated_user": ""
    }
  }
}
```

```
curl -X POST http://localhost:8081/api/machine-integration/results \
-H "Content-Type: application/json" \
-H "X-Simulator-Key: <SIMULATOR_KEY_dari_ENV>" \
-d
```

```
{
  "application_id": 38,
  "scoring_parameter_code": "MOB",
  "score": 75.0,
  "machine_id": "MESIN-BALISTIK-01",
  "notes": "Kondisi normal."
}
```

Opsi 2: Via Node-RED sebagai HTTP Webhook (Untuk Peralatan Legacy/Serial)

Peralatan —HTTP POST—► Node-RED (port 1880) —HTTP POST—► LIMS (port 8081)

Konfigurasi **Function Node** di Node-RED untuk *transform* payload dan injeksi API Key:

```
const raw = msg.payload;
msg.payload = {
  application_id: raw.app_id || raw.application_id,
  scoring_parameter_code: raw.param_code || raw.scoring_parameter_code,
  score: parseFloat(raw.value || raw.score || 0),
  machine_id: raw.device_id || raw.machine_id || "UNKNOWN",
}
```

```

    notes: raw.notes || raw.description || ""
  };
  msg.headers = {
    "Content-Type": "application/json",
    "X-Simulator-Key": global.get('LIMS_API_KEY') // Ambil dari env Node-RED
  };
  return msg;
[!IMPORTANT]

```

- **Keamanan API Key:** Simpan X-Simulator-Key di *Environment Variable* Node-RED, jangan di-*hardcode* langsung di dalam *flow*. Edit `~/.node-red/settings.js` dan tambahkan `LIMS_API_KEY: process.env.LIMS_API_KEY` di dalam `functionGlobalContext`.
- **Resolusi Loopback Host (WSL 2):** Jika LIMS Backend berjalan di dalam WSL 2 dan Node-RED berjalan di Windows Host, konfigurasi URL pada node **HTTP Request** di Node-RED harus menggunakan alamat IPv4 eksplisit `http://127.0.0.1:8081/...` bukan `localhost`. Jika menggunakan `localhost`, Node.js di Windows akan mencoba meresolusi alamat loopback IPv6 (`::1`) yang tidak didukung port forwarding bawaan WSL 2, mengakibatkan *timeout* (no response from server).
- **Karakteristik Logging Node-RED:**
 - **Jalur MQTT:** Dirancang bersuara/verbose. Setiap data MQTT yang masuk dicetak ke konsol debug Node-RED melalui node `mecs_debug` dan `log` fungsi.
 - **Jalur HTTP (/data-peralatan):** Dirancang senyap (*silent*). Selama pengiriman dari skrip PowerShell/HTTP client berhasil, Node-RED akan merespon secara instan ke pengirim dengan status 200 OK tanpa menulis log apa pun ke konsol untuk menghemat performa. Keberhasilan pengujian harus divalidasi langsung melalui output PowerShell (SUCCESS [HTTP 200]) atau dengan memeriksa record baru di tabel `staging.simulator_data_logs` database LIMS.

B. Alur Data Simulator ke Scoring

Peralatan/Simulator —POST—► `/api/machine-integration/results`

- Validasi X-Simulator-Key
- Simpan ke `mecs.simulator_data_logs` (staging)
- Pelaksanaan Pengujian membaca log (`is_simulator = true`)
- Skor auto-fill ke `testing_results` (read-only di UI)

Model SimulatorDataLog (file: `models/models.go`):

```

type SimulatorDataLog struct {
  ID          uint   `gorm:"primaryKey" json:"id"`
  ApplicationID uint   `json:"application_id"`
  ScoringParameterCode string `gorm:"type:varchar(5);index" json:"scoring_parameter_code"`
  Score        float64 `json:"score"`
  MachineID     string  `gorm:"type:varchar(60)" json:"machine_id"`
  Notes         string  `gorm:"type:varchar(200)" json:"notes"`
  IsUsed        bool    `gorm:"default:false" json:"is_used"`
}

```



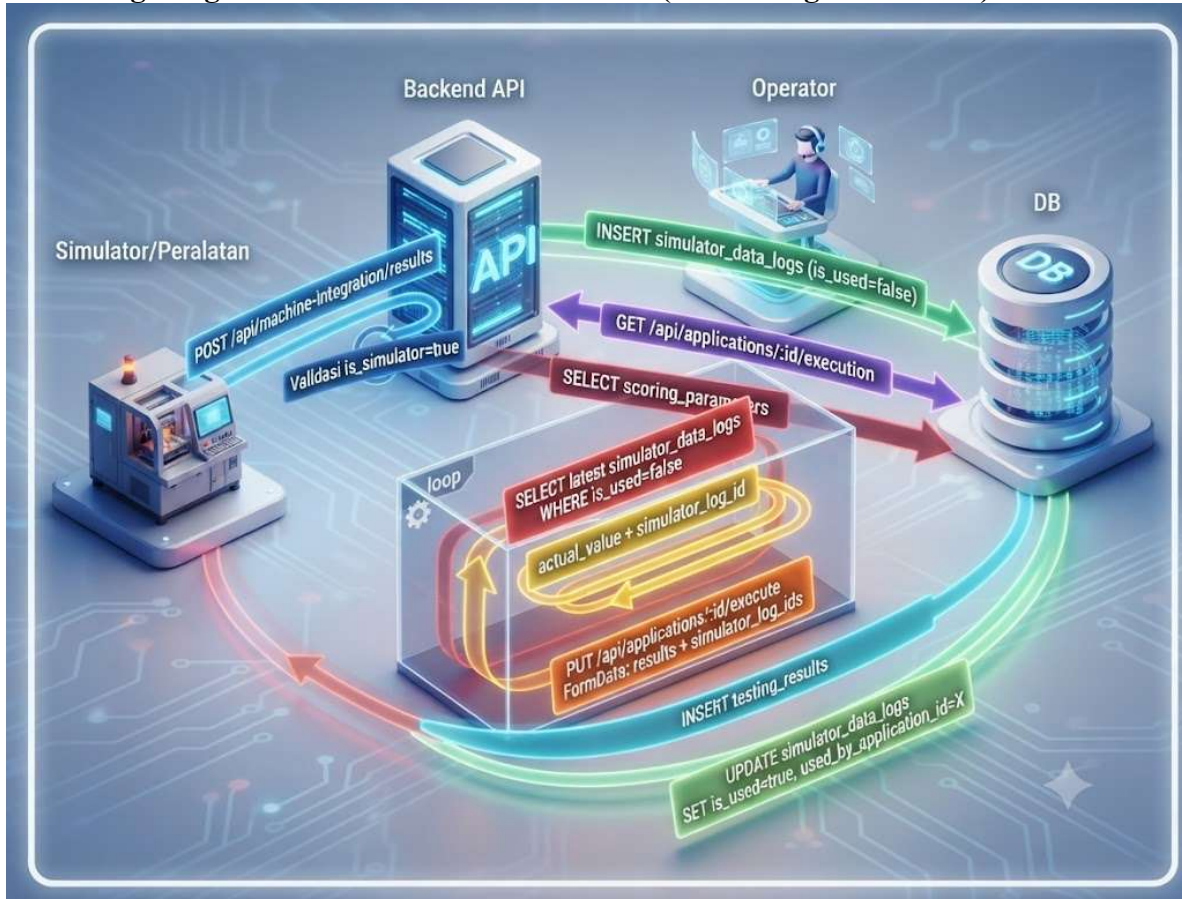
```

    CreatedAt      time.Time `json:"created_at"`
}

```

Field `is_simulator = true` pada `scoring_sub_aspects` menandai parameter yang nilainya di-*auto-fill* dari log simulator dan bersifat *read-only* di UI Pelaksanaan Pengujian.

C. Testing dengan Hardware Simulator HTML (Same-Origin & Secure)



Untuk keperluan pengujian integrasi tanpa perangkat fisik, LIMS menyediakan file simulator interaktif `hardware_simulator.html` yang terintegrasi di dalam proyek.

Lokasi Berkas Fisik:

- **Repositori Proyek (Windows):** `backend/public/hardware_simulator.html`
- **Deployment VM (Ubuntu/WSL):**
 - `/home/lims/lims1/backend/public/hardware_simulator.html` (LIMS Instance 1)
 - `/home/lims/lims2/backend/public/hardware_simulator.html` (LIMS Instance 2)
- **Berkas Cadangan (UAT):** `D:\Data_NK\Project5\Test-Militer\UAT\hardware_simulator.html`

[!CAUTION] **Kebijakan Keamanan CORS & Bahaya file:// (Origin: null):** Jangan pernah membuka file simulator UAT langsung sebagai file lokal (`file:///D:/.../hardware_simulator.html`) di browser pada server produksi. Membuka file lokal akan mengirimkan header `Origin: null`. Jika backend memperbolehkan `null` di `ALLOWED_ORIGINS` CORS, hal ini menciptakan celah

keamanan berat (*CORS null origin vulnerability*) di mana penyerang dapat membaca data LIMS melalui script lokal berbahaya.

Implementasi Verifikasi Origin: Logika validasi CORS di LIMS diimplementasikan pada middleware Go di berkas

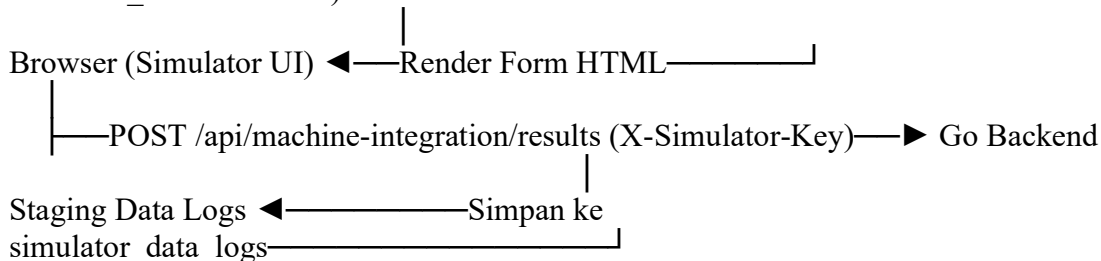
[security_headers.go](file:///d:/Data_NK/Project5/AI/LIM_System_Linux_OK/backend/middleware/security_headers.go#L60-L108) melalui fungsi `CORSWithWhitelist()`.

Solusi Secure (Same-Origin Hosting): Simulator kini di-host secara *native* dan aman oleh Go Backend pada endpoint `/simulator`. Karena halaman simulator dimuat dari domain yang sama dengan API LIMS, semua request bersifat **Same-Origin** dan terbebas dari issue CORS tanpa perlu melonggarkan keamanan server.

Keamanan Integrasi Perangkat IoT Fisik (Real IoT): Ketika menggunakan perangkat keras IoT riil (fisik), koneksi dilakukan secara langsung (*Direct HTTP Client*) tanpa menggunakan browser. Karena request ini dikirim langsung oleh sistem tertanam (*embedded system*) atau *gateway*, request tidak menyertakan header Origin. Middleware LIMS secara otomatis melewati pemeriksaan CORS apabila header Origin kosong, sehingga perangkat IoT riil tetap dapat menyalurkan data telemetri dengan aman tanpa memerlukan celah CORS null di `.env`.

Alur Proses Pengujian (DFD Level 1 - M2M Simulator):

Browser (Same-Origin) —GET /simulator—► Go Backend (Serve hardware_simulator.html)



Panduan Langkah Pengujian:

1. Buka browser dan akses simulator pada URL:
 - o Development / Lokal: `http://localhost:8081/simulator`
 - o Load Balancer / Produksi: `http://lims.local:8091/simulator`
2. Ubah **ID Registrasi** (gunakan Application ID yang aktif di database LIMS), **Mesin Penguji** (Terminal ID), dan **Parameter Pengujian** (misal KONSİ atau KONPE).
3. Ubah **Hasil / Skor Terukur** (misal 98.50).
4. Klik tombol **KIRIM DATA TELEMETRI KE MEC**.
5. Konsol log di bawah tombol akan menampilkan status koneksi dan respon sukses dari server. Data telemetri tersebut secara otomatis tersimpan ke tabel staging simulator_data_logs.

[!NOTE] **Pastikan ID & Kode Parameter valid:** Database akan menolak data dengan Application ID atau Kode Parameter yang tidak terdaftar (*Foreign Key Constraint*). Gunakan ID registrasi dan kode parameter yang benar-benar ada di database LIMS.

D. Pengujian Terbimbing MQTT (Simulasi MQTT Publisher)

Untuk pengujian alur MQTT menggunakan skrip PowerShell yang tersedia di folder Test-Militer/UAT/:

1. **Prasyarat Layanan:**

- LIMS Backend berjalan pada port 8081 (atau 8091 produksi).
- MQTT Broker (Mosquitto) aktif pada port 1883.
- Node-RED aktif pada port 1880 dengan alur mecs/peralatan/hasil terpasang.

2. Langkah Pengujian:

- Buka **PowerShell** pada Windows Host.
- Arahkan ke folder UAT:
- `cd D:\Data_NK\Project5\Test-Militer\UAT\`
- Jalankan skrip `test_mqtt_scoring.ps1` dengan parameter yang sesuai:
- `.\test_mqtt_scoring.ps1 -AppId 205 -ParamCode "KONSI" -Values @(70, 80, 90)`

3. Verifikasi Log di Node-RED:

- Buka antarmuka Node-RED debug console (atau pantau log konsol startup Node-RED).
- Log akan mencetak proses parsing dan transformasi payload seperti berikut:
- [Parse JSON] JSON Parsed
- [Transform + Auth] Transformed payload
- [HTTP POST] HTTP 200
- [Log] HTTP 200: Data simulator berhasil diterima...

10. Modul Monitoring

- **Deskripsi:** Menyediakan visualisasi data secara real-time dan pemantauan SLA pengujian harian.
- **API Endpoints:**
 - `GET /api/dashboard/summary` - Mengambil statistik ringkasan aplikasi (jumlah registrasi, verifikasi, selesai).
- **Penyajian Log GoAccess:**
 - `/report.html` - Menyajikan berkas HTML statis lalu lintas sistem secara real-time, terhubung via WebSocket ke GoAccess server (port 7890).
- **Tabel Terkait:** `lims.user_activity_logs`.

11. Modul Reporting

- **Deskripsi:** Modul terpusat yang menangani pembuatan seluruh berkas laporan LIMS — mulai dari Sertifikat Hasil Pengujian (SHP), laporan teknis, laporan operasional pengujian, laporan keuangan & perjalanan, hingga laporan inventaris aset. Semua fitur laporan dikonsolidasikan dalam modul ini.
- **Daftar Fitur Laporan:**

No	Laporan	Deskripsi	Sumber Data
1	Sertifikat Hasil Pengujian (SHP)	Laporan kelulusan dan laporan teknis akhir yang dicetak/didistribusikan ke klien.	<code>lims.testing_applications</code> , <code>lims.testing_aspect_scores</code>
2	Laporan Summary Testing	Ringkasan statistik seluruh hasil pengujian per periode/status.	<code>lims.testing_applications</code>

No	Laporan	Deskripsi	Sumber Data
3	Laporan Detil Testing	Rincian lengkap parameter, skor, dan hasil uji per aplikasi.	lims.testing_results, lims.testing_aspect_scores
4	Laporan Daftar Aset	Inventarisasi seluruh aset klien beserta status dan lokasinya.	lims.testing equipments, lims.master_asset_statuses
5	Laporan Serah Terima	Berita acara serah terima aset kepada klien setelah pengujian selesai.	lims.asset_handovers
6	Laporan Tagihan	Daftar seluruh invoice dan status pembayarannya per periode.	lims.invoices
7	Laporan Pembayaran	Rekap transaksi pembayaran invoice yang telah diterima.	lims.payments
8	Laporan SPD	Rekapitulasi Surat Perjalanan Dinas beserta status persetujuannya.	lims.travel_requests, lims.travel_request_counters
9	Laporan Cash Advance	Rincian pengajuan dan pencairan uang muka operasional dinas.	lims.cash_advances, lims.cash_advance_counters
10	Laporan Reimbursement	Rekap klaim penggantian biaya aktual beserta bukti struk/nota.	lims.reimbursements

- **API Endpoints:**

- GET /api/applications/:id/pdf - Mengambil data terformat untuk di-render menjadi PDF Laporan Teknis / SHP.
- GET /api/reports/testing-summary - Laporan Summary Testing per periode.
- GET /api/reports/testing-detail/:id - Laporan Detil Testing per aplikasi.
- GET /api/reports/assets - Laporan Daftar Aset klien.
- GET /api/reports/handovers - Laporan Serah Terima Aset.
- GET /api/reports/invoices - Laporan Tagihan per periode.
- GET /api/reports/payments - Laporan Pembayaran invoice.
- GET /api/reports/travel-requests - Laporan SPD (Surat Perjalanan Dinas).
- GET /api/reports/cash-advances - Laporan Cash Advance.

- GET /api/reports/reimbursements - Laporan Reimbursement.
- **Arsitektur Rendering:**
 - **SHP & Laporan Teknis:** Menggunakan utilitas utils/print.js untuk merender template cetak secara langsung melalui dialog cetak browser (*print styles CSS*).
 - **Laporan Operasional & Keuangan:** Di-render di sisi frontend (React) menggunakan data JSON dari API, dengan opsi ekspor ke PDF atau cetak langsung via browser.

- **Tabel**

Terkait: lims.testing_applications, lims.testing_results, lims.testing_aspect_scores, lims.testing equipments, lims.asset_handovers, lims.invoices, lims.payments, lims.travel_requests, lims.cash_advances, lims.reimbursements.

12. Modul Pemeliharaan Sistem

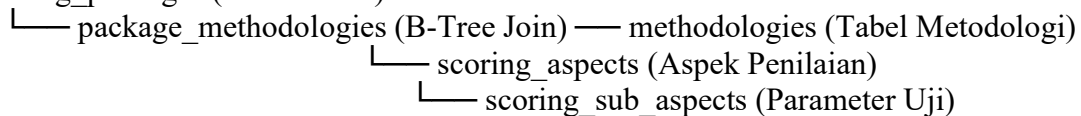
- **Deskripsi:** Layanan backend terpusat untuk operasi perawatan basis data, pembagian partisi secara dinamis, identifikasi tabel bermasalah, dan pengamanan data (*backup/restore*).
- **Mekanisme & Komponen:**
 - **Manajemen Partisi:** Menggunakan models/schema_manager.go untuk mengeksekusi PostgreSQL Native Partitioning tabel testing_applications dan testing_results.
 - **Analisa Struktur & Partisi:** Pemeriksaan kondisi struktur tabel dan status partisi aktif di database — menampilkan daftar partisi yang sudah dibuat, rentang tanggal partisi, jumlah baris, dan ukuran masing-masing partisi — untuk verifikasi integritas dan perencanaan kapasitas.
 - **Analisa Bloat & Vacuum:** Logika di backend untuk mendeteksi *dead tuples* dan memicu instruksi VACUUM ANALYZE otomatis.
 - **Backup & Restore:** Endpoint internal untuk memicu utilitas pg_dump dan pengembalian data melalui backend LIMS secara aman.
- **Tabel Terkait:** Seluruh tabel di dalam skema lims dan chat_sch.

6. Arsitektur & Konfigurasi Paket Layanan (Testing Packages)

Hierarki hubungan konfigurasi paket dirancang untuk mendukung modifikasi visibilitas parameter pengujian khusus tanpa memengaruhi data master metodologi induk.

Diagram Hierarki Hubungan Data Master

testing_packages (Tabel Paket)



Whitelist Filtering dengan Tabel Jembatan

assigns

references

has aspects

has sub-aspects

filters active

whitelists aspect_code

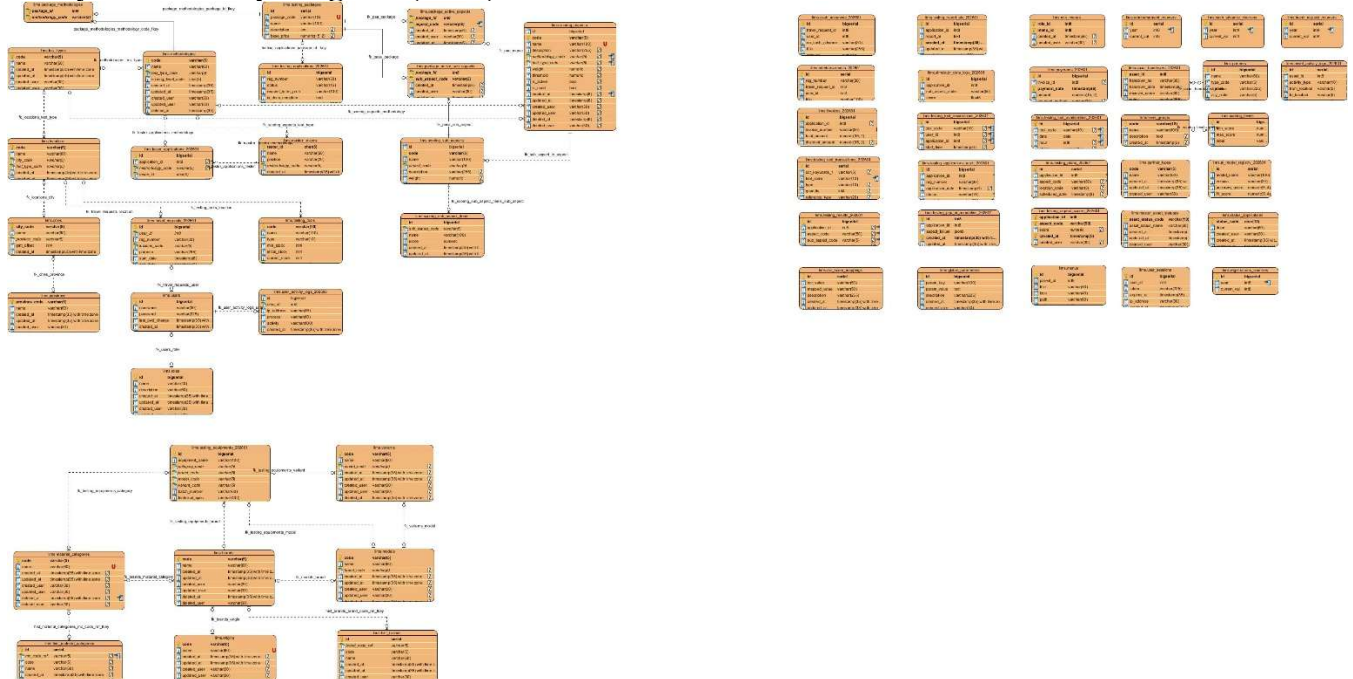
filters active

whitelists sub_aspect_code
 testing_packages
 package_methodologies
 methodologies
 scoring_aspects
 scoring_sub_aspects
 package_active_aspects
 package_active_sub_aspects

- **package_active_aspects:** Memetakan testing_packages langsung ke scoring_aspects. Jika didefinisikan, aspek lain di luar daftar diabaikan saat transaksi dibuat.
- **package_active_sub_aspects:** Memetakan testing_packages ke scoring_sub_aspects untuk membatasi parameter uji mana saja yang aktif dan harus diisi nilainya oleh tester.

7. Tabel Basis Data (Database Tables Directory)

Entity Relationship Diagram (ERD)



[!NOTE] **Penyederhanaan ERD:** Untuk menjaga keterbacaan diagram, tabel *history* (berawalan hist_) dan tabel *archive* (berawalan _arc) tidak ditampilkan dalam ERD di atas. Selain itu, beberapa tabel yang menggunakan skema partisi (*partitioned tables*) sengaja tidak dipasang *Foreign Key (FK) constraint* karena keterbatasan PostgreSQL Native Partitioning yang tidak mendukung FK referensi langsung ke tabel induk terpartisi. Berikut adalah informasi dari seluruh tabel yang beroperasi dalam ekosistem database LIMS (lims_prod_db maupun schema khusus seperti chat_sch), beserta keterangan pembagian partisinya:

No	Databa se	Nama Tabel	Par tisi (Y/ N)	Arch ive (Y/N)	His tor y (Y/ N)	Modul	Fungsi
1	lims_pr od_db	lims.ai_model _registry	N	N	N	Konfig urasi AI	Registrasi model AI yang digunakan sistem
2	lims_pr od_db	lims.asset_act ivity_logs	Y	Y	N	Pelaca kan Aset	Riwayat pergerakan & log pemakaian aset internal
3	lims_pr od_db	lims.asset_ha ndovers	Y	Y	N	Pelaca kan Aset	Log pencatatan serah terima (handover) barang
4	lims_pr od_db	lims.brands	N	N	Y	Master Data	Master merk/brand aset atau alat uji
5	lims_pr od_db	lims.cash_adv ance_counters	N	N	N	Keuan gan & Perjala nan	Penomoran otomatis pengajuan uang muka
6	lims_pr od_db	lims.cash_adv ances	Y	Y	N	Keuan gan & Perjala nan	Permintaan Uang Muka dinas dan operasional
7	lims_pr od_db	lims.cities	N	N	Y	Master Data	Master data kota/kabupaten
8	lims_pr od_db	lims.global_p arameters	N	N	Y	Master Konfig urasi	Ambang batas skor global, kredensial AI, versi app
9	lims_pr od_db	lims.invoices	Y	Y	N	Penagi han (Billin g)	Dokumen tagihan / invoice layanan LIMS

No	Database	Nama Tabel	Partisi (Y/N)	Archive (Y/N)	History (Y/N)	Modul	Fungsi
10	lims_prod_db	lims.level_groups	N	N	N	Penilaian & Scoring	Pengelompokan level parameter skor
11	lims_prod_db	lims.locations	N	N	Y	Master Data	Master data lokasi uji
12	lims_prod_db	lims.master_asset_statuses	N	N	Y	Pelacakan Aset	Referensi status aset (Dipinjam, Gudang, dll)
13	lims_prod_db	lims.master_testers	N	N	Y	Perencanaan Uji	Katalog personel bersertifikat pengujian lab
14	lims_prod_db	lims.material_categories	N	N	Y	Master Data	Kategori material objek pengujian
15	lims_prod_db	lims.menus	N	N	Y	Auth & Access Control	Master rute menu navigasi aplikasi
16	lims_prod_db	lims.methodologies	N	N	Y	Penilaian & Scoring	Master metodologi panduan pelaksanaan uji
17	lims_prod_db	lims.models	N	N	Y	Master Data	Master data model dari brand
18	lims_prod_db	lims.ocr_score_mappings	N	N	N	Rekomendasi AI	Pemetaan skor dari hasil pembacaan OCR AI
19	lims_prod_db	lims.origins	N	N	Y	Master Data	Negara/asal usul material

No	Database	Nama Tabel	Partisi (Y/N)	Archive (Y/N)	History (Y/N)	Modul	Fungsi
20	lims_prod_db	lims.package_active_aspects	N	N	Y	Konfigurasi Paket	Relasi aspek aktif dalam paket uji
21	lims_prod_db	lims.package_active_sub_aspects	N	N	Y	Konfigurasi Paket	Relasi sub-aspek aktif dalam paket uji
22	lims_prod_db	lims.package_methodologies	N	N	N	Konfigurasi Paket	Relasi metodologi per paket pengujian
23	lims_prod_db	lims.partner_types	N	N	Y	Registrasi & Pengajuan	Master tipe rekanan (Pemerintah, Swasta, dll)
24	lims_prod_db	lims.partners	N	N	Y	Registrasi & Pengajuan	Direktori kontak klien / mitra pengujian
25	lims_prod_db	lims.payments	Y	Y	N	Penagihan (Billing)	Entri transaksi pembayaran dan validasinya
26	lims_prod_db	lims.provinces	N	N	Y	Master Data	Master data provinsi
27	lims_prod_db	lims.registrations_counters	N	N	N	Registrasi & Pengajuan	Penomoran otomatis pendaftaran aplikasi uji
28	lims_prod_db	lims.reimbursement_counters	N	N	N	Keuangan & Perjalanan	Penomoran otomatis klaim reimbursement
29	lims_prod_db	lims.reimbursements	Y	Y	N	Keuangan & Perjalanan	Catatan laporan penggantian biaya dan nota struk

No	Database	Nama Tabel	Partisi (Y/N)	Archive (Y/N)	History (Y/N)	Modul	Fungsi
30	lims_prod_db	lims.role_menus	N	N	Y	Auth & Access Control	Tabel jembatan hak akses menu ke role
31	lims_prod_db	lims.roles	N	N	Y	Auth & Access Control	Data level otoritas (Admin, Analis, dsb)
32	lims_prod_db	lims.scoring_aspects	N	N	Y	Penilaian & Scoring	Master klasifikasi aspek penilaian standar (Level 3)
33	lims_prod_db	lims.scoring_levels	N	N	Y	Penilaian & Scoring	Rubrik ambang batas kelas kelulusan akhir (Level 5)
34	lims_prod_db	lims.scoring_sub_aspect_items	N	N	Y	Penilaian & Scoring	Daftar item pengecekan detail parameter fisik
35	lims_prod_db	lims.scoring_sub_aspects	N	N	Y	Penilaian & Scoring	Master parameter detail fisik yang diukur (Level 4)
36	lims_prod_db	mecs.simulator_data_logs	Y	Y	N	Integrasi IoT	Perekaman payload data <i>real-time</i> dari alat luar
37	lims_prod_db	lims.status_applications	N	N	Y	Master Data	Master status tahapan

No	Database	Nama Tabel	Partisi (Y/N)	Archive (Y/N)	History (Y/N)	Modul	Fungsi
							aplikasi (Draft, Verified, dll)
38	lims_prod_db	lims.test_types	N	N	Y	Penilaian & Scoring	Level tertinggi penentuan jenis pengujian (Level 1)
39	lims_prod_db	lims.tester_applications	Y	Y	N	Perencanaan Uji	Tim penguji (Laboratorium / Lapangan) bertugas
40	lims_prod_db	lims.testing_applications	Y	Y	N	Registrasi & Pengajuan	Master transaksi pendaftaran pengujian
41	lims_prod_db	lims.testing_applications_audit	Y	Y	N	Registrasi & Pengajuan	Audit trail pendaftaran pengujian
42	lims_prod_db	lims.testing_aspect_scores	Y	Y	N	Penilaian & Scoring	Agregasi rata-rata nilai tertimbang per aspek uji
43	lims_prod_db	lims.testing equipments	Y	Y	N	Registrasi & Pengajuan	Rincian data alat fisik klien yang akan diuji
44	lims_prod_db	lims.testing_packages	N	N	Y	Konfigurasi Paket	Master direktori paket layanan pengujian
45	lims_prod_db	lims.testing_plans	Y	Y	N	Perencanaan Uji	Plotting jadwal dan rencana pelaksanaan uji

No	Database	Nama Tabel	Partisi (Y/N)	Archive (Y/N)	History (Y/N)	Modul	Fungsi
46	lims_prod_db	lims.testing_pqc_ai_anomalies	Y	Y	N	Rekomendasi AI	Catatan log anomali kualitas data oleh AI PQC
47	lims_prod_db	lims.testing_report_ais	Y	Y	N	Rekomendasi AI	Log ringkasan laporan teks <i>generate</i> LLM
48	lims_prod_db	lims.testing_results	Y	Y	N	Pelaksanaan Uji	Hasil pengukuran parameter fisik pengujian
49	lims_prod_db	lims.testing_tool_availability	Y	Y	N	Pemantauan Alat Uji	Informasi ketersediaan harian alat uji
50	lims_prod_db	lims.testing_tool_reservations	Y	Y	N	Pemantauan Alat Uji	Log penjadwalan/booking alat lab oleh tester
51	lims_prod_db	lims.testing_tool_transactions	Y	Y	N	Pemantauan Alat Uji	Rekaman historis pemakaian alat di lapangan
52	lims_prod_db	lims.testing_tools	N	N	Y	Pemantauan Alat Uji	Inventaris peralatan lab dan status kalibrasinya
53	lims_prod_db	lims.travel_requests_counters	N	N	N	Keuangan & Perjalanan	Penomoran otomatis pengajuan SPD
54	lims_prod_db	lims.travel_requests	Y	Y	N	Keuangan & Perjalanan	Pengajuan dan status Surat Perjalanan Dinas (SPD)

No	Database	Nama Tabel	Partisi (Y/N)	Archive (Y/N)	History (Y/N)	Modul	Fungsi
55	lims_prod_db	lims.user_activity_logs	Y	N	N	Auth & Access Control	Perekaman log navigasi dan aksi seluruh pengguna
56	lims_prod_db	lims.user_sessions	N	N	N	Auth & Access Control	Penyimpanan token JWT untuk manajemen sesi aktif
57	lims_prod_db	lims.users	N	N	Y	Auth & Access Control	Data utama pengguna aplikasi LIMS
58	lims_prod_db	lims.variants	N	N	Y	Master Data	Sub-model / varian dari objek material
59	chat_db	chat_sch.agent_chats	N	N	N	Chatbot RAG AI	Riwayat percakapan pengguna dengan asisten AI
60	chat_db	chat_sch.documents	N	N	N	Chatbot RAG AI	Metadata direktori PDF SOP teknis referensi
61	chat_db	chat_sch.document_chunks	N	N	N	Chatbot RAG AI	Vektor semantik (embeddings) dari potongan dokumen
62	chat_db	chat_sch.user_social_accounts	N	N	N	Chatbot RAG AI	Penyimpanan sesi otentikasi login chat agent

(Catatan: Total keseluruhan tabel fisik yang beroperasi di `lims_prod_db` adalah 106 tabel, dan di `chat_db` adalah 4 tabel. Hal ini dikarenakan setiap tabel yang ditandai dengan **History (Y)** akan memiliki tabel pendamping berawalan `hist_` (misalnya `hist_roles`), serta tabel yang ditandai **Partisi (Y)** dan **Archive (Y)** akan terus melahirkan pecahan sub-tabel secara dinamis di tingkat basis data, seperti `testing_applications_202606`, `testing_applications_202607`, dsb).

Mekanisme Arsip & Manajemen Partisi

Seluruh **20 tabel** yang memiliki indikator **Archive (Y)** di atas terintegrasi secara otomatis dengan fitur *Database Management* di aplikasi. Sistem pemeliharaan ini memungkinkan administrator untuk memindahkan data operasional yang berusia lebih dari batas tertentu (konfigurasi `DATA_ARCHIVE_THRESHOLD_MONTHS`, *default* 3 bulan) ke dalam partisi arsip (`_arc`) guna menjaga performa kueri basis data utama tetap optimal.

8. Panduan Deployment Produksi LIMS (Multi-Direktori Backend & Frontend)

Dokumen ini menjelaskan konfigurasi deployment **LIMS (Laboratory Information Management System)** di lingkungan produksi dengan arsitektur multi-direktori untuk pembagian beban kerja (*Load Balancing*) dan ketersediaan tinggi (*High Availability*).

Dalam panduan ini, semua berkas ditempatkan langsung di bawah direktori *user* (`/home/lims`):

- **Frontend 1 (Port 3000):** `/home/lims/lims1/frontend`
- **Frontend 2 (Port 3001):** `/home/lims/lims2/frontend`
- **Backend 1 (Port 8081):** `/home/lims/lims1/backend`
- **Backend 2 (Port 8091):** `/home/lims/lims2/backend`

A. Arsitektur Produksi (Multi-Direktori vs Multi-VM)

Skenario 1: Multi-Direktori di 1 VM (Skenario Saat Ini)

Jika semua server berjalan di **1 VM yang sama**, direktori `/uploads` dipisahkan menjadi folder bersama (*Shared Storage*) lokal di `/home/lims/shared_uploads` dan dihubungkan via **Symbolic Link** ke masing-masing folder backend.

Skenario 2: Multi-VM (2 VM Terpisah dengan Port Sama, misal 8081)

Jika Anda menggunakan **2 VM terpisah**, disk lokal kedua VM tersebut secara fisik terpisah. Anda **tidak bisa** hanya menggunakan perintah `mkdir` dan `ln -s` lokal karena file di VM 1 tidak akan terbaca di VM 2.

Solusi untuk Skenario Multi-VM:

1. **NFS (Network File System) [Rekomendasi Disk Share]:** Membuat folder `shared_uploads` di satu storage server (atau di VM 1), lalu me-mount folder tersebut menggunakan protokol NFS ke VM 2 pada path `/home/lims/shared_uploads`. Setelah di-mount, symbolic link `ln -s` lokal akan bekerja seolah-olah folder tersebut berada di satu mesin.
2. **Object Storage (S3 / MinIO) [Rekomendasi Cloud/Modern]:** Mengubah konfigurasi kode Go backend agar langsung mengunggah file ke Object Storage (seperti MinIO server atau AWS S3) via API, bukan menyimpan ke disk lokal. Dengan cara ini, VM 1 and VM 2 tidak perlu berbagi filesystem lokal sama sekali.
3. **GoAccess shared_reports di Multi-VM:** Setiap VM akan memiliki file log NGINX lokal masing-masing. Anda **tidak perlu** membagikan folder `shared_reports` antar VM. Biarkan setiap VM melayani `report.html` lokalnya sendiri untuk melihat trafik masing-masing VM.

Parse error on line 13:

```
... BE1 & BE2 -.-->|Symlink ke /uploads
-----^
```

Expecting 'TAGEND', 'STR', 'MD_STR', 'UNICODE_TEXT', 'TEXT', 'TAGSTART', got 'PIPE'

B. Prasyarat Sistem & Manajemen Pengguna Linux

Sebelum memulai deployment, Anda harus membuat pengguna (*user*) dan grup (*group*) Linux yang sesuai serta mengatur hak akses direktori agar NGINX (user *www-data*) dan layanan sistem backend/frontend dapat saling mengakses berkas.

Membuat User dan Group Linux

Untuk keamanan produksi, semua proses backend, frontend (PM2), dan crontab dijalankan di bawah pengguna (*user*) dan grup (*group*) **lims**.

Jalankan perintah berikut untuk membuat pengguna dan grup tersebut serta menetapkan password-nya:

```
# Membuat grup lims
```

```
sudo groupadd lims
```

```
# Membuat user lims dengan grup utama lims
```

```
sudo useradd -m -g lims -s /bin/bash lims
```

```
# Menetapkan password untuk user lims (Masukkan password baru saat diminta)
```

```
sudo passwd lims
```

Struktur Direktori lims1 dan lims2

Untuk menjalankan deployment dual-instance, buat struktur direktori di bawah */home/lims* dengan menjalankan perintah berikut sebagai pengguna *lims* (atau gunakan *sudo* dan ubah kepemilikannya):

```
# 1. Membuat direktori untuk instansi lims1
```

```
mkdir -p /home/lims/lims1/backend/ai_service
```

```
mkdir -p /home/lims/lims1/backend/lib
```

```
mkdir -p /home/lims/lims1/backend/logs
```

```
mkdir -p /home/lims/lims1/backend/public
```

```
mkdir -p /home/lims/lims1/backend/uploads
```

```
mkdir -p /home/lims/lims1/frontend
```

```
# 2. Membuat direktori untuk instansi lims2
```

```
mkdir -p /home/lims/lims2/backend/ai_service
```

```
mkdir -p /home/lims/lims2/backend/lib
```

```
mkdir -p /home/lims/lims2/backend/logs
```

```
mkdir -p /home/lims/lims2/backend/public
```

```
mkdir -p /home/lims/lims2/backend/uploads
```

```
mkdir -p /home/lims/lims2/frontend
```

```
# 3. Mengatur kepemilikan seluruh folder ke pengguna lims
```

```
sudo chown -R lims:lims /home/lims/lims1
```

```
sudo chown -R lims:lims /home/lims/lims2
```

Berikut adalah penjelasan isi dari struktur direktori tersebut:

- /home/lims/lims[1/2]/frontend/: Tempat meletakkan berkas *build* aplikasi React frontend.
- /home/lims/lims[1/2]/backend/: Folder utama Go backend instance.
 - main: File binary executables Go backend hasil kompilasi.
 - paddle_ocr.py: Script python integrasi OCR.
 - ai_service/: Menyimpan konfigurasi dan berkas pendukung model AI.
 - lib/: Tempat meletakkan shared library C++ (libonnxruntime.so).
 - logs/: Tempat log aplikasi backend disimpan.
 - public/: Folder aset statis backend, termasuk berkas simulator (hardware_simulator.html).
 - uploads/: Folder penampungan unggahan lokal sementara.

Konfigurasi Izin Direktori (Permissions)

Berikan izin baca dan masuk (*execute*) ke dalam folder home lims agar NGINX (www-data) dapat menyajikan berkas di dalam subfolder bersama:

Berikan hak akses masuk ke folder home lims

```
sudo chmod 755 /home/lims
```

Buat folder bersama untuk berkas unggahan, laporan analitik, & unduhan APK

```
mkdir -p /home/lims/shared_uploads
```

```
mkdir -p /home/lims/shared_reports
```

```
mkdir -p /home/lims/shared_downloads
```

```
sudo chmod -R 755 /home/lims/shared_uploads
```

```
sudo chmod -R 755 /home/lims/shared_reports
```

```
sudo chmod -R 755 /home/lims/shared_downloads
```

Berikan kepemilikan folder bersama ke user 'lims' dan grup 'www-data' (agar Nginx bisa membaca/menulis)

```
sudo chown -R lims:www-data /home/lims/shared_uploads
```

```
sudo chown -R lims:www-data /home/lims/shared_reports
```

```
sudo chown -R lims:www-data /home/lims/shared_downloads
```

C. Port Mapping & Manajemen Konflik Port

Untuk mencegah kegagalan startup layanan akibat port yang tabrakan (*port conflict*), pastikan port-port berikut tersedia sebelum dijalankan.

Daftar Port Ekosistem LIMS

Komponen	Nama Layanan	Protokol	Port Default	Status Akses
Nginx HTTP	HTTP Redirect	TCP	8088 (Redirect ke HTTPS)	Publik / Eksternal
Nginx HTTPS	Gateway Utama	TCP	8082 (atau 443 di prod asli)	Publik / Eksternal
Nginx Telegram	Webhook Gateway	TCP	8443	Publik / Eksternal
Frontend 1	PM2 Node.js Instance 1	TCP	3000	Internal (Lokal)

Komponen	Nama Layanan	Protokol	Port Default	Status Akses
Frontend 2	PM2 Node.js Instance 2	TCP	3001	Internal (Lokal)
Backend 1	Go Backend Instance 1	TCP	8081	Internal (Lokal)
Backend 2	Go Backend Instance 2	TCP	8091	Internal (Lokal)
GoAccess WS	Real-time WebSocket	TCP	7890	Internal / Nginx Proxy
Database	PostgreSQL	TCP	5433 (atau 5432 standar)	Internal (Lokal)
Object Storage	MinIO API	TCP	9000	Internal / Nginx Proxy
MinIO UI	MinIO Console UI	TCP	9001	Eksternal (Opsional)
Workflow Engine	Camunda BPM Engine	TCP	8085	Internal / Nginx Proxy

Perintah Memeriksa Penggunaan Port (Single & Multiple)

Gunakan perintah berikut untuk memverifikasi apakah ada port yang sedang digunakan:

1. Memeriksa Port Tunggal (misal port 8081)

```
sudo ss -tulpn | grep :8081
```

atau menggunakan lsof

```
sudo lsof -i :8081
```

2. Memeriksa Banyak Port Sekaligus (Multiple Ports)

Memeriksa seluruh ekosistem port LIMS

```
sudo ss -tulpn | grep -E
```

```
'8088|8082|8443|3000|3001|8081|8091|7890|5433|9000|9001|8085'
```

Menghentikan/Membunuh Proses pada Port yang Bermasalah (Stop/Kill Port)

Jika suatu port terkunci oleh proses lama yang menggantung (*zombie process*), Anda dapat membunuhnya dengan cara:

1. Mendapatkan PID (Process ID) dari port yang terkunci

```
sudo lsof -t -i :8081
```

2. Membunuh proses berdasarkan nomor port (Menggunakan fuser)

```
sudo fuser -k 8081/tcp
```

3. Membunuh proses secara paksa (Force Kill) berdasarkan lsof PID

```
sudo kill -9 $(sudo lsof -t -i :8081)
```

Menghentikan Layanan Secara Permanen (Untuk Kebutuhan Development/Pengembangan)

Jika Anda ingin mematikan semua layanan background secara permanen agar port-port tersebut kosong (misalnya selama development agar bisa menjalankan go run . di dalam folder backend), jalankan perintah berikut:

1. Stop & Disable Go Backend (Systemd)

Menghentikan service secara instan

```
sudo systemctl stop lims-backend-8081.service lims-backend-8091.service
```

Mencegah service berjalan otomatis saat boot

```
sudo systemctl disable lims-backend-8081.service lims-backend-8091.service
```

2. Stop & Hapus PM2 Frontend (PM2)

Menghentikan semua aplikasi frontend di PM2

```
sudo -u lims pm2 stop all
```

Menghapus daftar aplikasi dari memori PM2

```
sudo -u lims pm2 delete all
```

Menyimpan konfigurasi kosong agar PM2 tidak memuat ulang aplikasi saat reboot

```
sudo -u lims pm2 save
```

Mematikan daemon PM2 secara total

```
sudo -u lims pm2 kill
```

3. Stop & Disable NGINX (Systemd)

Menghentikan NGINX dan mencegahnya berjalan otomatis saat boot

```
sudo systemctl stop nginx
```

```
sudo systemctl disable nginx
```

D. Deployment & Konfigurasi Frontend (PM2) di VPS

Alur Kompilasi & Pemindahan File

Proses instalasi dependensi (npm install) dan kompilasi/build (npm run build) dilakukan di dalam folder kode sumber lokal (source code) Anda,

yaitu: /mnt/d/Data_NK/Project5/AI/LIM_System_Linux_OK/frontend/

1. Struktur Folder Frontend (Source Code):

- src/ (Komponen, state, dan logika React)
- public/ (Aset publik mentah)
- node_modules/ (Library dependensi)
- package.json & package-lock.json
- vite.config.js
- dist/ (Folder hasil kompilasi statis)

2. Menjalankan Kompilasi:

Pindah ke folder kode sumber

```
cd /mnt/d/Data_NK/Project5/AI/LIM_System_Linux_OK/frontend/
```

Jalankan instalasi & build

```
npm install
npm run build
```

3. Berkas yang Dickey ke /home/lims/:

Karena kita menggunakan serve untuk menyajikan web statis, **HANYA isi folder dist/** yang perlu dicopy ke target deployment. Folder src/, node_modules/, dan file konfigurasi JavaScript lainnya **tidak perlu dicopy**.

Buat folder tujuan di server

```
mkdir -p /home/lims/lims1/frontend/dist
mkdir -p /home/lims/lims2/frontend/dist
```

Salin HANYA isi folder 'dist' ke folder tujuan

```
cp -r /mnt/d/Data_NK/Project5/AI/LIM_System_Linux_OK/frontend/dist/*
/home/lims/lims1/frontend/dist/
cp -r /mnt/d/Data_NK/Project5/AI/LIM_System_Linux_OK/frontend/dist/*
/home/lims/lims2/frontend/dist/
```

Jalankan Kluster dengan PM2 di VPS

Untuk menjalankan frontend LIMS di server produksi VPS, Anda memiliki dua opsi. Menggunakan berkas konfigurasi ecosystem.config.js sangat direkomendasikan karena rapi, terstruktur, dan mudah dikelola dalam satu perintah.

📁 Persyaratan Awal di VPS:

Pastikan paket pm2 dan HTTP server statis serve sudah terpasang secara global di VPS Anda:

```
sudo npm install -g pm2 serve
```

Opsi A: Menggunakan Berkas Konfigurasi ecosystem.config.js (Sangat Direkomendasikan)

1. Buat berkas konfigurasi PM2 baru di folder /home/lims/ VPS Anda:
2. nano /home/lims/ecosystem.config.js
3. Salin dan tempelkan (*paste*) konfigurasi kluster di bawah ini:
4. module.exports = {
5. apps: [
6. {
7. name: 'lims-frontend-3000',
8. script: 'serve',
9. args: '-s /home/lims/lims1/frontend/dist -l 3000',
10. instances: 1,
11. autorestart: true,
12. watch: false,
13. max_memory_restart: '1G',
14. env: {
15. NODE_ENV: 'production'
16. }
17. },
18. {
19. name: 'lims-frontend-3001',
20. script: 'serve',
21. args: '-s /home/lims/lims2/frontend/dist -l 3001',

```

22.   instances: 1,
23.   autorestart: true,
24.   watch: false,
25.   max_memory_restart: '1G',
26.   env: {
27.     NODE_ENV: 'production'
28.   }
29. }
30. ]
31. };

```

(Simpan perubahan berkas dengan menekan *Ctrl + O*, *Enter*, lalu *Ctrl + X*).

32. Jalankan seluruh kluster frontend LIMS dengan satu perintah mudah:

33. `pm2 start /home/lims/ecosystem.config.js`

Opsi B: Menggunakan Perintah CLI Langsung

Jalankan perintah `pm2` di bawah user **lims** secara terpisah untuk memicu masing-masing server instansi:

Menjalankan Server Frontend 1 (Port 3000)

`pm2 start "serve -s /home/lims/lims1/frontend/dist -l 3000" --name "lims-frontend-3000"`

Menjalankan Server Frontend 2 (Port 3001)

`pm2 start "serve -s /home/lims/lims2/frontend/dist -l 3001" --name "lims-frontend-3001"`

Tip Operasional: Setelah menjalankan PM2, jalankan perintah `pm2 save` untuk menyimpan daftar proses aktif agar otomatis menyala kembali ketika server VPS direboot.

Daftar Perintah PM2 (Cheat Sheet Operasional)

Gunakan perintah-perintah berikut untuk mengelola layanan frontend:

- **Melihat Status Layanan (Status):**
 - `pm2 list`
 - *# atau*
 - `pm2 status`
- **Memantau Metrik secara Live (Monitoring):**
 - `pm2 monit`
- **Melihat Log Aplikasi (Logs):**
 - *# Log semua aplikasi*
 - `pm2 logs`
 - *# Log khusus layanan tertentu*
 - `pm2 logs lims-frontend-3000`
- **Menghentikan Layanan (Stop):**
 - *# Menghentikan satu layanan*
 - `pm2 stop lims-frontend-3000`
 - *# Menghentikan semua layanan*
 - `pm2 stop all`
- **Menyalakan/Merestart Layanan (Restart):**
 - *# Restart satu layanan*
 - `pm2 restart lims-frontend-3000`
 - *# Restart semua layanan*

- pm2 restart all

E. Deployment & Konfigurasi Backend (Multi-Direktori) di VPS

Alur Kompilasi & Pemindahan File

Proses kompilasi Go backend dilakukan di folder kode

sumber: /mnt/d/Data_NK/Project5/AI/LIM_System_Linux_OK/backend/

1. Struktur Folder Backend (Source Code):

- controllers/, database/, middleware/, models/, routes/, services/, utils/ (Seluruh kode logika Go)
- lib/ (**Library pendukung C++ untuk AI ONNX Runtime - libonnxruntime.so**)
- ai_service/ (**Model AI .onnx dan metadata statistik**)
- paddle_ocr.py (Script Python untuk AI-OCR)
- go.mod & go.sum
- main.go (Entrypoint aplikasi)

2. Menjalankan Kompilasi (Build):

Karena Go adalah bahasa terkompilasi (*compiled language*), seluruh kode logika Go di folder-folder di atas akan disatukan ke dalam satu file biner bernama main.

Masuk ke direktori kode sumber backend

```
cd /mnt/d/Data_NK/Project5/AI/LIM_System_Linux_OK/backend/
```

Lakukan build biner Linux (Gunakan "." agar semua file dalam package dikompilasi, termasuk onnx_init_linux.go)

```
GOOS=linux GOARCH=amd64 go build -ldflags="-s -w" -o main .
```

3. Berkas yang Dicopy ke /home/lims/:

Anda **TIDAK** perlu menyalin folder kode sumber .go (seperti controllers, models, dll).

Cukup salin berkas-berkas berikut:

1. Biner hasil build (main)
2. File .env produksi (Pastikan MINIO_* variables sudah diatur untuk modul SPD/Reimbursement)
3. Folder lib/ (untuk AI ONNX)
4. Folder ai_service/ (jika berisi file model .onnx)
5. Script paddle_ocr.py (untuk OCR)
6. *Optional (Alat Bantu)*: generate_db_password.go dan generate_bcrypt_hash.go (untuk generate password di VPS)

[!TIP] **Mempertahankan Timestamp File (Preserve Timestamps)** Saat Anda menyalin file biner main ke server VPS (Production), sangat disarankan untuk mempertahankan stempel waktu (waktu modifikasi) aslinya menggunakan *flag* khusus agar Anda dapat melacak kapan versi tersebut dikompilasi secara presisi.

Opsi A: Menggunakan rsync (Direkomendasikan) Gunakan *flag* -vt (verbose, times) atau -av (archive, verbose):

```
rsync -av /mnt/d/Data_NK/Project5/AI/LIM_System_Linux_OK/backend/main
```

```
lims@<IP_VPS>:/home/lims/lims1/backend/
```

```
rsync -av /mnt/d/Data_NK/Project5/AI/LIM_System_Linux_OK/backend/main
```

```
lims@<IP_VPS>:/home/lims/lims2/backend/
```

Opsi B: Menggunakan scp Gunakan atribut -p (preserve modification times):

```
scp -p /mnt/d/Data_NK/Project5/AI/LIM_System_Linux_OK/backend/main  
lims@<IP_VPS>:/home/lims/lims1/backend/  
(Jika Anda melakukan deployment langsung di dalam OS yang sama / WSL, Anda bisa  
menggunakan perintah cp -p):
```

```
mkdir -p /home/lims/lims1/backend  
mkdir -p /home/lims/lims2/backend
```

Salin berkas biner, library, dan konfigurasi

```
cp -p /mnt/d/Data_NK/Project5/AI/LIM_System_Linux_OK/backend/main  
/home/lims/lims1/backend/
```

```
cp -p /mnt/d/Data_NK/Project5/AI/LIM_System_Linux_OK/backend/main  
/home/lims/lims2/backend/
```

```
cp -r /mnt/d/Data_NK/Project5/AI/LIM_System_Linux_OK/backend/lib  
/home/lims/lims1/backend/
```

```
cp -r /mnt/d/Data_NK/Project5/AI/LIM_System_Linux_OK/backend/lib  
/home/lims/lims2/backend/
```

```
cp -r /mnt/d/Data_NK/Project5/AI/LIM_System_Linux_OK/backend/ai_service  
/home/lims/lims1/backend/
```

```
cp -r /mnt/d/Data_NK/Project5/AI/LIM_System_Linux_OK/backend/ai_service  
/home/lims/lims2/backend/
```

```
cp /mnt/d/Data_NK/Project5/AI/LIM_System_Linux_OK/backend/paddle_ocr.py  
/home/lims/lims1/backend/
```

```
cp /mnt/d/Data_NK/Project5/AI/LIM_System_Linux_OK/backend/paddle_ocr.py  
/home/lims/lims2/backend/
```

Salin utilitas pembantu password

```
cp  
/mnt/d/Data_NK/Project5/AI/LIM_System_Linux_OK/backend/generate_db_password.g  
o /home/lims/lims1/backend/
```

```
cp  
/mnt/d/Data_NK/Project5/AI/LIM_System_Linux_OK/backend/generate_bcrypt_hash.go  
/home/lims/lims1/backend/
```

Instalasi Dependensi Python & Mesin OCR (PaddleOCR / Tesseract)

Fitur AI_OCR_ENABLED membutuhkan instalasi modul eksternal Python dan pustaka pembaca gambar di OS VPS.

PERINGATAN STANDAR PRODUCTION: Karena OS server rilis terbaru (seperti Ubuntu 24.04 atau yang lebih baru) umumnya datang dengan versi Python mutakhir (3.13 / 3.14) yang seringkali **tidak kompatibel** dengan beberapa pustaka Kecerdasan Buatan (AI), Anda **SANGAT DIWAJIBKAN** untuk membungkus instalasi mesin AI-nya menggunakan lingkungan yang terisolasi (*Virtual Environment*) dengan **Python 3.12**. Jalankan runtutan perintah ini di terminal VPS Anda:

1. Install Pustaka Sistem OS (PDF, Tesseract OCR, Dependensi Grafik OpenCV, dan Compiler C++)

Catatan: build-essential, ccache, dan ninja-build SANGAT diwajibkan agar model C++ PaddleX tidak mengalami crash!

```
sudo apt update
```

```
sudo apt install -y software-properties-common poppler-utils tesseract-ocr tesseract-ocr-ind tesseract-ocr-eng libgl1 libgl2-mesa-dev build-essential ccache ninja-build
```

2. Tambahkan PPA Deadsnakes & Install Python 3.12

(Ini tidak merusak Python utama bawaan Ubuntu, ia hanya menambahkannya secara berdampingan)

```
sudo add-apt-repository ppa:deadsnakes/ppa -y
```

```
sudo apt update
```

```
sudo apt install python3.12 python3.12-venv -y
```

3. Buat Kotak Isolasi (Venv) khusus AI di dalam folder backend LIMS

(PENTING: Pastikan Anda menjalankan ini DENGAN AKUN BIASA, JANGAN GUNAKAN SUDO!)

```
python3.12 -m venv venv_ocr
```

3.1. [TROUBLESHOOTING] Jika Anda tidak sengaja menggunakan sudo di atas, # kembalikan hak milik folder venv ke akun Anda agar pip tidak error "Permission denied"

```
# sudo chown -R lims:lims venv_ocr
```

4. Masuk ke dalam Venv tersebut dan pasang semua library yang dibutuhkan

(SEKALI LAGI: JANGAN GUNAKAN SUDO SAAT MENJALANKAN PIP DI DALAM VENV)

```
source venv_ocr/bin/activate
```

```
pip install --upgrade pip
```

```
pip install Pillow numpy paddlepaddle paddleocr
```

```
deactivate
```

Info - Fitur Pencarian Otomatis & Format File (Update Terbaru):

- **Format File yang Didukung OCR:** Sistem kini mendukung unggahan multi-format.
 - **Gambar (.jpg, .png) dan .pdf:** Akan diproses secara digital (jika layout PDF dipertahankan) atau menggunakan modul ML cerdas (Tesseract/PaddleOCR) untuk membaca karakter visual (memakan waktu ekstra).
 - **Teks Murni (.txt, .csv, .log):** Sistem akan **mengabaikan** proses *machine learning* dan langsung mengekstrak teksnya secara harfiah. Ini membuat proses 100x lebih cepat (sangat disarankan jika data berasal dari log alat).

Panduan Penulisan Format Teks (.txt, .csv, .log): Sistem LIMS dirancang sangat toleran karena menggunakan *parser* yang sama dengan mesin OCR gambar. Syarat utamanya: teks tersebut harus mengandung **Kode Parameter** (kapital) dan **Nilainya** di baris yang sama.

- **Contoh Teks Sederhana:** (Gunakan spasi, titik dua, atau sama dengan) KONS1
1 KOPEN: 2 KOKON=3

- **Contoh CSV / Tabel Excel:** Parameter, Kode, Hasil Konsistensi Tanah, KONSI, 1 Kondisi Fisik, KOKON, 3
- **Contoh Log Alat yang Berantakan:** [06:00:00] Memulai alat. Hasil uji KOPEN adalah 2 (Normal). (*Sistem akan otomatis mengabaikan teks lain dan mengekstrak KOPEN = 2*)

Peringatan Kamus OCR Global (General OCR Score Mappings): LIMS memiliki fitur untuk memperbaiki kesalahan baca OCR melalui kamus/mapping OCR (di dalam menu Pengaturan Master Data). Terdapat dua tingkatan *mapping*:

1. **Mapping Global (ocr_score_mappings):** Segala aturan yang dimasukkan ke sini akan berlaku **untuk SEMUA parameter uji tanpa terkecuali**. Hindari membuat aturan angka-ke-angka (misal: "91 diubah ke 16") karena jika parameter lain benar-benar mendapat skor "91" (misal KOFAS 91), skor tersebut akan dihancurkan/diganti paksa menjadi 16.
 2. **Mapping Konteks (Hardcoded & Parameter-Specific):** Sistem sudah diprogram agar secara pintar mengenali kesalahan (misal 91 jadi 16) **HANYA** jika parameter tersebut memang mengharapkan nilai 16 (misal parameter PEHAN). Jika Anda masih butuh memodifikasi nilai di luar bawaan, gunakan kolom OCR Keywords pada parameter (Sub-Aspect) spesifik dengan format keywords | nilaiAsal:nilaiBaru agar tidak menimbulkan efek berantai pada parameter uji lainnya.
- **Auto-Discovery Virtual Environment:** LIMS Go dilengkapi pelacakan otomatis (auto-detect) untuk mencari folder venv_ocr/bin/python di dalam aplikasi lokal Anda. Anda **TIDAK PERLU** melakukan pengaturan PYTHON_CMD di file konfigurasi .env.
 - **Penggunaan Parameter PYTHON_CMD di .env (Override):** Parameter PYTHON_CMD=/path/to/python di dalam .env merupakan fitur pengecualian (*manual override*). Jika Anda mengaktifkan konfigurasi ini (menghapus tanda #), sistem **mematikan** fitur *auto-detect* dan wajib merujuk ke path absolut tersebut. **JANGAN MENGGUNAKAN** parameter ini kecuali Anda meletakkan folder *venv* jauh di luar instalasi standar LIMS.

Langkah Verifikasi (Pengecekan): Jika Anda menemui kendala terkait versi Python di kemudian hari, Anda dapat mengecek lokasi *Virtual Environment* dan versi Python yang tertanam di dalamnya menggunakan perintah berikut:

```
# Mengecek letak direktori venv_ocr (atau venv) berada di VPS
find /home/lims -type d -name "venv_ocr" -o -name "venv" 2>/dev/null
```

```
# Mengecek versi Python yang terinstal di dalam venv tersebut
# (Ganti path-nya sesuai hasil temuan dari perintah find di atas)
/home/lims/venv_ocr/bin/python -V
# Pastikan outputnya menampilkan Python 3.12.x atau 3.10.x, BUKAN 3.14.x
[!WARNING]
```

Konfigurasi Timeout Nginx Wajib untuk OCR

Proses inferensi PaddleOCR untuk satu halaman penuh dapat memakan waktu 20 hingga 40 detik. Karena Nginx memutus koneksi secara *default* dalam 60 detik, Anda **DIWAJIBKAN** menambahkan konfigurasi *timeout* di blok server Nginx Anda

untuk menghindari pesan error HTML 504 Gateway Timeout di *frontend*.

Tambahkan baris berikut ke dalam `/etc/nginx/sites-available/default` di blok *port API*

Anda (misal `listen 8082 ssl;`):

```
proxy_read_timeout 300s;
```

```
proxy_connect_timeout 300s;
```

```
proxy_send_timeout 300s;
```

Hubungkan Symbolic Link (Shared Storage)

Guna menyatukan folder penyimpanan hasil uji uploads, hubungkan folder uploads lokal ke folder bersama `shared_uploads`:

```
# Hapus folder uploads lokal bawaan jika ada
```

```
rm -rf /home/lims/lims1/backend/uploads
```

```
rm -rf /home/lims/lims2/backend/uploads
```

```
# Hubungkan symbolic link
```

```
ln -s /home/lims/shared_uploads /home/lims/lims1/backend/uploads
```

```
ln -s /home/lims/shared_uploads /home/lims/lims2/backend/uploads
```

Membuat Systemd Services

Karena biner berjalan di bawah user `lims`, Anda wajib mengeset nilai **User=lims** dan **Group=lims** di `systemd` service.

1. **Service Port 8081** (`/etc/systemd/system/lims-backend-8081.service`):

2. [Unit]

3. Description=LIMS Go Backend Service (Port 8081)

4. After=network.target postgresql.service

- 5.

6. [Service]

7. Type=simple

8. User=lims

9. Group=lims

10. WorkingDirectory=/home/lims/lims1/backend

11. ExecStart=/home/lims/lims1/backend/main

12. Restart=always

13. RestartSec=5

14. LimitNOFILE=65535

15. EnvironmentFile=/home/lims/lims1/backend/.env

- 16.

17. [Install]

18. WantedBy=multi-user.target

19. **Service Port 8091** (`/etc/systemd/system/lims-backend-8091.service`):

20. [Unit]

21. Description=LIMS Go Backend Service (Port 8091)

22. After=network.target postgresql.service

- 23.

24. [Service]

25. Type=simple

26. User=lims

27. Group=lims

28. WorkingDirectory=/home/lims/lims2/backend
29. ExecStart=/home/lims/lims2/backend/main
30. Restart=always
31. RestartSec=5
32. LimitNOFILE=65535
33. EnvironmentFile=/home/lims/lims2/backend/.env
- 34.
35. [Install]
36. WantedBy=multi-user.target

Aktifkan dan jalankan kedua backend:

Memuat ulang konfigurasi systemd (wajib jika file .service baru dibuat atau diubah)

sudo systemctl daemon-reload

Mengaktifkan agar otomatis jalan saat boot & langsung menyalakan service

sudo systemctl enable --now lims-backend-8081.service lims-backend-8091.service

Daftar Perintah Systemd (Cheat Sheet Operasional Backend)

Gunkan perintah-perintah berikut untuk mengelola layanan backend LIMS di server produksi Anda:

- **Melihat Status Layanan (Status):**
- *# Mengecek status dan logs ringkas Backend 8081*
- sudo systemctl status lims-backend-8081.service
-
- *# Mengecek status dan logs ringkas Backend 8091*
- sudo systemctl status lims-backend-8091.service
- **Menghentikan Layanan (Stop):**
- *# Menghentikan salah satu service*
- sudo systemctl stop lims-backend-8081.service
-
- *# Menghentikan kedua service sekaligus*
- sudo systemctl stop lims-backend-8081.service lims-backend-8091.service
- **Menyalakan Layanan (Start):**
- *# Menyalakan kembali service*
- sudo systemctl start lims-backend-8081.service lims-backend-8091.service
- **Merestart Layanan (Restart):**
- *# Merestart layanan (biasa dilakukan saat ada update file biner 'main' atau berkas '.env' diubah)*
- sudo systemctl restart lims-backend-8081.service lims-backend-8091.service
- **Menonaktifkan Autostart saat Boot (Disable):**
- *# Page log status (proses tidak akan berjalan otomatis saat server direboot)*
- sudo systemctl disable lims-backend-8081.service lims-backend-8091.service

F. Konfigurasi NGINX di VPS

Edit Konfigurasi Utama (/etc/nginx/nginx.conf)

Tambahkan pemetaan Client IP asli dan deklarasi log monitoring di dalam blok http { ... }:

```
map $http_x_forwarded_for $real_client_ip {
    ""    $remote_addr;
```

```

    default $http_x_forwarded_for;
}

log_format upstream_monitoring '$remote_addr - ClientIP: $real_client_ip -
[$time_local] '
    "$request" $status $body_bytes_sent '
    'to_server=$upstream_addr status=$upstream_status '
    'resp_time=$upstream_response_time '
    'agent="$http_user_agent" '
    'app_ver="$http_x_app_version" app_plat="$http_x_app_platform";

```

Konfigurasi Situs LIMS (/etc/nginx/conf.d/lims.conf)

Salin konfigurasi berikut untuk menangani pemisahan static assets, load-balancing backend, dan direktori bersama /uploads/:

```

#
=====
# FILE CONFIGURATION: /etc/nginx/conf.d/lims.conf
#
=====

# --- CLUSTER LOAD BALANCER BACKEND ---
# Mendistribusikan request API ke dua instance Go backend (Port 8081 & 8091)
upstream lims_backend_cluster {
    server 127.0.0.1:8081 max_fails=3 fail_timeout=10s;
    server 127.0.0.1:8091 max_fails=3 fail_timeout=10s;
}

# --- CLUSTER LOAD BALANCER FRONTEND ---
# Mendistribusikan request Frontend ke dua instance PM2 (Port 3000 & 3001)
upstream lims_frontend_cluster {
    server 127.0.0.1:3000 max_fails=3 fail_timeout=10s;
    server 127.0.0.1:3001 max_fails=3 fail_timeout=10s;
}

#
=====
# BLOK SERVER 1: PORT 8082 (HTTPS - WEB CLIENT & SECURE ACCESS)
#
=====
server {
    listen 8082 ssl;
    server_name lims-d4551821.nip.io lims.local localhost;

```

```

# --- SERTIFIKAT SSL ---
# Jika menggunakan Let's Encrypt (nip.io), sesuaikan path-nya:
ssl_certificate /etc/nginx/ssl/fullchain.pem;
ssl_certificate_key /etc/nginx/ssl/privkey.pem;
ssl_protocols TLSv1.2 TLSv1.3;
ssl_prefer_server_ciphers on;
ssl_ciphers HIGH:!aNULL:!MD5;

# --- PENANGANAN REAL IP ---
set_real_ip_from 127.0.0.1;
real_ip_header X-Forwarded-For;
real_ip_recursive on;

# Security Headers
add_header X-Frame-Options "SAMEORIGIN" always;
add_header X-XSS-Protection "1; mode=block" always;
add_header X-Content-Type-Options "nosniff" always;
add_header Referrer-Policy "no-referrer-when-downgrade" always;
add_header Content-Security-Policy "default-src 'self' http: https: data: blob: 'unsafe-inline' 'unsafe-eval';" always;

client_max_body_size 50M;

root /home/lims/lims1/frontend/dist;
index index.html;

# 1. Routing Frontend Web (Proxy ke Cluster PM2)
location / {
    proxy_pass http://lims_frontend_cluster;
    proxy_http_version 1.1;
    proxy_set_header Upgrade $http_upgrade;
    proxy_set_header Connection 'upgrade';
    proxy_set_header Host $host;
    proxy_cache_bypass $http_upgrade;
}

# 2. Halaman Laporan GoAccess (Dilindungi Autentikasi Dinamis Backend via
auth_request)
location = /report.html {
    # Mengirim sub-request ke endpoint backend untuk memeriksa JWT Session & Role
    (nur / ADMIN)
    auth_request /api/auth/check-report-access;

    # Jika backend mengembalikan HTTP 200 OK, sajikan file dari alias berikut:
    alias /home/lims/shared_reports/report.html;
}

```

2b. Redirect /report.html/ ke /report.html (Mengatasi isu trailing slash dari link)

```
location = /report.html/ {  
    return 301 $scheme://$http_host/report.html$sis_args$args;  
}
```

3. Lokasi Proxy Internal untuk Autentikasi Keamanan report.html

```
location = /api/auth/check-report-access {  
    internal; # Hanya bisa diakses secara internal oleh Nginx  
    proxy_pass http://lims_backend_cluster;
```

Optimasi performa: Jangan kirim body request ke backend

```
proxy_pass_request_body off;  
proxy_set_header Content-Length "";
```

Meneruskan header otentikasi dari klien

```
proxy_set_header X-Original-URI $request_uri;  
proxy_set_header Host $host;  
proxy_set_header X-Real-IP $remote_addr;  
proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;  
proxy_set_header X-Forwarded-Proto $scheme;  
}
```

4. Proxy WebSocket untuk GoAccess Real-Time

```
location /ws {  
    proxy_pass http://127.0.0.1:7890;  
    proxy_http_version 1.1;  
    proxy_set_header Upgrade $http_upgrade;  
    proxy_set_header Connection "Upgrade";  
}
```

5. Proxy API ke Go Backend Cluster

```
location /api {  
    proxy_pass http://lims_backend_cluster;  
    proxy_http_version 1.1;  
    proxy_set_header Upgrade $http_upgrade;  
    proxy_set_header Connection 'upgrade';  
    proxy_set_header Host $host;  
    proxy_cache_bypass $http_upgrade;
```

Meneruskan IP Client Asli

```
proxy_set_header X-Real-IP $remote_addr;  
proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;  
proxy_set_header X-Forwarded-Proto $scheme;
```

--- KONFIGURASI STREAMING AI & LONG-RUNNING REQUESTS ---

```

        gzip off;                # Matikan gzip kompresi Nginx khusus API agar stream SSE
tidak tertahan
        proxy_buffering off;      # Nonaktifkan buffering Nginx agar stream token AI
terkirim real-time
        proxy_cache off;         # Nonaktifkan caching respons API
        chunked_transfer_encoding on; # Aktifkan chunked transfer encoding untuk SSE
        proxy_connect_timeout 600s; # Timeout koneksi hingga 10 menit
        proxy_send_timeout 600s;   # Timeout pengiriman hingga 10 menit
        proxy_read_timeout 600s;   # Timeout membaca stream AI hingga 10 menit
    }

```

5b. Proxy Halaman Simulator ke Go Backend Cluster

```

location /simulator {
    proxy_pass http://lims_backend_cluster;
    proxy_http_version 1.1;
    proxy_set_header Upgrade $http_upgrade;
    proxy_set_header Connection 'upgrade';
    proxy_set_header Host $host;
    proxy_cache_bypass $http_upgrade;
}

```

6. Penyajian Berkas Unggahan (Direct Alias)

```

location /uploads/ {
    alias /home/lims/shared_uploads/;
    expires 30d;
    add_header Cache-Control "public, no-transform";
}

```

7. Penyajian Folder Unduhan APK LIMS (Direct Alias)

```

location /downloads/ {
    alias /home/lims/shared_downloads/;
    expires 30d;
    add_header Cache-Control "public, no-transform";
}

```

```

    access_log /var/log/nginx/lims_access.log upstream_monitoring;
    error_log /var/log/nginx/lims_error.log;
}

```

#

=====

BLOK SERVER 2: PORT 8087 (HTTP - WI-FI LOKAL MOBILE APP CLIENT)

Bebas dari kendala sertifikat SSL self-signed pada HP Android

```
#
=====
server {
    listen 8087;
    server_name localhost 192.168.0.103 212.85.24.33; # Sesuaikan IP ini dengan IP
    Laptop (Wi-Fi) atau IP Publik VPS Anda

    client_max_body_size 50M;

    # 1. Proxy API ke Go Backend Cluster (Load Balancer ke Port 8081 & 8091)
    location /api {
        proxy_pass http://lims_backend_cluster;
        proxy_http_version 1.1;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }

    # 2. Penyajian Berkas Unggahan (Direct Alias)
    location /uploads/ {
        alias /home/lims/shared_uploads/;
        expires 30d;
        add_header Cache-Control "public, no-transform";
    }

    # 3. Penyajian Folder Unduhan APK LIMS (Direct Alias)
    location /downloads/ {
        alias /home/lims/shared_downloads/;
        expires 30d;
        add_header Cache-Control "public, no-transform";
    }

    access_log /var/log/nginx/lims_access.log upstream_monitoring;
    error_log /var/log/nginx/lims_error.log;
}
```

Terapkan perubahan dan reload NGINX:

```
sudo nginx -t && sudo systemctl reload nginx
```

Best Practice Multi-VM & Dedicated Load Balancer

Jika ekosistem LIMS Anda berkembang ke arsitektur **multi-VM** (misalnya 2 VM dedicated untuk Frontend dan 2 VM dedicated untuk Backend):

1. **Konfigurasi Upstream pada Nginx:** Pada Nginx, blok upstream diatur menggunakan IP fisik masing-masing VM di jaringan internal (LAN). Karena aplikasi di masing-masing VM adalah instansi terpisah, mereka biasanya menggunakan port yang sama.

2. upstream lims_frontend_cluster {
3. server 192.168.10.11:3000 max_fails=3 fail_timeout=10s; # VM Frontend 1
4. server 192.168.10.12:3000 max_fails=3 fail_timeout=10s; # VM Frontend 2
5. }
- 6.
7. upstream lims_backend_cluster {
8. server 192.168.10.21:8081 max_fails=3 fail_timeout=10s; # VM Backend 1
9. server 192.168.10.22:8081 max_fails=3 fail_timeout=10s; # VM Backend 2
10. }
11. **Penggunaan Dedicated VM untuk NGINX (Sangat Direkomendasikan):** Pada lingkungan produksi, Nginx sebaiknya diletakkan pada **dedicated VM** (sebagai Load Balancer Gateway) yang terpisah dari VM Frontend maupun Backend.
 - **Mengapa?**
 - **Keamanan:** VM aplikasi tidak perlu memiliki IP publik. Hanya VM Nginx Gateway yang terekspos ke internet. Nginx bertindak sebagai benteng pertahanan pertama.
 - **SSL Termination:** Nginx memproses enkripsi/dekripsi SSL secara terpusat di satu VM, mengurangi beban kerja CPU pada VM aplikasi.
 - **Kemudahan Scaling:** Anda bisa menambah/mengurangi VM backend/frontend kapan saja di belakang Nginx tanpa perlu mengubah konfigurasi DNS atau client.

G. Rotasi Log Harian (Logrotate) & Penjadwalan di VPS

Rotasi log otomatis setiap hari dengan nama berakhiran `_YYYYMMDD.log` untuk menghemat ruang penyimpanan.

Mengecek Instalasi Logrotate

Sebelum mengatur rotasi log, pastikan logrotate sudah terinstall di sistem Linux/Ubuntu VPS Anda (secara umum aplikasi ini sudah terpasang bawaan). Anda dapat mengeceknya dengan menjalankan perintah di terminal server:

```
logrotate --version
```

atau cek di APT (Debian/Ubuntu)

```
dpkg -l | grep logrotate
```

Jika versi berhasil ditampilkan (misalnya *logrotate 3.14.0*), berarti aplikasi siap digunakan. Anda dapat melanjutkan ke langkah-langkah konfigurasi berikut.

Metode Rotasi yang Digunakan (Tanpa Downtime)

Sistem LIMS dikonfigurasi menggunakan **dua pendekatan rotasi berbeda** untuk memastikan log dapat diputar tanpa mematikan aplikasi (*Zero Downtime*):

1. **Metode postrotate & SinyalUSR1 (Khusus Nginx):** Logrotate akan memindahkan (*rename*) file log aktif, membuat file log kosong baru, lalu mengeksekusi blok kode postrotate yang mengirimkan sinyal kontrol USR1 ke proses Nginx. Sinyal ini menginstruksikan Nginx secara elegan untuk melepaskan pengikat file log lama dan mulai menulis ke file log baru, menjamin tidak ada satupun *request* yang hilang (presisi tingkat tinggi).
2. **Metode copytruncate (Untuk Backend Go & Frontend PM2):** Proses aplikasi internal (seperti *binary* Golang dan *daemon* Node.js PM2) berjalan dengan menahan *file descriptor* secara persisten. Jika file log sekadar diganti namanya,

proses-proses ini akan terus menulis ke file lama yang telah direname tersebut tanpa sadar. Oleh karena itu, kita menggunakan arahan copytruncate. Logrotate akan **menyalin (copy)** semua isi log aktif ke file arsip, lalu secara paksa **mengosongkan (truncate)** file log aktif kembali menjadi ukuran 0 *bytes*. Dengan demikian, aplikasi dapat terus menulis log secara kontinu tanpa menyadari intervensi tersebut dan tanpa perlu di-*restart* sama sekali.

Opsi Alternatif: Rotasi Berdasarkan Ukuran File (Size-Based)

Secara bawaan, konfigurasi di bawah ini menggunakan interval waktu harian (daily). Namun, LIMS juga bisa diatur untuk merotasi log secara otomatis saat file log membesar dan menyentuh batasan ukuran tertentu (misalnya 100MB). Anda bisa memodifikasi konfigurasi di bawah dengan menambahkan arahan berikut (menggantikan atau menemani daily):

- **size 100M:** Memutar file log **hanya** jika ukurannya telah melebihi 100 Megabytes (mengabaikan interval hari).
- **maxsize 100M:** Kombinasi hibrida. File log akan diputar jika sudah berganti hari (daily), **ATAU** jika ukurannya telah melebihi 100MB (mana yang tercapai lebih dulu). Sangat disarankan untuk skenario server sibuk.

[!WARNING] **Catatan Penting Presisi Waktu Eksekusi:** logrotate bukanlah program yang memonitor ukuran file secara *real-time* 24 jam penuh. Ia **hanya** akan mengecek ukuran file pada saat jadwal tugas *cron*-nya tereksekusi. Jika Anda mengatur *cron* logrotate untuk jalan 1x sehari di jam 01:00 dini hari (seperti panduan di bawah), ia baru akan tahu file sudah menyentuh 100MB pada jam tersebut (file bisa saja sudah membengkak jadi 500MB di siang hari). Jika Anda ingin sistem secara ketat memutar log tepat saat di kisaran 100MB, Anda harus mengubah jadwal eksekusi *cron* logrotate menjadi **setiap jam** (memindahkannya ke /etc/cron.hourly/ atau mengubah parameter cron menjadi 0 * * * *).

Rotasi Log Nginx (/etc/logrotate.d/nginx)

Edit berkas /etc/logrotate.d/nginx:

```
/var/log/nginx/lms_access.log
/var/log/nginx/lms_error.log {
    daily
    missingok
    rotate 14
    compress
    delaycompress
    notifempty
    create 0640 www-data adm
    sharedscripts
    dateext
    dateformat _%Y%m%d
    extension .log
    dateyesterday
    prerotate
        if [ -d /etc/logrotate.d/httpd-prerotate ]; then \
            run-parts /etc/logrotate.d/httpd-prerotate; \
        fi \
    
```

```

        endsript
    postrotate
        invoke-rc.d nginx rotate >/dev/null 2>&1
    endsript
}

/var/log/nginx/access.log
/var/log/nginx/error.log {
    daily
    missingok
    rotate 14
    compress
    delaycompress
    notifempty
    create 0640 www-data adm
    sharedscripts
    prerotate
        if [ -d /etc/logrotate.d/httpd-prerotate ]; then \
            run-parts /etc/logrotate.d/httpd-prerotate; \
        fi \
    endsript
    postrotate
        invoke-rc.d nginx rotate >/dev/null 2>&1
    endsript
}

```

Rotasi Log Backend Go (/etc/logrotate.d/lims-backend)

Untuk log yang dihasilkan oleh aplikasi backend (seperti file `api_traffic.log`, `db_query.log`, dan log lainnya di folder `logs/`), kita wajib menggunakan parameter **copytruncate** dan arahan **su**.

[!NOTE] **Pentingnya Parameter copytruncate:** Aplikasi backend Go menulis log dengan menahan *file descriptor* yang terbuka secara dinamis. Jika file log dipindah (renamed) secara langsung, program Go akan tetap menulis ke file log yang telah diganti namanya tersebut. Dengan `copytruncate`, sistem `logrotate` akan menyalin (copy) isi log lama terlebih dahulu kemudian memotong (truncate) log aktif menjadi kosong tanpa memutus penulisan biner Go.

Buat berkas konfigurasi baru `/etc/logrotate.d/lims-backend`:

Catatan: Berkas log pelatihan model (seperti `lims_ai-train-YYYYMMDD.log`)
 # sengaja tidak di-rotate di bawah ini karena namanya sudah mengandung format tanggal dari program.

```

/home/lims/lims1/backend/*.log
/home/lims/lims1/backend/logs/api_traffic.log
/home/lims/lims1/backend/logs/db_query.log
/home/lims/lims2/backend/*.log
/home/lims/lims2/backend/logs/api_traffic.log
/home/lims/lims2/backend/logs/db_query.log {
    su lims lims

```

```

daily
missingok
rotate 14
compress
delaycompress
copytruncate
notifempty
create 0664 lims lims
dateext
dateformat _%Y%m%d
extension .log
dateyesterday
}

```

Rotasi Log Frontend PM2 (/etc/logrotate.d/lims-frontend)

Frontend LIMS dijalankan sebagai HTTP server statis menggunakan PM2, yang mencatat console output (stdout/stderr) ke berkas log PM2. Untuk merotasi berkas log ini dengan aman menggunakan logrotate sistem, buat berkas konfigurasi baru /etc/logrotate.d/lims-frontend:

```

/home/lims/.pm2/logs/*.log {
    su lims lims
    daily
    missingok
    rotate 14
    compress
    delaycompress
    copytruncate
    notifempty
    create 0664 lims lims
    dateext
    dateformat _%Y%m%d
    extension .log
    dateyesterday
}

```

Pengujian Simulasi & Eksekusi Nyata

- **Melakukan Uji Coba Simulasi (Dry Run):** (Catatan: Parameter *-d* hanya mencetak simulasi dan *TIDAK* merename berkas fisik di disk).
- # Mencetak simulasi berkas Nginx yang akan diganti namanya:
- sudo logrotate -df /etc/logrotate.d/nginx
-
- # Mencetak simulasi berkas log backend:
- sudo logrotate -df /etc/logrotate.d/lims-backend
-
- # Mencetak simulasi berkas log frontend PM2:
- sudo logrotate -df /etc/logrotate.d/lims-frontend
- **Mengeksekusi Rotasi Secara Langsung (Nyata):**
- # Menjalankan rotasi fisik di disk secara langsung

- `sudo logrotate -fv /etc/logrotate.d/nginx`
- `sudo logrotate -fv /etc/logrotate.d/lims-backend`
- `sudo logrotate -fv /etc/logrotate.d/lims-frontend`
- **Mengecek Status Bawaan Logrotate (Status File):** Logrotate secara otomatis mencatat kapan terakhir kali ia berhasil memutar sebuah file log. Anda bisa mengecek catatan sistem (history) tersebut menggunakan perintah:
 - `cat /var/lib/logrotate/status | grep lims`
 - *# Atau jika di sistem lain: `cat /var/lib/logrotate.status | grep lims`*

Jika logrotate berhasil berjalan secara rutin, outputnya akan menunjukkan daftar file log LIMS beserta tanggal dan waktu terakhir ia dirotasi.

Konfigurasi Penjadwalan Otomatis Pukul 01:00 Dini Hari (Cron)

Secara bawaan, tugas cron harian system logrotate berjalan tidak menentu. Untuk memaksakan pemutaran berkas log dilakukan tepat pada **pukul 01:00 dini hari**, buatlah berkas cron khusus di `/etc/cron.d/logrotate-lims`:

```
sudo nano /etc/cron.d/logrotate-lims
```

Masukkan konfigurasi berikut:

```
0 1 * * * root /usr/sbin/logrotate /etc/logrotate.d/nginx >> /var/log/logrotate_nginx.log
2>&1
```

```
0 1 * * * root /usr/sbin/logrotate /etc/logrotate.d/lims-backend >>
/var/log/logrotate_backend.log 2>&1
```

```
0 1 * * * root /usr/sbin/logrotate /etc/logrotate.d/lims-frontend >>
/var/log/logrotate_frontend.log 2>&1
```

Set hak akses ke 0644 (wajib agar terbaca oleh cron daemon) dan restart layanan cron:

```
sudo chmod 0644 /etc/cron.d/logrotate-lims
```

```
sudo systemctl restart cron
```

Rotasi Log Frontend (PM2 - Opsi Alternatif pm2-logrotate)

Aplikasi frontend disajikan sebagai aset statis oleh NGINX (log akses webnya terekam di `/var/log/nginx/lims_access.log`). Namun, karena proses HTTP server-nya diluncurkan menggunakan PM2 (`lims-frontend-3000` dan `lims-frontend-3001`), PM2 merekam output konsol (`stdout` dan `stderr`) aplikasi tersebut ke dalam berkas log lokal PM2.

- **Lokasi Direktori Log PM2 Frontend:** Semua log frontend disimpan di direktori home user: `/home/lims/.pm2/logs/`
 - `/home/lims/.pm2/logs/lims-frontend-3000-out.log` (stdout) & `lims-frontend-3000-error.log` (stderr)
 - `/home/lims/.pm2/logs/lims-frontend-3001-out.log` (stdout) & `lims-frontend-3001-error.log` (stderr)

Meskipun saat ini kita telah mengonfigurasinya dengan utilitas **system logrotate** bawaan Linux di atas, sebagai alternatif internal PM2, Anda juga dapat menggunakan modul resmi **pm2-logrotate**:

1. **Instalasi Modul:**
2. `pm2 install pm2-logrotate`
3. **Konfigurasi Parameter Rotasi:** Secara bawaan, modul ini akan memutar log jika ukuran berkas melebihi batas tertentu atau secara terjadwal. Anda dapat mengaturnya sebagai berikut:
 - 4. *# Set batas ukuran maksimal berkas log sebelum di-rotate (misal: 10 Megabytes)*
 - 5. `pm2 set pm2-logrotate:max_size 10M`

- 6.
7. *# Set agar rotasi juga berjalan harian (tiap tengah malam) sebagai opsi tambahan*
8. `pm2 set pm2-logrotate:rotateInterval '0 0 * * *'`
- 9.
10. *# Set agar menyimpan arsip log maksimal selama 14 hari (sisanya otomatis dihapus)*
11. `pm2 set pm2-logrotate:retain 14`
- 12.
13. *# Mengaktifkan kompresi berkas log lama menjadi format zip (.gz)*
14. `pm2 set pm2-logrotate:compress true`

H. Real-Time Analytics dengan GoAccess

Dasbor dipasang di latar belakang dengan target file output di folder bersama `/home/lims/shared_reports/report.html`:

1. Pastikan folder output sudah terbuat dan memiliki izin akses yang benar

```
mkdir -p /home/lims/shared_reports
```

```
sudo chown -R lims:lims /home/lims/shared_reports
```

2. Jalankan GoAccess di latar belakang (Ganti `<IP_VPS_ANDA>` dengan IP VPS Anda, misal 212.85.24.33)

```
sudo goaccess /var/log/nginx/lims_access.log \
  --log-format='%h - ClientIP: %^ - [%d:%t %^]' "%r" %s %b to_server=%v status=%^
  resp_time=%^ agent="%u" \
  --date-format='%d/%b/%Y' \
  --time-format='%H:%M:%S' \
  --ws-url=wss://<IP_VPS_ANDA>:8082/ws \
  -o /home/lims/shared_reports/report.html \
  --real-time-html &
```

Kontrol Pengguna & Keamanan Akses GoAccess

Secara default, GoAccess menghasilkan file HTML statis (`report.html`) yang tidak memiliki fitur autentikasi bawaan. Agar data lalu lintas server Anda tidak diakses oleh pihak luar, terapkan salah satu metode pengamanan berikut.

OPSI 1: Proteksi Basic Authentication via Nginx (Praktis & Cepat - STATIS)

[!WARNING] **Keterbatasan Opsi 1:** Metode ini menggunakan password statis yang di-hash ke dalam file `.htpasswd` Nginx. Metode ini **tidak cocok** jika Anda ingin menggunakan kredensial database (seperti user `nur` yang password-nya berubah/kedaluwarsa maksimal setiap 90 hari). Perubahan password di database tidak akan menyinkronkan file `.htpasswd` secara otomatis.

Namun, jika Anda tetap ingin menggunakan Basic Auth statis, berikut adalah contoh konfigurasinya:

1. **Membuat File `.htpasswd` untuk User `nur` dan `admin`:** Jalankan perintah ini di terminal server Linux Anda:
2. *# Buat file baru dengan menambahkan user 'nur' (Masukkan password 'nur' saat diminta)*
3. `sudo apt install apache2-utils -y`

4. `sudo htpasswd -c /etc/nginx/.htpasswd nur`
- 5.
6. *# Tambahkan user 'admin' ke file yang sama (Masukkan password 'admin' saat diminta)*
7. `sudo htpasswd /etc/nginx/.htpasswd admin`
8. **Konfigurasi Nginx untuk Basic Auth:** Edit file konfigurasi Nginx Anda (`/etc/nginx/conf.d/lims.conf`), cari atau buat blok `/report.html` berikut:
9. *# --- COPY-PASTE KE KONFIGURASI NGINX ---*
10. `location = /report.html {`
11. `alias /home/lims/shared_reports/;`
12. `auth_basic "LIMS Analytics Restricted Area";`
13. `auth_basic_user_file /etc/nginx/.htpasswd;`
14. `}`
15. *# -----*

OPSI 2: Proxy Melalui API Otentikasi LIMS (Paling Aman - DINAMIS & TERINTEGRASI DB)

Metode ini sangat direkomendasikan karena **menyelesaikan masalah ekspirasi password 90 hari**. Autentikasi dilakukan oleh Go backend dengan memeriksa token session (JWT) pengguna yang login dari database. Jika password user nur berubah di database, token lamanya otomatis tidak valid dan ia harus login ulang dengan password baru untuk mengakses halaman laporan.

Langkah 1: Ubah Konfigurasi Nginx (`/etc/nginx/conf.d/lims.conf`)

Hapus atau beri tanda komentar pada blok `/report.html` lama agar file tidak bisa diakses secara langsung oleh siapa pun tanpa token:

```
# --- COPY-PASTE: HAPUS ATAU HILANGKAN BLOK INI ---
# location = /report.html {
#   alias /home/lims/shared_reports/;
# }
# -----
```

Terapkan dan reload Nginx: `sudo systemctl reload nginx`

Langkah 2: Buat Endpoint Handler di Backend Go (`backend/controllers/report_controller.go`)

Tambahkan handler fungsi berikut di bagian bawah berkas `report_controller.go` untuk membaca file secara dinamis dan memvalidasi user/role secara terintegrasi dengan database:

```
// --- COPY-PASTE KE backend/controllers/report_controller.go ---
func GetGoAccessReport(c *gin.Context) {
    // 1. Ambil informasi dari JWT Session (yang otomatis memvalidasi password database aktif)
    username, existsUsername := c.Get("username")
    role, existsRole := c.Get("role")

    if !existsUsername || !existsRole {
        c.JSON(http.StatusUnauthorized, gin.H{"error": "Unauthorized: Sesi tidak ditemukan"})
    }
}
```

```

        return
    }

    // CONTOH A: Membatasi akses KHUSUS untuk user 'nur' saja (karena
    password nur berubah maksimal 90 hari)
    if username.(string) != "nur" {
        // CONTOH B: Jika bukan 'nur', periksa apakah user memiliki Role
ADMIN
        if role.(string) != "ADMIN" {
            c.JSON(http.StatusForbidden, gin.H{"error": "Forbidden: Hanya
user 'nur' atau role ADMIN yang diizinkan mengakses analitik."})
            return
        }
    }

    // 2. Baca file report.html dari shared folder
    htmlPath := "/home/lims/shared_reports/report.html"
    htmlContent, err := os.ReadFile(htmlPath)
    if err != nil {
        c.JSON(http.StatusInternalServerError, gin.H{"error": "Gagal membaca
berkas laporan analitik GoAccess"})
        return
    }

    // 3. Kirimkan konten sebagai berkas HTML murni
    c.Data(http.StatusOK, "text/html; charset=utf-8", htmlContent)
}

```

Langkah 3: Daftarkan Rute API di Backend (backend/routes/routes.go)

Daftarkan rute di dalam grup protected (di bawah middleware AuthMiddleware):

// --- COPY-PASTE KE backend/routes/routes.go ---

// Cari grup protected di routes.go, lalu daftarkan rute ini:

```
protected.GET("/reports/analytics-report", controllers.GetGoAccessReport)
```

(Catatan: Rute ini dilindungi oleh AuthMiddleware() sehingga otomatis menolak token kedaluwarsa atau user dengan password yang sudah expired di database).

Langkah 4: Tampilan Frontend (Sisi Web)

Admin/User nur dapat mengakses laporan analitik langsung melalui URL

API: <https://lims.perusahaan.com/api/reports/analytics-report>

OPSI 3: Nginx auth_request Directive (Performa Terbaik - Hybrid Dinamis)

Metode ini adalah gabungan performa terbaik. Nginx akan bertugas **menyajikan file HTML secara langsung** dari disk (sangat cepat dan hemat memori backend), tetapi Nginx akan **meminta persetujuan Go Backend** terlebih dahulu via sub-request sebelum menyajikannya. Backend akan memvalidasi session/JWT/username user secara dinamis dari database.

Langkah 1: Tambahkan Rute Validasi Cepat di Go Backend (backend/controllers/report_controller.go)

Buat endpoint ringan yang hanya mengembalikan status HTTP 200 (jika diizinkan) atau 403/401 (jika ditolak) tanpa membaca file HTML ke memory Go:

// --- COPY-PASTE KE backend/controllers/report_controller.go ---

```
func CheckReportAccess(c *gin.Context) {
    username, existsUsername := c.Get("username")
    role, existsRole := c.Get("role")

    if !existsUsername || !existsRole {
        c.Status(http.StatusUnauthorized)
        return
    }

    // Validasi apakah user adalah 'nur' atau memiliki role 'ADMIN'
    if username.(string) == "nur" || role.(string) == "ADMIN" {
        c.Status(http.StatusOK) // Autentikasi Berhasil
        return
    }

    c.Status(http.StatusForbidden) // Ditolak
}
```

Langkah 2: Daftarkan Rute Validasi di backend/routes/routes.go

Daftarkan rute pemeriksaan di bawah AuthMiddleware:

// --- COPY-PASTE KE backend/routes/routes.go ---

```
protected.GET("/auth/check-report-access", controllers.CheckReportAccess)
```

Langkah 3: Konfigurasi Nginx dengan auth_request (/etc/nginx/conf.d/lims.conf)

Ganti blok /report.html lama Anda dengan konfigurasi copy-paste berikut:

1. Halaman Laporan GoAccess yang Dilindungi

```
location = /report.html {
    # Nginx akan mengirimkan sub-request internal ke path /api/auth/check-report-access
    auth_request /api/auth/check-report-access;

    # Jika sub-request mengembalikan HTTP 200 OK, sajikan file ini:
    alias /home/lims/shared_reports/report.html;
}
```

1b. Redirect /report.html/ ke /report.html (Mengatasi isu trailing slash dari link)

```
location = /report.html/ {
    return 301 $scheme://$http_host/report.html$is_args$args;
}
```

2. Lokasi Proxy Internal untuk Pemeriksaan Autentikasi Backend LIMS

```
location = /api/auth/check-report-access {
    internal; # Hanya bisa diakses secara internal oleh Nginx
    proxy_pass http://lims_backend_cluster;
```

Jangan kirim request body ke backend untuk optimasi kecepatan


```
proxy_pass_request_body off;
proxy_set_header Content-Length "";
```

```
# Teruskan cookie dan header otentikasi dari klien
```

```
proxy_set_header X-Original-URI $request_uri;
```

```
proxy_set_header Host $host;
```

```
proxy_set_header X-Real-IP $remote_addr;
```

```
proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
```

```
proxy_set_header X-Forwarded-Proto $scheme;
```

```
}
```

```
# -----
```

Terapkan dan reload Nginx: `sudo nginx -t && sudo systemctl reload nginx`

Perbandingan: GoAccess vs Grafana

Fitur	GoAccess	Grafana (Loki/Prometheus Stack)
Fokus Utama	Analisis log server web secara real-time.	Visualisasi data metrik server, database, dan logs secara menyeluruh.
Kebutuhan Resource	Sangat ringan (CPU/RAM di bawah 5%).	Cukup berat (memerlukan instalasi database pencatat metrik tambahan).
Keamanan & RBAC	Tidak ada bawaan (harus via Nginx/App proxy).	Memiliki sistem login dan Role-Based Access Control (RBAC) bawaan.
Historikal Data	Terbatas pada rentang log aktif saat ini.	Sangat baik (dapat menyimpan data dari hari/bulan yang lalu).
Kemudahan Setup	Sangat mudah (hanya satu perintah install & run).	Kompleks (perlu setup agen pengumpul log/metrik di setiap VM).
Kesimpulan Pemilihan	Pilih GoAccess untuk pemantauan traffic web harian yang cepat, ringan, dan gratis biaya server. Pilih Grafana untuk pemantauan infrastruktur skala enterprise yang membutuhkan audit historis jangka panjang dan alert otomatis.	

I. Pemantauan & Pemeliharaan (Checklist)

- **Melihat log systemd Go Backend:**
- `sudo journalctl -u lims-backend-8081.service -f`
- `sudo journalctl -u lims-backend-8091.service -f`
- **Melihat log PM2 Frontend:**

- pm2 logs lims-frontend-3000
- pm2 logs lims-frontend-3001
- **Melihat log Nginx:**
- tail -f /var/log/nginx/lims_access.log

[!NOTE] **Tanya Jawab Pengoperasian Layanan Backend:**

- **Bagaimana cara merestart salah satu instance backend saja (misal port 8081)?** Anda cukup menjalankan perintah berikut di terminal:
- sudo systemctl restart lims-backend-8081.service
- **Kapan saya perlu menjalankan sudo systemctl daemon-reload?** Perintah daemon-reload **hanya diperlukan** ketika Anda mengubah isi dari berkas konfigurasi unit .service (seperti memodifikasi isi file /etc/systemd/system/lims-backend-8081.service). Jika hanya ingin mematikan, menyalakan, atau merestart program backend biasa, Anda tidak perlu menjalankannya.

J. Pelatihan Model AI (PQC) & Penjadwalan Otomatis (Crontab)

LIMS mengintegrasikan modul *Predictive Quality Control* (PQC) berbasis AI (Isolation Forest) yang melatih model di latar belakang menggunakan skrip Python (train.py) dan mengeksport hasilnya dalam format ONNX.

Prasyarat Virtual Environment Python di Server

Sangat disarankan menggunakan virtual environment terisolasi (/home/lims/lims-ai-env) demi keamanan paket dependency:

1. Pastikan paket pendukung venv & pip sudah terpasang di VPS

```
sudo apt update && sudo apt install python3-venv python3-pip -y
```

2. Buat virtual environment

```
python3 -m venv /home/lims/lims-ai-env
```

Aktifkan virtual environment

```
source /home/lims/lims-ai-env/bin/activate
```

Install dependensi pustaka pendukung manipulasi data, database, AI, dan enkripsi

```
pip install pandas numpy scikit-learn==1.4.1.post1 sqlalchemy skl2onnx onnx
```

```
cryptography pycryptodome
```

Kebijakan Latihan Ulang (Retraining) & Data Flow Diagram (DFD) Level 1

Proses retraining dilakukan secara offline untuk memperbarui model tanpa membebani runtime Go Backend yang melayani user secara real-time. Alur kerjanya adalah sebagai berikut:

1. **Pengambilan Data:** Skrip train.py membaca data transaksi historis dari PostgreSQL (lims.testing_results).
2. **Pelatihan Model:** Isolation Forest dilatih ulang menggunakan scikit-learn dengan parameter terkini.
3. **Eksport Model:** Model diekspor ke file biner ONNX (pqc_<ASPECT>.onnx) dan statistik median/std diekspor ke file JSON (pqc_<ASPECT>_meta.json).
4. **Registrasi:** Skrip mencatat entry model baru ke tabel registri lims.ai_model_registry dengan status ACTIVE.

5. **Pemuatan Model (Go):** Go Backend secara dinamis membaca berkas model ONNX terupdate untuk deteksi transaksi berikutnya.

Mekanisme Penemuan Berkas Model oleh Go Backend

Untuk menemukan berkas model ONNX (pqc_<ASPECT>.onnx) dan metadatanya, Go Backend memanggil helper function findModelFile(aspectCode, ext) dengan urutan pencarian (precedence) sebagai berikut:

1. **Parameter Database:** Sistem mencari nilai parameter AI_METADATA_FOLDER pada tabel lims.global_parameters. Jika didefinisikan dan direktorinya ada, backend membaca berkas model dari path tersebut (misal: /home/lims/lims1/backend/ai_service/models).
2. **Direktori Relatif:** Jika parameter database kosong atau path tersebut tidak valid, backend mencari di direktori relatif bawaan proyek dengan urutan:
 - o ai_service/models/
 - o backend/ai_service/models/
 - o ../backend/ai_service/models/
 - o Direktori kerja aktif (*current working directory*).

Menjalankan Pelatihan Secara Manual

Anda dapat memicu pelatihan ulang (*offline retraining*) secara manual kapan saja dengan menjalankan perintah berikut menggunakan Python virtual environment:

```
/home/lims/lims-ai-env/bin/python /home/lims/lims1/backend/ai_service/train.py
```

Penjadwalan Otomatis Menggunakan Crontab (Cron Job)

Untuk menjamin akurasi model selalu ter-update berdasarkan data baru, pelatihan diatur agar berjalan otomatis secara berkala (setiap hari Senin pukul 02:00 dini hari).

1. Buka editor crontab untuk user **lims**:
2. crontab -e
3. Tambahkan baris penjadwalan berikut di bagian paling bawah berkas:
4. 0 2 * * 1 /home/lims/lims-ai-env/bin/python
/home/lims/lims1/backend/ai_service/train.py >>
/home/lims/lims1/backend/logs/train_cron.log 2>&1
5. Simpan dan tutup editor. Cron daemon akan otomatis memuat jadwal baru.

Penanganan Masalah (Troubleshooting): Error Permission Denied pada train.py

Jika saat menjalankan skrip train.py (baik secara manual maupun terjadwal via cron) Anda menemui error seperti berikut:

Error exporting aspect PESWT to ONNX: [Errno 13] Permission denied:

'/home/lims/lims1/backend/ai_service/models/pqc_PESWT.onnx'

Hal ini terjadi karena folder /home/lims/lims1/backend/ai_service/models atau berkas model di dalamnya dimiliki oleh user lain (misalnya root akibat menjalankan skrip dengan sudo), sehingga user lims tidak memiliki izin menulis/menimpa berkas tersebut.

Langkah Solusi:

1. **Ubah kepemilikan direktori** beserta berkas di dalamnya menjadi milik user lims:
2. sudo chown -R lims:lims /home/lims/lims1/backend/ai_service/models
3. **Atur hak akses** agar user lims dapat membaca dan menulis ke direktori tersebut:
4. sudo chmod -R 755 /home/lims/lims1/backend/ai_service/models

Penanganan Masalah (Troubleshooting): Error AttributeError: module 'ml_dtypes' has no attribute 'float4_e2m1fn'

Jika saat menjalankan skrip train.py Anda menemui kesalahan tipe data ml_dtypes seperti berikut: `AttributeError: module 'ml_dtypes' has no attribute 'float4_e2m1fn'`
Hal ini disebabkan oleh ketidakcocokan versi (*version mismatch*) antara pustaka onnx dengan pustaka ml_dtypes pada lingkungan Python (khususnya Python 3.14+).

Langkah Solusi: Perbarui pustaka ml_dtypes ke versi terbaru di dalam virtual environment Anda menggunakan perintah berikut:

```
/home/lims/lims-ai-env/bin/pip install --upgrade ml_dtypes
```

3. **Jalankan kembali skrip** sebagai user lims (tanpa sudo):

4. `/home/lims/lims-ai-env/bin/python /home/lims/lims1/backend/ai_service/train.py`

Penanganan Masalah (Troubleshooting): Error Generate Laporan AI (Invalid API Key / Error 401)

Jika saat menggunakan fitur "Generate Laporan AI" di UI LIMS Anda menemui pesan kesalahan seperti berikut: `Error generate laporan AI: AI API mengembalikan kode error 401: {"error":{"message":"Invalid API`

`Key","type":"invalid_request_error","code":"invalid_api_key"}}}`

Hal ini disebabkan karena kunci API (`AI_API_KEY`) yang dikonfigurasi pada berkas .env di VPS Anda salah, tidak valid, atau sudah kedaluwarsa.

Langkah Solusi:

1. Buka berkas .env pada folder instansi LIMS Anda di VPS:
2. `sudo nano /home/lims/lims1/backend/.env`
3. `sudo nano /home/lims/lims2/backend/.env`
4. Perbarui nilai variabel `AI_API_KEY` dengan kunci API yang valid (misalnya kunci Groq baru dari <https://console.groq.com/keys> atau Gemini dari konsol Google):
5. # Contoh jika menggunakan Groq:
6. `AI_API_URL=https://api.groq.com/openai/v1`
7. `AI_API_KEY=gsk_KUNCI_BARU_ANDA_DI_SINI`
8. `AI_MODEL=llama-3.1-8b-instant`
9. Simpan perubahan berkas, lalu restart layanan backend LIMS agar memuat konfigurasi yang baru:
10. `sudo systemctl restart lims-backend-8081.service`
11. `sudo systemctl restart lims-backend-8091.service`

Cara Membuat Kunci API Groq Baru: Jika Anda belum memiliki kunci Groq atau kunci lama Anda sudah kedaluwarsa, ikuti langkah berikut:

1. Buka *browser* dan kunjungi [Groq Cloud Console](#).
2. *Login* menggunakan akun Google atau GitHub Anda (layanan ini gratis).
3. Di menu sebelah kiri, pilih **API Keys**.
4. Klik tombol **Create API Key**.
5. Beri nama kunci Anda (misalnya: `LIMS_Dev_Laptop`) lalu klik **Submit**.
6. Sebuah teks rahasia yang diawali dengan `gsk_` akan muncul. **Segera salin (copy)** teks tersebut, karena Groq tidak akan pernah menampilkannya lagi.
7. *Paste* teks tersebut ke dalam file .env Anda pada baris `AI_API_KEY=....`

Panduan Instalasi & Konfigurasi LLM Lokal (Ollama - Qwen 2.5 3B / Llama 3.1 8B) di VPS

Prinsip Kerahasiaan & Keamanan Data (100% On-Premises Local AI)

Untuk menjamin **kerahasiaan data pengujian laboratorium (Zero External Data Leakage)**, sistem LIMS mendukung penggunaan **LLM Lokal On-Premises via Ollama**. Seluruh proses pengolahan data teknis, skor kelayakan, dan rekomendasi laporan diolah 100% secara lokal di dalam server VPS milik instansi tanpa pernah mengirimkan paket data ke pihak ketiga atau server cloud luar.

- **Jika VPS Anda menggunakan CPU-only (RAM terbatas):** Sangat direkomendasikan menggunakan model **Qwen 2.5 (3B)** karena sangat cepat dijalankan di CPU, ramah RAM (~1.9 GB), dan memiliki pemahaman Bahasa Indonesia yang luar biasa untuk ukurannya. Ini mencegah kendala *timeout*.
- **Jika VPS Anda memiliki GPU/vRAM Besar (8GB+ VRAM):** Model **Llama 3.1 (8B)** adalah pilihan terbaik untuk akurasi dan kompleksitas laporan tingkat tinggi.

Rekomendasi Pilihan Model LLM Lokal (Offline)

Tergantung pada kapasitas vRAM GPU atau memori VPS Anda, berikut adalah beberapa pilihan model AI offline terbaik yang didukung:

Model	Ukuran Parameter	Kebutuhan vRAM	Kelebihan	Perintah Unduh
Qwen 2.5 (3B) <i>(Rekomendasi Utama CPU)</i>	3 Miliar	~4 GB (RAM/vRAM)	Sangat ringan (~1.9GB), cepat di CPU biasa, dan pemahaman Bahasa Indonesia sangat baik.	ollama pull qwen2.5:3b
Llama 3.2 (3B)	3 Miliar	~4 GB (RAM/vRAM)	Ringan, cepat, dan robust untuk resource menengah.	ollama pull llama3.2
Qwen 2.5 (7B)	7 Miliar	~8 GB vRAM	Sangat cerdas dalam memahami Bahasa Indonesia dan struktur format JSON.	ollama pull qwen2.5:7b
Llama 3.1 (8B) <i>(Rekomendasi GPU)</i>	8 Miliar	~8 GB vRAM	Model standar industri, sangat cerdas, kreatif, dan robust untuk penyusunan laporan.	ollama pull llama3.1

1. Langkah Instalasi Ollama di VPS

Jalankan script instalasi resmi Ollama pada terminal VPS Anda:

```
curl -fsSL https://ollama.com/install.sh | sh
```

2. Mengunduh Model AI & Embedding

Unduh model LLM utama serta model embedding teks (RAG) ke dalam server lokal Anda:

```
# 1. Unduh model utama Qwen 2.5 (3B) - Sangat cocok untuk CPU-only
```

```
ollama pull qwen2.5:3b
```

```
# (Opsional) Unduh model utama Llama 3.1 (8B) jika VPS memiliki GPU/vRAM besar
```

```
# ollama pull llama3.1
```

```
# 2. Unduh model Embedding teks (nomic-embed-text)
```

```
ollama pull nomic-embed-text
```

2b. Optimasi Kecepatan, RAM Keep-Alive (OLLAMA_KEEP_ALIVE=-1) & Alokasi Resource CPU

Secara *default*, Ollama menghapus model dari memory RAM jika tidak ada aktivitas selama 5 menit (*idle*). Hal ini menyebabkan permintaan berikutnya harus membaca ulang berkas model 1.9GB dari disk ke RAM (memakan waktu ~35–45 detik).

A. Mengunci Model di RAM VPS (OLLAMA_KEEP_ALIVE=-1) & Prioritas CPU (Nice=10):

1. Buka berkas konfigurasi service Ollama di VPS:
2. `sudo nano /etc/systemd/system/ollama.service`
3. Tambahkan variabel `OLLAMA_KEEP_ALIVE=-1`, `OMP_NUM_THREADS=6` (isolasi 2 cores untuk LIMS), dan `Nice=10` di bawah blok [Service]:
4. [Service]
5. `Environment="OLLAMA_KEEP_ALIVE=-1"`
6. `Environment="OLLAMA_NUM_PARALLEL=1"`
7. `Environment="OMP_NUM_THREADS=6"`
8. `Nice=10`
9. Simpan berkas, lalu *reload* dan *restart* service:
10. `sudo systemctl daemon-reload`
11. `sudo systemctl restart ollama`

2c. Arsitektur Produksi: Dedicated VM Khusus Ollama (Best Practice Production)

Untuk lingkungan produksi (*Production Environment*), sangat direkomendasikan membuat **VM Terpisah (Dedicated VM)** khusus untuk Ollama AI Engine.

Keuntungan Dedicated VM:

- **Zero Degradation:** Beban pemrosesan AI 100% terisolasi di VM terpisah, sehingga performa LIMS App Server dan Database PostgreSQL tidak akan terpengaruh sama sekali.
- **Respons Super Cepat (1-3 Detik):** Dedicated VM dapat mengalokasikan 100% CPU/GPU tanpa perlu berbagi dengan proses lain, sehingga stream AI langsung keluar dalam **1 s/d 3 detik**.

Konfigurasi Dedicated VM Ollama:

1. Di Dedicated VM Ollama, izinkan listener IP jaringan privat dengan menambahkan `OLLAMA_HOST=0.0.0.0` pada `/etc/systemd/system/ollama.service`:
2. [Service]
3. `Environment="OLLAMA_HOST=0.0.0.0"`
4. `Environment="OLLAMA_KEEP_ALIVE=-1"`
5. `Environment="OMP_NUM_THREADS=8"`
6. Di LIMS Application Server, arahkan berkas `.env` ke IP Dedicated VM Ollama:
7. `# --- KOORDINAT MODEL AI LOKAL (DEDICATED OLLAMA VM) ---`
8. `AI_API_URL=http://192.168.10.50:11434/v1` # (Ganti dengan IP Dedicated VM Anda)
9. `AI_API_KEY=none`
10. `AI_MODEL=qwen2.5:1.5b` # (Atau `qwen2.5:3b`)
11. `AI_NUM_THREAD=8`
12. `AI_NUM_CTX=512`
- 13.
14. `# --- KOORDINAT EMBEDDING LOKAL (DEDICATED OLLAMA VM) ---`
15. `AI_EMBEDDING_API_URL=http://192.168.10.50:11434`
16. `AI_EMBEDDING_MODEL=nomic-embed-text`

3. Konfigurasi Berkas `.env` LIMS di VPS (Single Server)

Jika menggunakan satu server VPS yang sama, sesuaikan berkas konfigurasi `.env`:

1. Buka berkas konfigurasi `.env` instansi Anda:
2. `sudo nano /home/lims/lims1/backend/.env`
3. `sudo nano /home/lims/lims2/backend/.env`
4. Ubah bagian **AI Integration** menjadi konfigurasi lokal berikut:
5. `# --- KOORDINAT MODEL AI LOKAL (OLLAMA SINGLE SERVER) ---`
6. `AI_API_URL=http://127.0.0.1:11434/v1`
7. `AI_API_KEY=none`
8. `AI_MODEL=qwen2.5:1.5b` # (Atau `qwen2.5:3b`)
9. `AI_NUM_THREAD=6`
10. `AI_NUM_CTX=512`
- 11.
12. `# --- KOORDINAT EMBEDDING LOKAL (OLLAMA) ---`
13. `AI_EMBEDDING_API_URL=http://127.0.0.1:11434`
14. `AI_EMBEDDING_MODEL=nomic-embed-text`
15. Simpan berkas, lalu restart layanan backend LIMS:
16. `sudo systemctl restart lims-backend-8081.service`
17. `sudo systemctl restart lims-backend-8091.service`

4. Konfigurasi Nginx Reverse Proxy untuk Real-Time SSE Streaming (`/etc/nginx/conf.d/lims.conf`)

Agar aliran token AI terkirim secara *live* tanpa tertahan buffer Nginx:

```
location /api {
    proxy_pass http://lims_backend_cluster;
    proxy_http_version 1.1;
    proxy_set_header Upgrade $http_upgrade;
    proxy_set_header Connection 'upgrade';
```

```

proxy_set_header Host $host;
proxy_cache_bypass $http_upgrade;
proxy_set_header X-Real-IP $remote_addr;
proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
proxy_set_header X-Forwarded-Proto $scheme;

# --- KONFIGURASI STREAMING AI & LONG-RUNNING REQUESTS ---
gzip off;                # Matikan gzip kompresi Nginx khusus API agar stream SSE
tidak tertahan
proxy_buffering off;      # Nonaktifkan buffering Nginx agar stream token AI
terkirim real-time
proxy_cache off;          # Nonaktifkan caching respons API
chunked_transfer_encoding on; # Aktifkan chunked transfer encoding untuk SSE
proxy_connect_timeout 600s; # Timeout koneksi hingga 10 menit
proxy_send_timeout 600s;   # Timeout pengiriman hingga 10 menit
proxy_read_timeout 600s;   # Timeout membaca stream AI hingga 10 menit
}

```

5. Panduan Investigasi & Tracing SSE Streaming (Debugging End-to-End)

Jika terjadi kendala alur streaming (misalnya teks muncul sekaligus di akhir atau terjadi time-out), ikuti alur investigasi berjenjang berikut:

1. Investigasi Lapisan 1: Chrome / Browser DevTools (F12):

- Buka **DevTools (F12)** →→ tab **Network** →→ klik permintaan generate-report →→ buka tab **EventStream**.
- **Hasil A:** Jika event message muncul berkala satu demi satu di DevTools tetapi UI tidak bergerak →→ Kendala ada di lapisan React UI (AppDetail.jsx).
- **Hasil B:** Jika tab EventStream kosong selama 45 detik lalu semua data keluar bersamaan →→ Kendala ada di Nginx buffering atau Ollama cold-start loading.

2. Investigasi Lapisan 2: Tracing curl Unbuffered di Terminal VPS:

- Jalankan perintah berikut di terminal VPS untuk melihat *raw byte stream* tanpa buffer:
- `curl -N -v -H "Authorization: Bearer <TOKEN_ANDA>" -X POST https://lims-d4551821.nip.io:8082/api/applications/1/generate-report`
- Opsi `-N (--no-buffer)` memaksa curl menampilkan setiap baris token begitu diterima dari Nginx.
- Jika curl langsung ke port Go (`http://127.0.0.1:8081/api/...`) berjalan lancar namun via domain Nginx tertahan →→ Pastikan `gzip off`; dan `proxy_buffering off`; sudah aktif di `/etc/nginx/conf.d/lims.conf` lalu jalankan `sudo systemctl reload nginx`.

Skrip Peluncuran Ubuntu Terpadu (start_lims.sh dengan Tmux)

Di lingkungan produksi Ubuntu, seluruh komponen LIMS backend (port 8081 & 8091), frontend (PM2), dan services pendukung (Ollama) diluncurkan dalam sesi tmux terpisah menggunakan skrip `start_lims.sh`:

```
#!/bin/bash
```



```

SESSION="lims_system"
tmux has-session -t $SESSION 2>/dev/null

if [ $? -ne 0 ]; then
    # Buat sesi baru
    tmux new-session -d -s $SESSION -n "backend-8081"

    # Window 1: Backend Port 8081
    tmux send-keys -t $SESSION:"backend-8081" "cd /home/lims/lims1/backend &&
./main" C-m

    # Window 2: Backend Port 8091 (Replikasi Load Balancer)
    tmux new-window -t $SESSION -n "backend-8091"
    tmux send-keys -t $SESSION:"backend-8091" "cd /home/lims/lims1/backend &&
PORT=8091 ./main" C-m

    # Window 3: Ollama Service (Vektor Embedding AI)
    tmux new-window -t $SESSION -n "ollama"
    tmux send-keys -t $SESSION:"ollama" "ollama serve" C-m

    # Window 4: Frontend (PM2 Monitoring)
    tmux new-window -t $SESSION -n "frontend"
    tmux send-keys -t $SESSION:"frontend" "pm2 logs" C-m
fi

```

```
tmux attach-target -t $SESSION
```

K. Administrasi & Pemeliharaan Modul Pendukung (PostgreSQL, MinIO, Camunda)

Bagian ini mendokumentasikan instalasi, pemeriksaan (*healthcheck*), konfigurasi, status, pemantauan (*monitoring*), dan penelusuran kesalahan (*tracing/logs*) untuk modul-modul infrastruktur pendukung LIMS.

Database Server (PostgreSQL)

PostgreSQL digunakan sebagai penyimpanan data relasional utama untuk LIMS.

1. Instalasi

- **Melalui Paket Manager Linux (Standalone):**
 - `sudo apt update`
 - `sudo apt install postgresql postgresql-contrib -y`
 - *# Install ekstensi pgvector untuk keperluan pencarian vektor AI (Chatbot RAG LIMS)*
 - `sudo apt install postgresql-15-pgvector -y` *# Sesuaikan versi dengan PostgreSQL Anda (misal postgresql-16-pgvector)*
- **Melalui Docker Container (Rekomendasi Kontainerisasi):**
 - `docker run -d \`
 - `--name lims-postgres \`
 - `-p 5433:5432 \`

- -e POSTGRES_DB=lims_prod_db \
- -e POSTGRES_USER=lims_app \
- -e POSTGRES_PASSWORD=Nkl@130200 \
- -v postgres_data:/var/lib/postgresql/data \
- --restart always \
- postgres:15-alpine

2. Pemeriksaan Instalasi (Health Check)

Gunakan utilitas pg_isready untuk memeriksa status ketersediaan database secara instan:

Format: pg_isready -h [host] -p [port]

pg_isready -h 127.0.0.1 -p 5433

Output sukses: 127.0.0.1:5433 - accepting connections

3. Konfigurasi

Berkas konfigurasi utama terletak

di /etc/postgresql/[versi]/main/postgresql.conf (standalone) atau berada di dalam volume Docker.

- **Mengizinkan Koneksi dari Luar (WSL/Host):** Pastikan listen_addresses = '*' aktif di postgresql.conf, dan baris berikut terdaftar di /etc/postgresql/[versi]/main/pg_hba.conf:
 - host all all 0.0.0.0/0 scram-sha-256

4. Status & Pemantauan (Status & Monitoring)

- **Cek Status Service:**
 - *# Standalone*
 - *sudo systemctl status postgresql*
 - *# Docker*
 - *docker ps | grep lims-postgres*
- **Melihat Koneksi Aktif (SQL Query):** Masuk ke psql:
 - *PGPASSWORD=Nkl@130200 psql -U lims_app -h 127.0.0.1 -p 5433 -d lims_prod_db*

Jalankan query monitoring:

SELECT pid, username, client_addr, state, query FROM pg_stat_activity;

5. Penelusuran Error (Tracing & Logs)

- **Membaca Log PostgreSQL:**
 - *Standalone: /var/log/postgresql/postgresql-[versi]-main.log*
 - *Docker: docker logs -f lims-postgres*

6. Panduan Inisialisasi Database LIMS (Tablespace, User, Owner, Schema, & Extension)

Ikuti urutan inisialisasi database PostgreSQL berikut untuk menjamin keamanan dan keselarasan dengan arsitektur LIMS:

1. **Buat Folder Penyimpanan Data (Jika menggunakan Tablespace Kustom - Opsional):**
2. *sudo mkdir -p /var/lib/postgresql/lims_data*
3. *sudo chown -R postgres:postgres /var/lib/postgresql/lims_data*
4. **Buat User (Role) LIMS:** Buka terminal psql sebagai user postgres:
5. *-- Membuat user/role baru untuk aplikasi LIMS*
6. *CREATE USER lims_app WITH PASSWORD 'Nkl@130200';*
7. **Buat Tablespace Khusus LIMS (Opsional tapi Best Practice di Prod):**

8. -- Memisahkan penyimpanan fisik data LIMS ke folder khusus
9. CREATE TABLESPACE lims_tblspace LOCATION
'/var/lib/postgresql/lims_data';
10. **Buat Database dengan Owner dan Tablespace Terkait:**
11. -- Membuat database utama LIMS
12. CREATE DATABASE lims_prod_db OWNER lims_app TABLESPACE
lims_tblspace;
- 13.
14. -- Memberikan seluruh hak akses database ke user lims
15. GRANT ALL PRIVILEGES ON DATABASE lims_prod_db TO lims_app;
16. **Buat Schema Khusus LIMS:** Hubungkan ke database baru tersebut (\c
lims_prod_db) sebagai user lims_app (atau postgres lalu set schema owner):
17. \c lims_prod_db
- 18.
19. -- Membuat schema lims agar seluruh tabel rapi terkelompok
20. CREATE SCHEMA lims AUTHORIZATION lims_app;
21. **Aktifkan Ekstensi Chatbot (pgvector) & UUID:** Di dalam
database lims_prod_db, aktifkan ekstensi berikut (harus dijalankan oleh
superuser postgres):
22. -- Mengaktifkan ekstensi pencarian vektor untuk RAG Chatbot
23. CREATE EXTENSION IF NOT EXISTS vector SCHEMA public;
- 24.
25. -- Mengaktifkan generator UUID
26. CREATE EXTENSION IF NOT EXISTS "uuid-oss" SCHEMA public;

7. Backup & Restore Database (pg_dump & pg_restore)

LIMS secara bawaan menyediakan fitur *Backup & Restore* melalui UI halaman Maintenance. Namun, administrator sistem direkomendasikan untuk melakukan backup terjadwal secara mandiri menggunakan utilitas bawaan PostgreSQL.

A. Menyiapkan File Kredensial Otomatis (.pgpass)

Agar *cron job* dapat melakukan backup tanpa jeda menunggu *prompt password*, siapkan file kredensial rahasia di folder user Linux (misal: user lims):

1. Buat file: nano ~/.pgpass
2. Isi dengan format: hostname:port:database:username:password Jika Anda memiliki **satu** database: localhost:5432:lims_prod_db:admin_lims:PasswordSangatKuat123 Jika Anda memiliki **lebih dari satu** database dengan user yang sama (menggunakan *wildcard **): localhost:5432:*:admin_lims:PasswordSangatKuat123
3. Amankan hak aksesnya (WAJIB): chmod 0600 ~/.pgpass (Catatan: Jika hak akses bukan 0600, PostgreSQL akan mengabaikan file ini dan proses backup terjadwal akan gagal/hang).

B. Melakukan Backup (Dump)

Jalankan perintah berikut. Anda disarankan menempelkan atribut tanggal (*timestamp*) menggunakan \$(date +%Y%m%d) pada nama file agar backup harian tidak saling menimpa.

Backup Single Database

```
pg_dump -h localhost -p 5432 -U admin_lims lims_prod_db -F c >
/home/lims/backup/lims_backup_$(date +%Y%m%d).dump
```

Backup Multi-Database (Dijalankan berurutan dengan &&)

```
pg_dump -h localhost -p 5432 -U admin_lims lims_prod_db -F c >
/home/lims/backup/lims_backup_$(date +%Y%m%d).dump && pg_dump -h localhost -
p 5432 -U admin_lims chatbot_db -F c > /home/lims/backup/chatbot_backup_$(date
+%Y%m%d).dump
```

(Parameter -F c berarti file disimpan dalam format kompresi biner kustom PostgreSQL, bukan plain text SQL. Ini jauh lebih ringan dan aman).

C. Melakukan Restore (Pemulihan Data)

Opsi 1: Restore via Terminal (CLI) Jika terjadi insiden dan Anda perlu memulihkan data dari file .dump melalui terminal server:

=== Restore Database Pertama (LIMS) ===

1. Hapus dan buat ulang database kosong (opsional, disarankan agar bersih)

```
dropdb -h localhost -U admin_lims lims_prod_db
createdb -h localhost -U admin_lims lims_prod_db
```

2. Lakukan Restore LIMS

```
pg_restore -h localhost -p 5432 -U admin_lims -d lims_prod_db -1
/home/lims/backup/lims_backup_20260722.dump
```

=== Restore Database Kedua (Chatbot) ===

1. Hapus dan buat ulang database kosong

```
dropdb -h localhost -U admin_lims chatbot_db
createdb -h localhost -U admin_lims chatbot_db
```

2. Lakukan Restore Chatbot

```
pg_restore -h localhost -p 5432 -U admin_lims -d chatbot_db -1
/home/lims/backup/chatbot_backup_20260722.dump
```

Opsi 2: Restore via pgAdmin 4 (Visual/UI) Bagi Administrator yang lebih nyaman menggunakan *Graphical User Interface* (GUI):

1. **Download Backup:** Pastikan file hasil .dump dari VPS sudah Anda pindahkan (*download* via scp atau SFTP) ke komputer lokal Anda.
2. **Siapkan Database Kosong:** Di pgAdmin, buat atau pastikan Anda memiliki database kosong sebagai target *restore* (misal: lims_prod_db).
3. **Mulai Restore:** Klik kanan pada nama database target tersebut →→ Pilih **Restore...**
4. Pada tab **General:**
 - Kolom **Format:** Pilih **Custom or tar** (karena kita mem-backup menggunakan parameter -F c).
 - Kolom **Filename:** Klik ikon Folder . *Catatan Khusus:* Jika Anda menggunakan pgAdmin versi Web, Anda harus mengunggah filenya terlebih dahulu menggunakan ikon Upload (panah atas) di jendela *Storage Manager*. Jika menggunakan versi Desktop, cukup cari file di komputer Anda. Klik file tersebut lalu klik Select.

- Kolom **Role name**: Pilih user pemilik (misal admin_lims).
- 5. Pada tab **Data Options / Options**:
 - Centang **Clean before restore** (hanya jika database tujuan sudah ada isinya dan Anda ingin menyimpannya dari awal).
- 6. Terakhir, klik **Restore** dan tunggu hingga muncul notifikasi sukses berwarna hijau. Klik kanan database lalu pilih **Refresh** untuk melihat tabel yang berhasil dipulihkan.

D. Penjadwalan Otomatis dengan Crontab

Untuk menghindari risiko kelupaan, sangat disarankan untuk menjadwalkan proses backup (pg_dump) di atas agar berjalan otomatis setiap hari. **Opsi 1: Satu Baris di Crontab (Cocok untuk 1-2 Database)**

1. Buka editor cron untuk user aplikasi Anda (misal lims):
2. crontab -e
3. Tambahkan baris berikut di paling bawah untuk melakukan backup otomatis setiap jam **02:00 dini hari**:
4. `0 2 * * * pg_dump -h localhost -p 5432 -U admin_lims lims_prod_db -F c > /home/lims/backup/lims_backup_$(date +%Y%m%d).dump 2>> /home/lims/backup/backup_error.log`

(Catatan Teknis: Karakter % pada sintaks date di dalam crontab wajib didahului dengan backslash \ atau di-escape agar tidak dibaca sebagai perintah baris baru).

Opsi 2: Menggunakan Bash Script (Sangat Disarankan untuk Multi-Database) Jika Anda memiliki banyak database, menaruhnya di crontab akan sangat panjang dan rentan *typo*. Lebih baik gunakan skrip.

1. Buat file eksekusi: nano /home/lims/backup_semua_db.sh
2. Isi file tersebut dengan skrip berikut:
3. `#!/bin/bash`
4. `TANGGAL=$(date +%Y%m%d)`
- 5.
6. `# Backup LIMS`
7. `pg_dump -h localhost -p 5432 -U admin_lims lims_prod_db -F c > /home/lims/backup/lims_backup_$TANGGAL.dump`
- 8.
9. `# Backup Chatbot`
10. `pg_dump -h localhost -p 5432 -U admin_lims chatbot_db -F c > /home/lims/backup/chatbot_backup_$TANGGAL.dump`
11. Beri hak akses eksekusi: `chmod +x /home/lims/backup_semua_db.sh`
12. Panggil skrip tersebut di dalam **Crontab** (crontab -e):
13. `0 2 * * * /home/lims/backup_semua_db.sh 2>> /home/lims/backup/error.log`

Object Storage (MinIO)

MinIO digunakan secara ekstensif untuk menyimpan file mentah dan lampiran bukti dokumen (seperti dokumen PDF, foto hasil uji lapangan, lampiran nota Reimbursement, berkas SPD, dan bukti Cash Advance) yang tidak efisien jika dimasukkan ke database relasional. Modul Keuangan & Perjalanan (Finance & Travel) sepenuhnya bergantung pada MinIO.

1. Instalasi

- **Melalui Paket Manager Linux (Standalone):** Unduh biner MinIO:

- `wget https://dl.min.io/server/minio/release/linux-amd64/minio`
- `chmod +x minio`
- `sudo mv minio /usr/local/bin/`

- **Melalui Docker Container (Rekomendasi):**

- `docker run -d \`
- `-p 9000:9000 \`
- `-p 9001:9001 \`
- `--name lims-minio \`
- `-e "MINIO_ROOT_USER=adminmiliter" \`
- `-e "MINIO_ROOT_PASSWORD=password12345" \`
- `-v minio_data:/data \`
- `--restart always \`
- `minio/minio server /data --console-address ":9001"`

2. Pemeriksaan Instalasi (Health Check)

Lakukan query API healthcheck bawaan MinIO:

`curl -I http://localhost:9000/minio/health/live`

Output sukses: HTTP/1.1 200 OK

3. Konfigurasi

- Pastikan bucket bernama **mec-docs** telah dibuat saat instalasi awal.
- Anda dapat mengakses Dashboard UI Konsol di `http://[IP_SERVER]:9001` untuk membuat bucket secara visual.

4. Status & Pemantauan (Status & Monitoring)

- **Cek Status Proses:**
- *# Standalone*
- `ps aux | grep minio`
- *# Docker*
- `docker ps | grep lims-minio`
- **Memantau Dashboard Konsol:** Buka `http://localhost:9001` di browser Anda untuk melihat total memori terpakai, total bucket, dan kapasitas tersisa.

5. Penelusuran Error (Tracing & Logs)

- **Membaca Log MinIO:**
 - *Standalone:* Mengalihkan output ke file log saat dijalankan (minio server /data > /var/log/minio.log 2>&1 &)
 - *Docker:* `docker logs -f lims-minio`

6. Panduan Akses Konsol Web MinIO di VPS

Karena port admin Console 9001 biasanya ditutup demi alasan keamanan pada firewall VPS, Anda direkomendasikan mengaksesnya menggunakan **SSH Tunneling (Port Forwarding)** dari komputer lokal Anda:

1. **Jalankan SSH Tunneling di terminal lokal Anda (Windows/Linux):**
2. `ssh -L 9001:127.0.0.1:9001 -N username_vps@IP_VPS_ANDA`

Masukkan sandi SSH VPS Anda. Jendela terminal ini akan tetap terbuka secara "diam" untuk menahan terowongan.

3. **Buka Web Console di browser lokal Anda:** Akses alamat `http://localhost:9001`
4. **Kredensial Login Default:**
 - **Access Key (Username):** adminmiliter

- **Secret Key (Password):** password12345

7. Backup & Restore MinIO

Untuk melakukan *backup* (cadangan) dan *restore* (pemulihan) pada MinIO, disarankan menggunakan **Metode MinIO Client (mc)** (Logis) agar sistem bisa tetap berjalan tanpa *downtime*, atau **Metode File/Volume Fisik**. Panduan berikut meliputi platform Linux (Docker) dan Windows (Standalone).

A. Metode Menggunakan MinIO Client / mc (Rekomendasi - Zero Downtime)

Metode ini paling aman karena berjalan via API MinIO, memastikan tidak ada kerusakan file jika ada proses tulis (*write*) yang berlangsung bersamaan.

1. Instalasi dan Konfigurasi mc

- **Di Linux:**
- `wget https://dl.min.io/client/mc/release/linux-amd64/mc`
- `chmod +x mc`
- `sudo mv mc /usr/local/bin/`
- **Di Windows:** Unduh mc.exe dari <https://dl.min.io/client/mc/release/windows-amd64/mc.exe>, lalu ubah namanya menjadi mc.exe. Buka PowerShell di direktori tempat mc.exe tersebut berada.

2. Menghubungkan mc ke Server LIMS Anda: (Sama untuk Linux maupun Windows, pastikan Anda menggunakan port API yaitu 9000, BUKAN port Web UI 9001!)

- **Linux:**
- mc alias set myminio `http://localhost:9000 adminmiliter password12345`
- **Windows:** (Perhatikan penggunaan `.\` di depan perintah)
- `.\mc.exe` alias set myminio `http://localhost:9000 adminmiliter password12345`

3. Melakukan Backup (Mirroring): Menduplikasi isi *bucket* ke folder lokal komputer/server Anda (misal ke direktori `/backup/minio_backup` di Linux, atau `D:\backup\minio_backup` di Windows).

[NOTE] **Backup Inkremental:** Perintah mc mirror secara otomatis bekerja secara **inkremental**. Jika Anda menjalankannya secara berkala (misal tiap hari), ia hanya akan mengunduh file baru atau yang berubah, sehingga tidak membebani server dan selesai dalam hitungan detik.

Opsi Tambahan yang Berguna:

- `--remove` : Jika file dihapus di server MinIO, maka file tersebut juga akan dihapus di folder backup lokal Anda (membentuk *exact mirror*).
- `--watch` atau `-w` : *Backup* berjalan terus menerus secara instan/real-time tanpa perlu dijadwalkan setiap hari.
- *Menyimpan Log* : Secara default, log akan tampil di layar. Untuk menyimpannya ke dalam file, tambahkan operator redireksi `>> backup.log 2>&1` (di Linux) atau Tee-Object (di PowerShell).
- **Linux:**
- `mkdir -p ~/minio_backup`
-
- *# Backup Standar (Simpan Log ke file backup.log)*
- `mc mirror myminio/lims-docs ~/minio_backup/lims-docs >> ~/minio_backup/backup.log 2>&1`
-
- *# Backup Real-time dengan Penghapusan (Mirror Identik)*

- `mc mirror --watch --remove myminio/lims-docs ~/minio_backup/lims-docs >> ~/minio_backup/backup.log 2>&1`
- **Windows:**
- `mkdir D:\backup\minio_backup`
-
- *# Backup Standar (Simpan Log ke file backup.log)*
- `.\mc.exe mirror myminio/lims-docs D:\backup\minio_backup\lims-docs | Tee-Object -FilePath D:\backup\minio_backup\backup.log -Append`
-
- *# Backup Real-time dengan Penghapusan (Mirror Identik)*
- `.\mc.exe mirror --watch --remove myminio/lims-docs D:\backup\minio_backup\lims-docs | Tee-Object -FilePath D:\backup\minio_backup\backup.log -Append`

4. Melakukan Restore: Mengembalikan file dari backup lokal ke dalam MinIO. Balik urutan path mirrornya.

Buat bucket baru jika belum ada

Linux

`mc mb myminio/lims-docs`

Windows

`.\mc.exe mb myminio/lims-docs`

Restore (Linux)

`mc mirror ~/minio_backup/lims-docs myminio/lims-docs`

Restore (Windows)

`.\mc.exe mirror D:\backup\minio_backup\lims-docs myminio/lims-docs`

B. Metode File / Volume Level (Fisik)

Metode ini adalah *copy-paste* folder/data asli secara langsung dari sistem operasi ke penyimpanan eksternal. **Sangat disarankan untuk mematikan (stop) server MinIO terlebih dahulu** selama proses *copy* berlangsung agar file tidak korup.

- **Di Linux (Jika Menggunakan Docker):**
 1. Hentikan container: `docker stop lims-minio`
 2. Cari lokasi volume
(biasanya `/var/lib/docker/volumes/minio_data/_data`): `docker volume inspect minio_data`
 3. Backup (Arsipkan): `sudo tar -czvf /backup/minio_backup_data.tar.gz /var/lib/docker/volumes/minio_data/_data`
 4. Restore: Ekstrak file arsip (tar -xzvf) kembali ke direktori mount point volume tersebut.
 5. Jalankan kembali: `docker start lims-minio`
- **Di Windows (Jika Menggunakan .exe Standalone):**
 1. Hentikan (Stop) proses minio.exe atau service MinIO yang sedang berjalan (bisa melalui *Task Manager* atau dengan menekan Ctrl+C di PowerShell tempat aplikasi berjalan).
 2. Cari direktori tempat data MinIO Anda disimpan (misalnya `D:\Source\minio\minio_data`).

3. **Backup:** Jalankan perintah PowerShell berikut untuk mengarsipkan (*zip*) folder data:
4. `Compress-Archive -Path D:\Source\minio\minio_data -DestinationPath D:\backup\minio_backup_data.zip`
5. **Restore:** Kosongkan/hapus folder data lama yang rusak, lalu jalankan perintah PowerShell berikut untuk mengekstrak file backup ke lokasi asalnya:
6. `Remove-Item -Recurse -Force D:\Source\minio\minio_data\*`
7. `Expand-Archive -Path D:\backup\minio_backup_data.zip -DestinationPath D:\Source\minio\ -Force`
8. Jalankan kembali aplikasi minio.exe.

Workflow Engine (Camunda BPMN)

Camunda digunakan oleh LIMS sebagai pengendali workflow status persetujuan (*approval process*) hasil uji laboratorium secara dinamis sesuai standar BPMN 2.0.

1. Instalasi

- **Melalui Docker Container (Sangat Direkomendasikan):**

- `docker run -d \`
- `--name lims-camunda \`
- `-p 8085:8080 \`
- `--restart always \`
- `camunda/camunda-bpm-platform:run-latest`

2. Pemeriksaan Instalasi (Health Check)

Cek respon dari Rest API Engine Camunda:

`curl http://localhost:8085/engine-rest/engine`

Output sukses (JSON): [{"name": "default"}]

3. Konfigurasi

Aplikasi Go Backend berkomunikasi dengan Camunda melalui variabel `.env` berikut:

`CAMUNDA_URL=http://127.0.0.1:8085/engine-rest`

`CAMUNDA_USER=nurcamunda`

`CAMUNDA_PASSWORD_ENCRYPTED=7Qnql7OCZ046YGu2pY3FAOQFg8TLv3u5k2A=`

Kredensial login admin default untuk Web Console Camunda Cockpit adalah demo/demo.

4. Status & Pemantauan (Status & Monitoring)

- **Cek Status Proses:**
- `docker ps | grep lims-camunda`
- **Cockpit UI Monitoring:** Buka `http://localhost:8085/camunda/app/cockpit/` di browser untuk memantau instance workflow berjalan, riwayat tugas, kegagalan task (*incidents*), serta diagram BPMN aktif.

5. Penelusuran Error (Tracing & Logs)

- **Membaca Log Camunda (Tomcat/Spring-Boot):**
- `docker logs -f lims-camunda`

Cari baris yang mengandung tulisan ENGINE-00000 atau ERROR untuk melacak kegagalan eksekusi skrip Java delegate atau kegagalan koneksi database.

6. Panduan Akses Konsol Web Camunda Cockpit di VPS

Karena port 8085 biasanya ditutup demi alasan keamanan pada firewall VPS, gunakan **SSH Tunneling (Port Forwarding)** dari komputer lokal Anda:

1. **Jalankan SSH Tunneling di terminal lokal Anda (Windows/Linux):**

2. `ssh -L 8085:127.0.0.1:8085 -N username_vps@IP_VPS_ANDA`

Masukkan sandi SSH VPS Anda. Jendela terminal ini akan tetap terbuka secara "diam" untuk menahan terowongan.

3. **Buka Web Console di browser lokal Anda:** Akses alamat **`http://localhost:8085/camunda/app/cockpit/`**

4. **Kredensial Login Default:**

- **Username:** demo

- **Password:** demo

(Tip: Anda dapat memetakan port Camunda dan MinIO sekaligus dalam satu perintah: `ssh -L 8085:127.0.0.1:8085 -L 9001:127.0.0.1:9001 -N username_vps@IP_VPS_ANDA`)

Message Broker (MQTT - Mosquitto) di VPS

Mosquitto Broker digunakan untuk menerima dan mendistribusikan data telemetri dari peralatan uji militer secara *real-time*.

1. Instalasi & Aktivasi

Jalankan perintah berikut di terminal VPS Anda:

1. Perbarui daftar paket dan instal Mosquitto

```
sudo apt update
```

```
sudo apt install mosquitto mosquitto-clients -y
```

2. Aktifkan agar otomatis menyala saat boot

```
sudo systemctl enable mosquitto
```

3. Jalankan broker Mosquitto

```
sudo systemctl start mosquitto
```

2. Konfigurasi Akses Luar (Port 1883)

Pada Mosquitto versi 2.0+, akses eksternal secara default diblokir kecuali dikonfigurasi secara eksplisit.

1. Buat berkas konfigurasi baru di VPS Anda:

2. `sudo nano /etc/mosquitto/conf.d/lms-mqtt.conf`

3. Tempelkan baris berikut untuk membuka akses port 1883 tanpa sandi (untuk kebutuhan pengujian):

4. `listener 1883`

5. `allow_anonymous true`

6. Simpan berkas, lalu muat ulang Mosquitto:

7. `sudo systemctl restart mosquitto`

3. Pemeriksaan Status & Logs

Cek status berjalan

```
sudo systemctl status mosquitto
```

Pantau logs real-time Mosquitto

```
tail -f /var/log/mosquitto/mosquitto.log
```

Middleware Integrasi Data (Node-RED) di VPS

Node-RED digunakan sebagai data pipeline untuk menerjemahkan data MQTT/Serial dari peralatan uji sebelum dikirim ke LIMS API.

1. Instalasi Node-RED di VPS

Node-RED berjalan di atas Node.js. Jalankan perintah instalasi berikut secara global di VPS:

```
sudo npm install -g --unsafe-perm node-red
```

2. Menjalankan Node-RED di Latar Belakang (Menggunakan PM2)

Karena PM2 sudah terpasang di VPS Anda untuk menyajikan frontend, gunakan PM2 untuk mengelola dan memantau Node-RED dengan aman:

1. Jalankan Node-RED di PM2

```
pm2 start node-red --name "lims-nodered" -- --max-old-space-size=512
```

2. Simpan daftar proses aktif agar otomatis menyala saat reboot

```
pm2 save
```

3. Akses Dashboard Admin Node-RED

Secara default, Node-RED mendengarkan port **1880**. Untuk alasan keamanan, port ini ditutup pada firewall VPS. Akses dashboard Node-RED menggunakan SSH Tunneling dari komputer lokal Anda:

1. **Buka terowongan SSH dari terminal laptop lokal Anda:**
2. `ssh -L 1880:127.0.0.1:1880 -N lims@212.85.24.33`
3. **Akses via browser lokal Anda: → <http://localhost:1880>**

4. Memantau Log Node-RED

```
pm2 logs lims-nodered
```

5. Cara Mentransfer Konfigurasi/Alur (Flows) Node-RED dari Lokal ke VPS

Anda memiliki dua opsi mudah untuk memindahkan seluruh rancangan alur (*flows*) Node-RED dari laptop lokal Anda ke VPS:

Opsi A: Ekspor & Impor via Web Console (Paling Mudah & Direkomendasikan)

1. Buka **Node-RED Lokal** Anda (<http://localhost:1880>) di browser laptop.
2. Klik tombol menu (garis tiga) di sudut kanan atas -> pilih **Export**.
3. Pilih tab **JSON**, centang **All flows**, lalu klik **Download JSON** (atau pilih *Copy to Clipboard*).
4. Hubungkan laptop Anda ke **Node-RED VPS** menggunakan terowongan SSH (`ssh -L 1880:127.0.0.1:1880 ...`), lalu buka <http://localhost:1880> di browser.
5. Klik tombol menu kanan atas -> pilih **Import**.
6. Unggah berkas JSON yang diunduh tadi (atau tempelkan kodenya), lalu klik **Import**.
7. Klik tombol merah **Deploy** di kanan atas VPS Node-RED untuk mengaktifkan alur.

Opsi B: Salin Berkas flows.json Secara Langsung (CLI)

Node-RED menyimpan seluruh konfigurasi dalam satu berkas JSON di dalam folder `.node-red` pengguna.

1. **Lokasi Berkas Lokal (Windows):** `C:\Users\<Nama_User_Laptop>\.node-red\flows.json` (atau `flows_<nama_komputer>.json`)

2. **Lokasi Berkas Target di VPS:** /home/lims/.node-red/flows.json (atau nama host VPS /home/lims/.node-red/flows_srv1801975.json)
3. Kirim berkas lokal langsung menggunakan scp dari terminal Ubuntu/WSL lokal Anda ke VPS:
4. `scp /mnt/c/Users/<Nama_User_Laptop>/.node-red/flows.json lims@212.85.24.33:/home/lims/.node-red/flows.json`
5. Masuk ke terminal VPS, lalu restart Node-RED agar memuat berkas konfigurasi baru tersebut:
6. `pm2 restart lims-nodered`

L. Akses LIMS dari Internet (Ngrok Dev & Produksi Asli)

Untuk kebutuhan pengembangan (*development*), LIMS sering kali perlu diakses dari luar jaringan lokal (misalnya menggunakan HP via koneksi seluler). Sedangkan pada deployment produksi asli, kita menggunakan konfigurasi jaringan resmi.

Menggunakan Ngrok untuk Akses Internet saat Development

Ngrok membuat terowongan aman (*secure tunnel*) dari internet langsung ke port Nginx lokal Anda.

1. Cara Menjalankan Ngrok

Skenario A: Menghubungkan Ngrok ke Nginx Load Balancer (Mendukung Multiple Server - Sangat Direkomendasikan)

Jika Anda ingin agar lalu lintas dari luar tetap melewati Load Balancer Nginx (sehingga request dibagi secara merata ke backend port 8081 and 8091) menggunakan konfigurasi HTTPS port 8082 yang sudah diimplementasikan:

Jalankan Ngrok dengan mengarahkan langsung ke HTTPS port 8082 Nginx lokal Anda:

Menghubungkan Ngrok ke HTTPS Port 8082 Nginx Load Balancer

`ngrok http https://localhost:8082`

Catatan: Jika Nginx Anda mengaktifkan pencocokan host-name ketat pada server_name, Anda dapat menambahkan argumen --host-header="lims.local":

`ngrok http https://localhost:8082 --host-header="lims.local"`

Setelah dijalankan, Ngrok akan memberikan URL publik seperti: `https://a1b2-c3d4.ngrok-free.app`

Dengan cara ini, APK disetting dengan `VITE_API_URL=https://a1b2-c3d4.ngrok-free.app/api`. Request dari HP akan masuk ke Nginx port 8082, dan Nginx secara dinamis membagi beban kerja ke backend port 8081 & 8091 (Load Balancing) secara otomatis!

Skenario B: Menghubungkan Ngrok Langsung ke Salah Satu Server Backend (Tanpa Load Balancer)

Jika Anda hanya ingin melakukan debugging cepat pada satu backend saja (misal backend port 8081) tanpa load balancing:

Menghubungkan Ngrok langsung ke Go Backend port 8081

`ngrok http 8081`

Dengan cara ini, request langsung masuk ke Go backend port 8081 tanpa melewati Nginx, sehingga tidak ada pembagian beban kerja (no load balancing).

2. Konfigurasi Variabel .env (Jika URL Ngrok Digunakan)

Jika Anda menggunakan URL publik Ngrok untuk pengujian terintegrasi (seperti webhook atau callback pihak ketiga), perbarui variabel `SERVER_DOMAIN` di file `.env` backend Anda:

SERVER_DOMAIN=a1b2-c3d4.ngrok-free.app

Dampak jika Tidak Menjalankan Ngrok (Selama Development)

Jika Ngrok tidak dijalankan selama masa pengembangan:

1. **Akses Terbatas:** Aplikasi LIMS hanya dapat diakses dari mesin lokal (localhost) atau perangkat lain yang berada dalam satu jaringan lokal (LAN) yang sama.
2. **Tidak Bisa Diakses via Seluler:** Anda tidak dapat menguji aplikasi menggunakan browser HP jika HP terhubung ke paket data seluler (di luar jaringan Wi-Fi lokal).
3. **Webhook Terputus:** Sistem luar tidak dapat mengirimkan callback atau notifikasi langsung ke server LIMS Anda (misalnya integrasi API dari sistem luar).

Konfigurasi Akses Jaringan Permanen Setelah Deployment (Produksi Asli)

Pada lingkungan produksi asli, kita **not** menggunakan Ngrok karena URL Ngrok bersifat sementara dan memiliki keterbatasan bandwidth. Sebagai gantinya, gunakan langkah-langkah standar berikut:

1. IP Publik Statis & DNS

- Dapatkan **IP Publik Statis** dari ISP Anda atau gunakan layanan Cloud Virtual Machine (AWS, GCP, DigitalOcean, Alibaba Cloud) yang sudah menyediakan IP publik.
- Daftarkan nama domain resmi (contoh: lims.perusahaan.com).
- Pada panel penyedia domain Anda, arahkan **A Record** DNS domain tersebut ke IP Publik Server Anda:
- Nama Host: lims.perusahaan.com --> Alamat IP: [IP_PUBLIK_SERVER]

2. Port Forwarding di Router / Firewall (Jika Server On-Premise)

Jika server LIMS diletakkan di dalam jaringan kantor/pabrik (on-premise) di balik router:

- Buka pengaturan port forwarding (NAT) pada router utama Anda.
- Teruskan lalu lintas dari port internet luar berikut ke IP lokal server LIMS:
 - Port luar 80 (HTTP) --> Port lokal server LIMS 8088 (atau 80)
 - Port luar 443 (HTTPS) --> Port lokal server LIMS 8082 (atau 443)

3. Penyesuaian Konfigurasi Nginx (server_name & SSL Let's Encrypt)

Perbarui berkas /etc/nginx/conf.d/lims.conf agar mengenali nama domain resmi Anda pada parameter server_name:

```
server {  
    listen 80;  
    server_name lims.perusahaan.com;  
    return 301 https://$host$request_uri; # Redirect otomatis ke HTTPS  
}
```

```
server {  
    listen 443 ssl; # Port HTTPS standar produksi  
    server_name lims.perusahaan.com;
```

```
    root /home/lims/lims1/frontend/dist;
```

```
    ...
```

```
}
```

Di lingkungan produksi asli, jangan gunakan sertifikat *self-signed*. Gunakan sertifikat SSL terpercaya dari **Let's Encrypt** secara gratis:

```
# Instal certbot untuk Nginx
```

```
sudo apt install certbot python3-certbot-nginx -y
```

```
# Jalankan certbot untuk mendapatkan & memasang SSL otomatis di konfigurasi Nginx
```

```
sudo certbot --nginx -d lims.perusahaan.com
```

Certbot akan secara otomatis memperbarui file konfigurasi Nginx untuk menggunakan sertifikat SSL terpercaya dan mengatur pengalihan (redirect) otomatis dari HTTP ke HTTPS.

4. Penyesuaian Arsitektur Dedicated VM Nginx (Real VM)

Jika Nginx Load Balancer dipasang pada **dedicated VM** (VM khusus) yang terpisah dari VM Backend Aplikasi LIMS, ikuti penyesuaian parameter berikut:

A. Konfigurasi `set_real_ip_from` (Membaca IP Client Asli)

- **Pada VM Nginx Load Balancer (Edge Gateway):**
 - Jika Nginx Load Balancer menerima request langsung dari internet (tanpa ada proxy lain di depannya seperti Cloudflare, AWS ALB, dll), Anda **tidak perlu** menggunakan `set_real_ip_from` karena Nginx dapat membaca IP client asli secara langsung melalui variabel bawaan `$remote_addr`.
 - Namun, jika Nginx berada di balik CDN/Proxy luar (seperti Cloudflare), Anda wajib mengatur IP trust range Cloudflare:
 - *# Percayai header X-Forwarded-For dari subnet IP Cloudflare*
 - `set_real_ip_from 103.21.244.0/22;`
 - `set_real_ip_from 103.22.200.0/22;`
 - *# ... (tambahkan IP Cloudflare lainnya)*
 - `real_ip_header X-Forwarded-For;`
- **Pada VM Go Backend (Internal Server):**
 - Jika di VM Backend juga dipasang Nginx lokal sebelum masuk ke aplikasi Go, Nginx lokal tersebut harus mempercayai IP VM Nginx Load Balancer agar tidak mencatat IP Load Balancer sebagai pengirim request:
 - *# Misal IP VM Nginx Load Balancer adalah 192.168.10.10*
 - `set_real_ip_from 192.168.10.10;`
 - `real_ip_header X-Forwarded-For;`
 - `real_ip_recursive on;`

B. Konfigurasi `proxy_pass` untuk GoAccess (/ws)

- Pada Nginx Load Balancer, parameter `proxy_pass` `http://127.0.0.1:7890;` digunakan untuk meneruskan koneksi WebSocket GoAccess.
- **Jika GoAccess berjalan di VM yang sama dengan Nginx:** Tetap gunakan `127.0.0.1:7890`.
- **Jika GoAccess berjalan di VM Backend terpisah (misal VM-1 Backend IP 192.168.10.21):** Ubah parameter tersebut untuk menunjuk ke IP internal VM Backend tersebut:
- `location /ws {`

- proxy_pass http://192.168.10.21:7890; # Arahkan ke VM tempat GoAccess berjalan
- proxy_http_version 1.1;
- proxy_set_header Upgrade \$http_upgrade;
- proxy_set_header Connection "Upgrade";
- }

C. Konfigurasi Aplikasi Mobile (HP) dengan Domain Resmi

Ketika LIMS sudah berjalan menggunakan domain resmi dengan SSL Let's Encrypt yang valid (misal <https://lims.perusahaan.com>):

1. HP client **tidak perlu lagi menggunakan port HTTP khusus 8087** untuk bypass SSL.
2. HP client dapat langsung terhubung via HTTPS port standar 443 karena Let's Encrypt adalah Certificate Authority resmi yang dipercayai oleh sistem operasi Android secara bawaan.
3. Di HP Android, buka roda gigi (**Gear Icon**), pilih mode **Internet (Ngrok/Cloud)**, lalu masukkan alamat domain resmi Anda: <https://lims.perusahaan.com>. HP akan secara aman mengakses LIMS baik via Wi-Fi kantor maupun paket data internet seluler secara *seamless*.

M. Ringkasan Lokasi Berkas Log LIMS

Untuk memudahkan pemantauan (*monitoring*) dan pelacakan kesalahan (*debugging*), berikut adalah daftar seluruh berkas log dalam ekosistem LIMS beserta perintah untuk membacanya:

Komponen	Nama Berkas Log	Path Absolut	Format	Perintah Membaca Log
Nginx Web Server	Access Log	/var/log/nginx/lims_access.log	Teks	tail -f /var/log/nginx/lims_access.log
Nginx Web Server	Error Log	/var/log/nginx/lims_error.log	Teks	tail -f /var/log/nginx/lims_error.log
Go Backend 1 (8081)	Systemd Log	<i>Terekam di journald</i>	Biner (systemd)	sudo journalctl -u lims-backend-8081.service -f -n 100
Go Backend 2 (8091)	Systemd Log	<i>Terekam di journald</i>	Biner (systemd)	sudo journalctl -u lims-backend-8091.service -f -n 100
Go Backend 1 (8081)	Traffic API Log	/home/lims/lims1/backend/logs/api_traffic.log	Teks	tail -f /home/lims/lims1/backend/logs/api_traffic.log

Komponen	Nama Berkas Log	Path Absolut	Format	Perintah Membaca Log
Go Backend 1 (8081)	SQL Query Log	/home/lims/lims1/backend/logs/db_query.log	Teks	tail -f /home/lims/lims1/backend/logs/db_query.log
Go Backend 2 (8091)	Traffic API Log	/home/lims/lims2/backend/logs/api_traffic.log	Teks	tail -f /home/lims/lims2/backend/logs/api_traffic.log
Go Backend 2 (8091)	SQL Query Log	/home/lims/lims2/backend/logs/db_query.log	Teks	tail -f /home/lims/lims2/backend/logs/db_query.log
Model AI Retraining	AI Training Log	/home/lims/lims1/backend/logs/lims_ai-train-YYYYMMDD.log	Teks	cat /home/lims/lims1/backend/logs/lims_ai-train-*.log
Frontend 1 (3000)	PM2 Output Log	/home/lims/.pm2/logs/lims-frontend-3000-out.log	Teks	pm2 logs lims-frontend-3000
Frontend 1 (3000)	PM2 Error Log	/home/lims/.pm2/logs/lims-frontend-3000-error.log	Teks	pm2 logs lims-frontend-3000
Frontend 2 (3001)	PM2 Output Log	/home/lims/.pm2/logs/lims-frontend-3001-out.log	Teks	pm2 logs lims-frontend-3001
Frontend 2 (3001)	PM2 Error Log	/home/lims/.pm2/logs/lims-frontend-3001-error.log	Teks	pm2 logs lims-frontend-3001
Logrotate Schedule	Nginx Rotate Log	/var/log/logrotate_nginx.log	Teks	cat /var/log/logrotate_nginx.log
Logrotate Schedule	Backend Rotate Log	/var/log/logrotate_backend.log	Teks	cat /var/log/logrotate_backend.log
Logrotate Schedule	Frontend Rotate Log	/var/log/logrotate_frontend.log	Teks	cat /var/log/logrotate_frontend.log

[!NOTE] **Penyimpanan Berkas Systemd Log (journald):** Berkas log systemd backend yang diakses menggunakan journalctl disimpan secara fisik (persisten) di disk Linux pada direktori: /var/log/journal/[machine-id]/ (contoh: /var/log/journal/1f32c6465a29450093352d06f0af7a2e/). Berkas ini disimpan dalam format **biner** (bukan teks biasa) demi keamanan, penghematan ruang penyimpanan, dan optimalisasi pencarian cepat. Untuk membacanya, Anda harus menggunakan perintah journalctl.

A. Pembersihan & Pengosongan Log Biner secara Manual (Clean-Up Journald)

Jika direktori log biner systemd mulai memenuhi kapasitas penyimpanan disk Anda, Anda dapat melakukan pembersihan log lama secara manual menggunakan perintah berikut:

1. Bersihkan semua log yang berumur lebih dari 7 hari (*Keep 7 hari terakhir*)
sudo journalctl --vacuum-time=7d

2. Batasi kapasitas total penyimpanan berkas log biner menjadi maksimal 500MB
sudo journalctl --vacuum-size=500M

B. Konfigurasi Batasan Masa Simpan Log Otomatis (Retention Configuration)

Agar sistem secara otomatis membatasi kapasitas log dan membuang log lama tanpa intervensi manual, Anda wajib mengonfigurasi berkas /etc/systemd/journald.conf:

1. Buka dan edit file konfigurasi:
2. sudo nano /etc/systemd/journald.conf
3. Cari dan edit/tambahkan baris konfigurasi berikut (hilangkan tanda komentar # jika ada):
4. [Journal]
5. # Menjamin penyimpanan log ditulis permanen ke disk /var/log/journal/
6. Storage=persistent
- 7.
8. # Batas maksimal ukuran total folder log biner di disk (misal: 1 Gigabyte)
9. SystemMaxUse=1G
- 10.
11. # Batas ukuran maksimal per file log tunggal sebelum dirotasikan (misal: 100 Megabytes)
12. SystemMaxFileSize=100M
- 13.
14. # Batas waktu retensi penyimpanan log secara otomatis (misal: disimpan selama 14 hari saja)
15. MaxRetentionSec=14day
16. Simpan berkas tersebut, lalu jalankan perintah berikut untuk memuat ulang dan menerapkan konfigurasi journald yang baru:
17. sudo systemctl restart systemd-journald

N. Ringkasan Perintah Manajemen Modul LIMS (Start, Stop, Status)

Berikut adalah tabel referensi cepat untuk memulai (*start*), menghentikan (*stop*), dan memeriksa status (*status check*) seluruh layanan/modul dalam ekosistem LIMS:

Komponen	Nama Layanan / Proses	Perintah Menjalankan (Start)	Perintah Menghentikan (Stop)	Perintah Restart / Reload	Perintah Cek Status
Nginx Web Server	Nginx Gateway	sudo systemctl start nginx	sudo systemctl stop nginx	sudo systemctl restart nginx (atau reload)	sudo systemctl status nginx
Go Backend 1 (8081)	Systemd: lims-backend-8081	sudo systemctl start lims-backend-8081.service	sudo systemctl stop lims-backend-8081.service	sudo systemctl restart lims-backend-8081.service	sudo systemctl status lims-backend-8081.service
Go Backend 2 (8091)	Systemd: lims-backend-8091	sudo systemctl start lims-backend-8091.service	sudo systemctl stop lims-backend-8091.service	sudo systemctl restart lims-backend-8091.service	sudo systemctl status lims-backend-8091.service
Frontend 1 (3000)	PM2: lims-frontend-3000	pm2 start lims-frontend-3000	pm2 stop lims-frontend-3000	pm2 restart lims-frontend-3000	pm2 status lims-frontend-3000
Frontend 2 (3001)	PM2: lims-frontend-3001	pm2 start lims-frontend-3001	pm2 stop lims-frontend-3001	pm2 restart lims-frontend-3001	pm2 status lims-frontend-3001
PostgreS QL DB (Local)	Standalone Postgres	sudo systemctl start postgresql	sudo systemctl stop postgresql	sudo systemctl restart postgresql	sudo systemctl status postgresql
PostgreS QL DB (Docker)	Container: lims-postgres	docker start lims-postgres	docker stop lims-postgres	docker restart lims-postgres	docker ps -f name=lims-postgres

Komponen	Nama Layanan / Proses	Perintah Menjalankan (Start)	Perintah Menghentikan (Stop)	Perintah Restart / Reload	Perintah Cek Status
Object Storage	Container: lims-minio	docker start lims-minio	docker stop lims-minio	docker restart lims-minio	docker ps -f name=lims-minio
Workflow Engine	Container: lims-camunda	docker start lims-camunda	docker stop lims-camunda	docker restart lims-camunda	docker ps -f name=lims-camunda
Cron Jobs	Linux Cron Daemon	sudo systemctl start cron	sudo systemctl stop cron	sudo systemctl restart cron	sudo systemctl status cron
Real-Time Dashboard	GoAccess WebSocket	<i>Jalankan otomatis di background jika di-spawn</i>	pgrep goaccess	<i>Jalankan kembali kueri GoAccess</i>	ps aux grep goaccess

[!NOTE] Seluruh perintah PM2 wajib dijalankan di bawah pengguna **lims** (gunakan sudo su - lims terlebih dahulu) untuk mencegah ketidakcocokan hak akses file log atau sesi PM2 antar-user.

O. Strategi Manajemen Versi Aplikasi Android LIMS

Bagian ini menjelaskan arsitektur untuk memantau versi aplikasi Android LIMS yang aktif digunakan oleh user serta mekanisme pemblokiran paksa (*force upgrade*) untuk menjamin keselarasan versi API backend. Fitur ini telah sepenuhnya terintegrasi dalam sistem LIMS.

Mekanisme Pemantauan Versi Android Klien

1. **Custom Header HTTP (Direkomendasikan):** Setiap kali aplikasi Android mengirimkan permintaan API ke backend, aplikasi menyertakan informasi versi pada header HTTP:
 - o X-App-Version: 1.0 (dikonfigurasi via VITE_APP_VERSION di berkas .env frontend)
 - o X-App-Platform: Android (dideteksi dinamis menggunakan Capacitor)
2. **Pencatatan Audit Sesi Login (user_sessions):** Saat operator sukses melakukan login dari HP Android, backend menangkap header di atas dan mencatat informasi versi klien ke kolom client_version dan client_platform pada tabel lims.user_sessions.

DDL untuk menambahkan kolom:

```
ALTER TABLE lims.user_sessions
ADD COLUMN client_version VARCHAR(50) DEFAULT NULL,
```

ADD COLUMN client_platform VARCHAR(50) DEFAULT NULL;

Admin dapat memonitor statistik versi aktif via query SQL:

```
SELECT client_version, client_platform, COUNT(*)
```

```
FROM lims.user_sessions
```

```
GROUP BY client_version, client_platform;
```

3. **Analisis Traffic User-Agent (NGINX Logs):** Informasi versi juga dapat ditangkap oleh NGINX dengan memetakan User-Agent khusus (misalnya LIMS-Android/1.0) dan divisualisasikan melalui GoAccess.

Mekanisme Pemblokiran & Paksa Upgrade (Force Upgrade)

Untuk memblokir pengguna yang menggunakan versi aplikasi Android di bawah batas minimum (misal di bawah 1.1), sistem menerapkan langkah-langkah berikut:

1. **Parameterisasi Batas Versi Minimum di Database:** Tambahkan parameter global baru pada tabel lims.global_parameters (diatur dinamis via database):
 - MIN_ANDROID_VERSION (default: 1.1) - Versi minimal aplikasi Android LIMS yang diperbolehkan.
 - ANDROID_DOWNLOAD_URL - URL unduh langsung APK LIMS versi terbaru.
2. **Endpoint Pemeriksaan Versi (Public Endpoint):** Backend Go menyediakan rute publik /api/check-version yang diakses oleh Android klien saat aplikasi pertama kali dibuka (startup) dengan menyuplai query parameter version and platform:
 - **Respons API (Upgrade Dipaksa):**
 - {
 - "status": "FORCE_UPGRADE",
 - "minimum_version": "1.1",
 - "download_url": "https://lims.perusahaan.com/downloads/lims-v1.1.apk",
 - "message": "Versi aplikasi Anda (1.0) sudah tidak didukung. Harap perbarui ke versi 1.1 demi keamanan dan stabilitas."
 - }
3. **Penanganan Sisi Android (Blocking Overlay):** Jika aplikasi React Frontend mendeteksi respons "status": "FORCE_UPGRADE", aplikasi menampilkan **blocking dialog overlay** glassmorphism premium berwarna merah-gelap yang mengunci seluruh interaksi sistem dan mengarahkan pengguna untuk mengunduh APK LIMS terbaru melalui link download_url.
4. **Middleware Validasi API (Backend Hard-Block Guard):** Sebagai perlindungan lapis kedua, backend menerapkan middleware **VersionCheckMiddleware** pada seluruh rute API. Middleware ini otomatis memeriksa header X-App-Version pada request masuk. Jika versi di bawah MIN_ANDROID_VERSION, request langsung ditolak dengan status HTTP 426 Upgrade Required dan response:
 5. {
 6. "error": "UPGRADE_REQUIRED",
 7. "message": "Versi aplikasi LIMS Android Anda sudah tidak didukung. Harap perbarui ke versi terbaru."
 8. }

9. **Mode Debug vs Release (Cara Bypass saat Development):** Backend LIMS tidak membedakan apakah request datang dari APK mode *Release* ataupun APK mode *Debug*. Backend hanya membaca angka yang dikirimkan pada header *X-App-Version*. Akibatnya, APK mode *Debug* Anda akan ikut terblokir jika versinya lebih rendah dari `MIN_ANDROID_VERSION`. Untuk **menghindari pemblokiran** saat proses pengetesan (testing/development) secara lokal:
- o Buat file **.env.development** di folder frontend.
 - o Isi dengan nilai versi raksasa, contoh: `VITE_APP_VERSION=99.9`.
 - o Saat Anda menjalankan mode debug (`npm run dev`), Vite otomatis memprioritaskan membaca file `.env.development` ini, sehingga *build* lokal Anda dikenali sebagai versi 99.9 dan terhindar dari layar *Force Upgrade*.
 - o Saat di-build untuk rilis resmi (`npm run build`), sistem otomatis mengabaikan file ini dan kembali menggunakan versi asli (misal 1.4) dari file `.env` atau `.env.production`.

P. Panduan Pembuatan APK Android (Berbagai Skema Jaringan & Target)

Proses kompilasi APK Android LIMS dibagi menjadi beberapa tipe build (Debug vs Release) dan skema jaringan (Ngrok vs Wi-Fi Lokal vs Domain Resmi). Semua konfigurasi API Endpoint dipasang secara statis (*baked-in*) ke aset web pada saat proses build frontend.

Matriks Konfigurasi API Endpoint (Sisi Frontend)

Sebelum melakukan build, sesuaikan berkas frontend/`.env.production` (atau `.env`) sesuai dengan target pengujian Anda:

Skema Jaringan	Kasus Penggunaan	Konfigurasi VITE_API_URL	Konfigurasi VITE_APP_VERSION	Contoh Nilai
1. Ngrok Tunnel	Pengujian jarak jauh via internet (HP client via paket data, Server di WSL laptop).	<code>https://[SUBDOMAIN].ngrok-free.dev/api</code>	Harus sama dengan version Name di Gradle.	<code>https://police-sacred-pound.ngrok-free.dev/api</code>
2. Wi-Fi Lokal (Non-Ngrok)	Pengujian cepat tanpa internet. HP client dan laptop server harus terhubung ke satu router/Wi-Fi yang sama.	<code>https://lims.local:8082/api</code> (Lewat Nginx HTTPS Load Balancer dengan pemetaan IP)	Harus sama dengan version Name di Gradle.	<code>https://lims.local:8082/api</code>
3. Domain Resmi & IP Publik	Deployment rilis produksi final untuk pengguna nyata dengan IP	<code>https://[DOMAIN_RESMI_PERUSAHAAN]/api</code>	Harus sama dengan version Name di Gradle.	<code>https://lims.perusahaan.com/api</code>

Skema Jaringan	Kasus Penggunaan	Konfigurasi VITE_API_URL	Konfigurasi VITE_APP_VERSION	Contoh Nilai
	Publik / Domain Resmi.			

Penjelasan Penting Konfigurasi Berkas .env.production:

1. Untuk Deployment Web Server (lims1 & lims2):

- o Aset web untuk diakses via browser sebaiknya dikompilasi dengan VITE_API_URL=/api. Hal ini karena web disajikan dari domain yang sama, dan Nginx bertindak sebagai reverse proxy yang meneruskan path /api ke cluster backend secara otomatis.

2. Untuk Build Aplikasi Mobile (APK):

- o Aplikasi mobile membutuhkan **alamat absolut** (bukan relatif /api) karena webview berjalan secara internal di dalam HP (https://localhost). Jika menggunakan /api, aplikasi mobile akan crash dengan error Unexpected token '<'.

3. Skenario Wi-Fi Lokal (Mendukung Multiple Server / Load Balancer Nginx):

- o Hubungkan laptop server dan HP Android ke satu router Wi-Fi yang sama.
- o Cari IP lokal laptop server Anda (misal 192.168.0.103 - dapat dilihat via perintah ipconfig di Windows Command Prompt).

Metode A: Lewat HTTPS Port 8082 (Membutuhkan Instalasi Sertifikat SSL di HP)

- o **Pemetaan Host Lokal (lims.local):** Petakan nama domain lims.local ke IP lokal laptop server Anda (192.168.0.103) di jaringan lokal atau file hosts perangkat.
- o Atur VITE_MOBILE_API_URL=https://lims.local:8082/api pada berkas .env.production sebelum melakukan build APK.
- o *Catatan:* Karena menggunakan SSL HTTPS lokal, pastikan sertifikat SSL untuk lims.local (atau root CA pembuatnya) telah di-install dan dipercaya (*trusted*) pada HP Android Anda agar WebView tidak memblokir koneksi akibat error SSL self-signed.

Metode B: Lewat HTTP Port 8087 (Paling Mudah, Tanpa Internet, Bebas Kendala SSL)

- o Karena kita telah mengaktifkan **android:usesCleartextTraffic="true"** pada berkas AndroidManifest.xml aplikasi, HP Android diizinkan melakukan request HTTP biasa tanpa enkripsi (bebas dari pemblokiran SSL self-signed).
- o **Tambahkan Blok Server HTTP Baru di Nginx (/etc/nginx/conf.d/lims.conf):** Tambahkan server block ini di sebelah blok port 8082 Anda:
 - o *# --- COPY-PASTE KE CONFIG NGINX (UNTUK WI-FI LOKAL LOAD BALANCER HTTP) ---*
 - o server {
 - o listen 8087;

- `server_name localhost 192.168.0.103 212.85.24.33; # Ganti dengan IP laptop (Wi-Fi) atau IP Publik VPS Anda`
-
- `client_max_body_size 50M;`
-
- `# Proxy API ke Go Backend Cluster (Load Balancer ke Port 8081 & 8091)`
- `location /api {`
- `proxy_pass http://lims_backend_cluster;`
- `proxy_http_version 1.1;`
- `proxy_set_header Host $host;`
- `proxy_set_header X-Real-IP $remote_addr;`
- `proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;`
- `proxy_set_header X-Forwarded-Proto $scheme;`
- `}`
-
- `# Sajikan uploads langsung dari shared folder`
- `location /uploads/ {`
- `alias /home/lims/shared_uploads/;`
- `expires 30d;`
- `}`
-
- `# Sajikan unduhan APK langsung dari shared folder`
- `location /downloads/ {`
- `alias /home/lims/shared_downloads/;`
- `expires 30d;`
- `}`
- `}`
- `# -----`

Reload Nginx setelah disimpan: `sudo nginx -t && sudo systemctl reload nginx`

- Atur `VITE_MOBILE_API_URL=http://192.168.0.103:8087/api` pada berkas `.env.production` laptop Anda sebelum melakukan build APK.
- Dengan cara ini, aplikasi mobile di HP Anda dapat mengakses API melalui HTTP biasa di port 8087 (bebas error SSL), dan Nginx tetap membagi bebannya secara merata ke backend port 8081 & 8091.

4. Skenario Domain dengan IP Publik (Produksi Asli):

- Petakan IP Publik server Anda ke domain resmi (misal `lims.perusahaan.com`) di DNS panel.
- Atur `VITE_MOBILE_API_URL=https://lims.perusahaan.com/api` (jika Nginx menggunakan port HTTPS standar 443) atau `https://lims.perusahaan.com:8082/api` (jika menggunakan port HTTPS custom 8082).

[!CAUTION] Jangan menggunakan URL relatif `/api` pada build APK Android. Hal ini akan menyebabkan error Unexpected token '<' karena WebView akan memanggil `https://localhost/api` yang mengembalikan berkas HTML statis.

Alur Kompilasi Aset Web (Vite & Capacitor Sync)

Setiap kali Anda mengubah konfigurasi .env atau berkas source code frontend, Anda **WAJIB** menyelaraskan aset web ke dalam proyek native Android sebelum melakukan kompilasi APK.

Jalankan perintah berikut di terminal komputer Windows Anda:

Pindah ke direktori frontend

cd D:\Data_NK\Project5\AI\LIM_System_Linux_OK\frontend

Jalankan kompilasi React dan salin otomatis ke folder assets android

npm run cap:sync

(Perintah di atas akan menjalankan npm run build -> npx cap copy -> npx cap sync secara otomatis).

Panduan Pembuatan APK Android

Masuk ke folder proyek native Android Anda:

cd D:\Data_NK\Project5\AI\LIM_System_Linux_OK\frontend\android

1. Membuat APK Versi Debug (Development / Pengujian Cepat)

APK Debug digunakan untuk pengembangan harian dan proses instalasi cepat tanpa perlu verifikasi tanda tangan digital rilis resmi.

- **Via Terminal CLI:**
- # Langsung jalankan build (cepat karena menggunakan cache)
- ./gradlew assembleDebug

(Catatan: Jika Anda baru saja menambahkan plugin baru atau mengalami error cache, jalankan ./gradlew clean terlebih dahulu sebelum ./gradlew assembleDebug). File output berada di: frontend/android/app/build/outputs/apk/debug/app-debug.apk

- **Via Android Studio (Rekomendasi untuk Live Debugging):**
 1. Buka folder frontend/android menggunakan Android Studio.
 2. Sambungkan HP Android Anda via kabel USB (aktifkan USB Debugging di HP).
 3. Klik tombol **Run 'app'** (ikon Play hijau) atau jalankan npx cap run android di terminal. Aplikasi akan otomatis terkompilasi, terpasang, dan langsung berjalan di HP Anda.

2. Membuat APK Versi Release (Produksi / Siap Distribusi)

APK Release digunakan untuk pengetesan resmi, distribusi internal, atau unggahan ke toko aplikasi. Versi ini sudah ditandatangani secara resmi menggunakan kunci rilis LIMS.

- **Perintah Build:**
- ./gradlew clean
- ./gradlew assembleRelease

File output berada di: frontend/android/app/build/outputs/apk/release/app-release.apk

(Berkas ini ditandatangani menggunakan berkas lims-release-key.jks dengan password Nkl@130200 sesuai dengan konfigurasi signingConfigs di build.gradle).

[!IMPORTANT] **Penting untuk Versi Rilis:**

- **Perbedaan Berkas build.gradle (Penting Agar Tidak Salah Edit):** Di proyek Android, terdapat **dua** berkas build.gradle dengan tujuan berbeda. Pastikan Anda membuka berkas **Modul / App-Level**:
 - **→ Modul / App-Level (Edit yang ini):** [frontend/android/app/build.gradle](file:///d:/Data_NK/Project5/AI/L

IM_System_Linux_OK/frontend/android/app/build.gradle) (menyimpan konfigurasi aplikasi seperti versionCode, versionName, signingConfigs, dan dependencies).

- → **Root / Project-Level (Jangan edit yang ini):** [frontend/android/build.gradle](file:///d:/Data_NK/Project5/AI/LIM_System_Linux_OK/frontend/android/build.gradle) (menyimpan repositori buildscript global).

1. Buka berkas [build.gradle App-level](file:///d:/Data_NK/Project5/AI/LIM_System_Linux_OK/frontend/android/app/build.gradle), temukan blok defaultConfig, lalu naikkan versionCode (angka bulat naik terus) dan sesuaikan versionName dengan versi frontend:
2. android {
3. defaultConfig {
4. applicationId "com.lims.app"
5. versionCode 4 // <-- *UBAH INI (Angka bulat naik terus, misal 2 -> 3 -> 4)*
6. versionName "1.3" // <-- *UBAH INI (Harus sama dengan VITE_APP_VERSION di .env.production)*
7. }
8. }
9. Pastikan Anda melakukan **Uninstall total** aplikasi LIMS lama yang terpasang di HP Anda sebelum memasang APK rilis baru ini untuk menghindari error *App not installed* akibat bentrok tanda tangan digital debug/release.

Konsep In-App Auto-Update (Self-Update) Tanpa Google Play Store

Karena aplikasi APK didistribusikan secara mandiri di luar Google Play Store, proses instalasi pembaruan otomatis di latar belakang secara diam-diam (*silent background update*) tidak diizinkan oleh sistem keamanan Android (kecuali perangkat di-root).

Namun, LIMS dapat diintegrasikan dengan fitur **In-App Auto-Update (Self-**

Update) dengan alur sebagai berikut:

1. **Deteksi Pembaruan:** Aplikasi mendeteksi respon "status": "FORCE_UPGRADE" melalui API /api/check-version saat startup.
2. **Download File APK:** Aplikasi menggunakan plugin Capacitor (seperti @capacitor-community/http or @capgo/capacitor-updater) untuk mengunduh berkas APK baru langsung dari server (ANDROID_DOWNLOAD_URL) ke penyimpanan lokal cache perangkat Android.
3. **Memicu Proses Instalasi (Package Installer Intent):** Setelah pengunduhan selesai, aplikasi memanggil fungsi native Android menggunakan **FileProvider** untuk meluncurkan antarmuka sistem penginstalan aplikasi:
4. Intent intent = new Intent(Intent.ACTION_VIEW);
5. intent.setDataAndType(apkUri, "application/vnd.android.package-archive");
6. intent.setFlags(Intent.FLAG_ACTIVITY_NEW_TASK | Intent.FLAG_GRANT_READ_URI_PERMISSION);
7. context.startActivity(intent);

8. **Interaksi Pengguna:** Layar sistem Android akan menampilkan dialog konfirmasi kepada pengguna: *"Apakah Anda ingin memasang pembaruan untuk aplikasi yang sudah ada ini?"*. Pengguna cukup mengetuk **"Update / Perbarui"** dan sistem akan menyelesaikan instalasinya secara otomatis dan membuka kembali aplikasi LIMS yang baru.

Troubleshooting Koneksi Jaringan Wi-Fi Lokal & Windows Firewall

Ketika menguji aplikasi LIMS di HP Android Anda menggunakan skema **Wi-Fi Lokal Tanpa Internet (Metode B - Port 8087)** atau **Ngrok (Internet)**, lakukan pengecekan berikut untuk menyelesaikan error Failed to fetch atau kendala konektivitas:

1. Fitur Pengaturan Server API Dinamis (Bypass Rebuild APK)

Mulai versi 1.4, Anda tidak perlu lagi melakukan kompilasi ulang APK jika IP laptop server Anda berubah. Kami telah menambahkan tombol roda gigi (**Gear Icon**) di halaman Login aplikasi mobile LIMS:

- **Cara Menggunakan:**

1. Buka aplikasi LIMS di HP Anda.
2. Ketuk ikon roda gigi (**Gear**) di pojok kanan atas kartu Login.
3. Pilih mode **Wi-Fi Lokal** jika ingin terhubung ke laptop via router lokal, lalu masukkan **IP Laptop Server** aktif Anda (misal 192.168.1.50). Port HTTP 8087 akan otomatis digunakan.
4. Pilih mode **Internet (Ngrok)** jika ingin mengakses dari luar menggunakan data seluler, lalu masukkan **URL Ngrok** aktif Anda (misal <https://xxxx.ngrok-free.app>).
5. Ketuk tombol **Tes Koneksi** untuk memverifikasi. Jika berhasil terhubung, indikator status akan berubah menjadi hijau **"Terhubung"**.
6. Ketuk **Simpan & Terapkan** untuk menyimpan konfigurasi dan memicu pemuatan ulang (*reload*) aplikasi.

2. Masalah Perubahan IP Laptop Dinamis (DHCP Router)

Router Wi-Fi sering memberikan IP baru secara dinamis ke laptop Anda ketika terhubung kembali. Jika koneksi HP tiba-tiba terputus:

- **Solusi:**

1. Periksa IP laptop aktif saat ini (buka Command Prompt di Windows, jalankan ipconfig, cari bagian *IPv4 Address* di bawah adapter Wi-Fi aktif, misal 192.168.1.50).
2. Buka menu roda gigi di HP Anda, ganti IP lama dengan IP aktif saat ini, ketuk **Tes Koneksi**, lalu **Simpan & Terapkan**. Anda tidak perlu menginstal ulang APK.

3. Masalah Pengalihan Paket Data Seluler (Smart Network Switch)

Pada HP Android modern, sistem akan secara otomatis mengalihkan koneksi ke paket data seluler jika mendeteksi bahwa Wi-Fi lokal yang terhubung **tidak memiliki akses internet** (karena Anda mematikan koneksi internet eksternal pada router tersebut).

- **Solusi:** Matikan **Paket Data Seluler (Mobile Data)** di HP Anda secara total selama pengujian offline, dan biarkan HP Anda hanya terhubung ke Wi-Fi lokal laptop server Anda.

4. Windows Defender Firewall Memblokir Koneksi Masuk

Sistem operasi Windows pada laptop Anda secara default memblokir semua request masuk dari perangkat lain di jaringan lokal ke port custom Nginx (seperti port 8082 untuk HTTPS atau port 8087 untuk HTTP).

- **Solusi:** Jalankan **PowerShell sebagai Administrator** pada komputer Windows Anda, kemudian salin dan jalankan perintah berikut untuk membuka port Nginx di Windows Firewall:
- *# Membuka port HTTPS 8082*
- `New-NetFirewallRule -DisplayName "LIMS Nginx HTTPS" -Direction Inbound -LocalPort 8082 -Protocol TCP -Action Action`
-
- *# Membuka port HTTP 8087 (Wi-Fi Mobile)*
- `New-NetFirewallRule -DisplayName "LIMS Nginx HTTP Mobile" -Direction Inbound -LocalPort 8087 -Protocol TCP -Action Action`

5. Pemblokiran Mixed Content oleh Android WebView

Capacitor menyajikan file aplikasi di dalam HP menggunakan skema HTTPS (<https://localhost>). Jika aplikasi menembak URL HTTP biasa (<http://192.168.1.50:8087/api>), WebView Android akan memblokirnya karena dianggap tidak aman (*Mixed Content*).

- **Solusi:** Pastikan parameter "androidScheme" pada berkas capacitor.config.json diatur ke "**http**":
- `"server": {`
- `"androidScheme": "http",`
- `"cleartext": true`
- `}`

(Dengan skema [http](http://localhost), aplikasi dijalankan pada <http://localhost> sehingga WebView mengizinkan request HTTP ke laptop port 8087 tanpa pemblokiran).

6. Uji Coba Diagnostik Awal Melalui Browser HP

Sebelum menjalankan aplikasi LIMS, pastikan HP Anda secara fisik bisa menghubungi laptop server:

- Buka browser (Google Chrome / Safari) di HP Anda, lalu buka URL: `http://<IP_LAPTOP_SERVER>:8087/api/check-version?version=1.4&platform=android` (misal `http://192.168.1.50:8087/api/check-version?version=1.4&platform=android`)
- Jika browser menampilkan teks data JSON seperti `{"status":"OK"}`, artinya jalur koneksi Wi-Fi lokal dan firewall Anda sudah benar. Jika loading terus-menerus atau error (ERR_ADDRESS_UNREACHABLE), router, segmen subnet, atau firewall laptop Anda masih bermasalah.

F. Cara Remote Debugging (Tracing) WebView HP via Chrome DevTools

Jika aplikasi LIMS di HP Anda mengalami error Failed to fetch tetapi Anda tidak tahu apa penyebab pastinya, Anda dapat melacak (*tracing*) log console dan network request yang dikirimkan oleh WebView HP secara langsung dari laptop Anda:

1. **Gunakan APK Versi Debug (app-debug.apk):**
 - Android mematikan fitur inspeksi WebView secara default pada APK Release demi alasan keamanan.
 - Pastikan Anda menginstal berkas debug APK yang terletak di: `frontend/android/app/build/outputs/apk/debug/app-debug.apk`

2. **Aktifkan USB Debugging di HP Android Anda:**
 - Masuk ke **Pengaturan HP → Tentang Ponsel → Informasi Perangkat Lunak**.
 - Ketuk **Nomor Versi (Build Number)** sebanyak 7 kali hingga muncul pesan "Mode pengembang aktif".
 - Buka menu baru **Pilihan Pengembang (Developer Options)** di Pengaturan, dan aktifkan **USB Debugging (Mendebug USB)**.
3. **Hubungkan HP ke Laptop Menggunakan Kabel USB Data:**
 - Pastikan kabel USB yang Anda gunakan mendukung transfer file (bukan hanya kabel charger).
 - Pada HP Anda, ubah mode koneksi USB menjadi **MTP / File Transfer**.
 - Izinkan pop-up konfirmasi "*Izinkan mendebug USB dari komputer ini?*" yang muncul di layar HP.
4. **Buka Inspektur Chrome di Laptop:**
 - Buka browser Google Chrome di laptop Anda, lalu masuk ke alamat: `chrome://inspect`
 - Pastikan aplikasi LIMS sudah dibuka di HP Anda.
 - Nama HP Anda dan tulisan **LIM System (com.lims.app)** akan muncul di bawah bagian **Remote Target / Devices**.
 - Klik tombol **inspect** di bawah nama aplikasi tersebut. Jendela Developer Tools laptop akan terbuka untuk memantau log error konsol dan jaringan HP secara real-time.

Q. Migrasi Konfigurasi dari Lokal (Laptop) ke VPS

Bagian ini memuat panduan teknis konfigurasi lengkap untuk memindahkan sistem LIMS dari berjalan lokal di laptop (127.0.0.1) ke server produksi VPS (212.85.24.33).

Panduan Metode Transfer Berkas dari Lokal (Laptop) ke VPS

Untuk memindahkan kode, hasil kompilasi (build), atau folder konfigurasi dari komputer lokal Anda ke VPS, gunakan salah satu dari tiga metode berikut:

Ops A: Menggunakan Perintah scp (Melalui Terminal / WSL) - Direkomendasikan untuk CLI

Perintah scp bawaan terminal Linux/WSL sangat cocok untuk mentransfer berkas biner tunggal, file .env, atau folder aset secara instan.

- **Mengirim Biner Backend main ke VPS:**
- `scp /mnt/d/Data_NK/Project5/AI/LIM_System_Linux_OK/backend/main lims@212.85.24.33:/home/lims/lims1/backend/`
- `scp /mnt/d/Data_NK/Project5/AI/LIM_System_Linux_OK/backend/main lims@212.85.24.33:/home/lims/lims2/backend/`
- **Mengirim Folder dist/ Frontend ke VPS:**
- `# Kirim folder dist ke instansi lims1`
- `scp -r /mnt/d/Data_NK/Project5/AI/LIM_System_Linux_OK/frontend/dist lims@212.85.24.33:/home/lims/lims1/frontend/`
- `# Kirim folder dist ke instansi lims2`
- `scp -r /mnt/d/Data_NK/Project5/AI/LIM_System_Linux_OK/frontend/dist lims@212.85.24.33:/home/lims/lims2/frontend/`

- **Mengirim Berkas Konfigurasi .env:**
- `scp /mnt/d/Data_NK/Project5/AI/LIM_System_Linux_OK/backend/.env
lims@212.85.24.33:/home/lims/lims1/backend/`

Opsi B: Menggunakan FileZilla atau WinSCP (Antarmuka Grafis / GUI) - Paling Mudah

Jika Anda lebih menyukai metode seret-dan-lepas (*drag-and-drop*) file secara visual:

1. Unduh dan buka aplikasi **FileZilla** atau **WinSCP** di laptop Anda.
2. Buat koneksi baru dengan pengaturan berikut:
 - **Protokol:** SFTP (SSH File Transfer Protocol)
 - **Host (IP):** 212.85.24.33
 - **Port:** 22
 - **Logon Type:** Normal
 - **User:** lims
 - **Password:** *Masukkan password SSH VPS untuk user lims.*
3. Klik **Connect**. Anda dapat langsung mengunggah file/folder dari panel kiri (Lokal Laptop) ke panel kanan (Direktori /home/lims/... di VPS) secara visual.

Opsi C: Menggunakan Perintah rsync (Paling Cepat untuk Perubahan Berulang)

rsync hanya mengirimkan bagian berkas yang mengalami perubahan (delta sync), sehingga sangat hemat bandwidth dan cepat.

Mengirim backend dengan mengecualikan node_modules, logs, dan folder .git

`rsync -avz -e ssh --exclude 'node_modules' --exclude '.git' --exclude 'logs'`

`/mnt/d/Data_NK/Project5/AI/LIM_System_Linux_OK/backend/`

`lims@212.85.24.33:/home/lims/lims1/backend/`

1. Templat Lengkap File .env Go Backend di VPS

(/home/lims/lims1/backend/.env dan /home/lims/lims2/backend/.env)

Salin konfigurasi berikut untuk berkas .env di VPS Anda. Sesuaikan port (PORT=8081 untuk instance 1, dan PORT=8091 untuk instance 2).

```
# =====
# LIMS Production Environment Configuration
# =====
```

1. Aplikasi & Domain

`SERVER_DOMAIN=212.85.24.33`

`PORT=8081`

`GIN_MODE=release`

2. Database Server (Menggunakan IP local loopback VPS karena DB di server yang sama)

`DB_HOST=127.0.0.1`

`DB_PORT=5432`

`DB_NAME=lims_prod_db`

`DB_USER=lims_app`

Gunakan utility generate_db_password.go untuk mengisi nilai ini

`DB_PASSWORD_ENCRYPTED=Isi_Nilai_Hasil_Enkripsi_AES_Di_Sini`

`DB_SCHEMA=lims`

DB_SSLMODE=disable

3. Kunci Pengaman Token (Gunakan openssl rand -hex 32 untuk membuat token acak)
JWT_SECRET=Ganti_Dengan_JWT_SECRET_Heksadesimal_Sangat_Rahasia_Anda
JWT_EXPIRY_HOURS=24

4. Object Storage (MinIO)
MINIO_ENDPOINT=127.0.0.1:9000
MINIO_ACCESS_KEY=adminmilitar
Kunci rahasia MinIO yang sudah dienkripsi
MINIO_SECRET_KEY_ENCRYPTED=KZCFAMAtbDEiXlnaTCUCXcTdFykCe3ybMSQAsCA=
MINIO_BUCKET=lims-docs
MINIO_USE_SSL=false
MINIO_ENV=production

5. Workflow Engine (Camunda BPMN)
CAMUNDA_URL=http://127.0.0.1:8085/engine-rest
CAMUNDA_USER=demo
Sandi Camunda yang terenkripsi
CAMUNDA_PASSWORD_ENCRYPTED=_4HLoRpb4_qnqBm2VlX7wLg5nrg=
CAMUNDA_ENV=production

6. Keamanan & Reverse Proxy
Menerima IP client asli dari Nginx proxy lokal
TRUSTED_PROXIES=127.0.0.1
Pengaturan CORS (ALLOWED_ORIGINS - pisahkan dengan koma, tanpa spasi)
ALLOWED_ORIGINS=http://212.85.24.33,http://212.85.24.33:8081,http://212.85.24.33:8092

7. Pengaturan Tambahan
TRACE_LEVEL=0
DB_DEBUG=false
DB_AUTO_MIGRATE=false
SEED_DB=false
MAX_UPLOAD_SIZE=4096
COMPANY_NAME="LIM System"
HEADER_TITLE="Laboratory Information Management System"
APP_FOOTER="Laboratory Information Management System - Dokumen Rahasia & Terbatas"
APP_ADMIN_NAME="LIMS Administrator"

⚠ Mengamankan JWT_SECRET dan Enkripsi Sandi Database (VPS)

1. **Membuat Kunci JWT_SECRET Acak:** Gunakan terminal VPS atau lokal laptop Anda untuk menghasilkan kunci rahasia 256-bit acak yang aman.
 - o **Ops 1: Menggunakan OpenSSL (Bawaan VPS Linux):**
 - o openssl rand -hex 32

- **Opsi 2: Menggunakan Python 3:**

- `python3 -c "import secrets; print(secrets.token_hex(32))"`

Salin hasilnya dan jadikan nilai variabel `JWT_SECRET=hasil_kunci_acak_tadi` di file `.env` VPS.

2. **Mendapatkan Nilai DB_PASSWORD_ENCRYPTED:** Pastikan `.env` sudah memuat `JWT_SECRET` terbaru, lalu jalankan utilitas pembuat sandi di direktori backend/:

3. `go run generate_db_password.go "Password_Database_Lims_App_Asl"`

Salin string keluaran yang dihasilkan ke dalam variabel `DB_PASSWORD_ENCRYPTED` di file `.env`.

[!WARNING] **PERBEDAAN PENTING: Sandi .env vs Sandi Pengguna (Tabel Users)**

- **Sandi Database di .env (DB_PASSWORD_ENCRYPTED)** menggunakan enkripsi **AES-CFB (simetris)** yang didekripsi menggunakan `JWT_SECRET`. Berkas `generate_db_password.go` dibuat **hanya** untuk mengenkripsi sandi database ini.
- **Sandi Pengguna (Users) di Database** (tabel `lims.users` kolom `password`) menggunakan hash **Bcrypt (satu arah)** yang dibuat dengan library `golang.org/x/crypto/bcrypt`.
- **Jangan memasukkan sandi hasil enkripsi AES ke tabel users.** Jika Anda memasukkannya, Go backend tidak akan bisa mendekripsinya dan akan menolak login Anda dengan error `Invalid username or password`.
- **Cara Membuat Hash Bcrypt Sendiri:** Anda dapat menggunakan script utilitas `generate_bcrypt_hash.go` yang sudah disediakan di folder `backend/`:
 - `# Masuk ke folder backend`
 - `cd backend/`
 - `# Jalankan utilitas pembuat hash Bcrypt (Ganti "password_baru" dengan sandi Anda)`
 - `go run generate_bcrypt_hash.go "password_baru"`

Salin hash keluaran (misal: `$2a$10$...`) dan gunakan untuk memperbarui password di database.

- Jika Anda ingin mengubah password user nur langsung lewat SQL menjadi **Nkl@130200**, gunakan hash **Bcrypt** berikut:
- `UPDATE lims.users SET password = '$2a$10$814uSRqX1VfqwNnEYZAYTui4O5A7STRErwy3LIeJSa8gaKYmuOaQi' WHERE username = 'nur';`

Panduan & Solusi Error ONNX Runtime (libonnxruntime.so) Tidak Ditemukan di VPS

Jika saat menjalankan backend LIMS di VPS (`lims-backend-8081` atau `lims-backend-8091`), layanan langsung mati (*crash*) dan logs `journalctl` menampilkan pesan kesalahan seperti berikut: `Failed to initialize ONNX Runtime: Platform-specific initialization failed: Error loading ONNX shared library "lib/libonnxruntime.so": lib/libonnxruntime.so: cannot open shared object file: No such file or directory`

Hal ini disebabkan karena folder **lib/** yang berisi pustaka AI (`libonnxruntime.so`) belum disalin dari laptop lokal Anda ke direktori backend di VPS.

Langkah Pemecahan Masalah:

1. Buka terminal **Ubuntu Lokal (WSL/Laptop)** Anda, lalu masuk ke direktori backend/.
2. Jalankan perintah scp ini untuk mentransfer folder lib/ ke direktori instansi lims1 dan lims2 di VPS:
3. *# Mengirim ke instansi lims1 (Port 8081)*
4. `scp -r lib/ lims@212.85.24.33:/home/lims/lims1/backend/`
- 5.
6. *# Mengirim ke instansi lims2 (Port 8091)*
7. `scp -r lib/ lims@212.85.24.33:/home/lims/lims2/backend/`
8. Masuk ke terminal **VPS Anda**, lalu jalankan ulang layanan backend LIMS untuk memuat pustaka tersebut:
9. `sudo systemctl daemon-reload`
10. `sudo systemctl restart lims-backend-8081.service`
11. `sudo systemctl restart lims-backend-8091.service`

2. Templat Lengkap File .env dan .env.production React Frontend di VPS

Frontend React menggunakan Vite untuk proses kompilasi. Berikut adalah konfigurasi berkas lingkungan untuk server VPS Anda:

A. Berkas .env (Untuk Konfigurasi Menjalankan PM2 / Dev Server di VPS)

```
# =====  
# LIMS Frontend Environment (PM2/Dev Server)  
# =====  
VITE_SERVER_DOMAIN=212.85.24.33  
VITE_API_URL=/api  
VITE_PORT=3000  
VITE_SHOW_DEBUG_INFO=false  
VITE_ENABLE_TERMINAL_LOG=false  
VITE_APP_VERSION=1.4
```

B. Berkas .env.production (Untuk Kompilasi Rilis / npm run build di VPS)

Berkas ini dibaca secara otomatis oleh Vite saat Anda menjalankan perintah kompilasi rilis produksi (npm run build).

```
# =====  
# LIMS Frontend Production Build Environment  
# =====  
# Mengarahkan pemicu API browser secara relatif (mengikuti asal origin browser secara dinamis)  
VITE_API_URL=/api  
  
# Digunakan khusus oleh Aplikasi Mobile Android APK (Tembak langsung ke API Nginx VPS)  
VITE_MOBILE_API_URL=https://lims-d4551821.nip.io:8082/api  
  
# Versi aplikasi (Wajib disamakan dengan versionName di build.gradle)  
VITE_APP_VERSION=1.4
```

Catatan Penting:

- **VITE_API_URL=/api** menggunakan path relatif. Karena diakses via browser, kode router API LIMS (frontend/src/models/api.js) akan otomatis menyesuaikan alamat asalnya (window.location.origin menjadi http://212.85.24.33/api) sehingga tidak terbentur masalah CORS.
- **VITE_MOBILE_API_URL**: Di lingkungan VPS produksi, ubah nilai ini dari sebelumnya menembak IP lokal pengembang/laptop (http://192.168.0.103:8087/api) menjadi menembak domain publik VPS HTTPS Anda (**https://lims-d4551821.nip.io:8082/api**) agar aplikasi Android LIMS APK yang diinstal di HP klien dapat berkomunikasi secara aman langsung ke server VPS Anda di internet.

Panduan & Solusi Error EPERM Saat Menjalankan npm run build di WSL

Jika Anda mengompilasi frontend di dalam lingkungan WSL (Ubuntu) dan menemui kendala perizinan berkas seperti: EPERM: operation not permitted, copyfile ... Hal ini disebabkan oleh bentrokan penguncian berkas (*file locking*) oleh layanan Windows Defender / Explorer saat WSL mencoba menulis langsung ke partisi Windows (/mnt/d/). Berikut adalah 2 opsi solusi yang dapat Anda gunakan untuk mengatasinya:

Opsi A: Menggunakan Windows PowerShell / Command Prompt (Sangat Direkomendasikan)

Kompilasi dilakukan langsung menggunakan Node.js Windows pada drive fisik Windows untuk menghindari lapisan translasi file system WSL:

1. Buka **PowerShell** atau **CMD** di Windows Anda.
2. Masuk ke direktori frontend:
3. `cd D:\Data_NK\Project5\AI\LIM_System_Linux_OK\frontend`
4. Jalankan perintah build:
5. `npm run build`

Opsi B: Melakukan Kompilasi di File System Native Linux (WSL /tmp/)

Jika Anda harus tetap menggunakan terminal WSL, salin folder proyek ke folder native Linux untuk dikompilasi, kemudian salin folder dist hasilnya kembali ke Windows:

1. Salin folder frontend ke direktori native Linux

```
cp -r /mnt/d/Data_NK/Project5/AI/LIM_System_Linux_OK/frontend /tmp/frontend-build
```

2. Jalankan build di folder native Linux (bebas dari error EPERM)

```
cd /tmp/frontend-build
```

```
npm run build
```

3. Pindahkan folder dist hasil build kembali ke folder proyek Windows Anda

```
rm -rf /mnt/d/Data_NK/Project5/AI/LIM_System_Linux_OK/frontend/dist
```

```
cp -r dist /mnt/d/Data_NK/Project5/AI/LIM_System_Linux_OK/frontend/
```

4. Hapus folder temporary

```
rm -rf /tmp/frontend-build
```

3. Panduan Langkah demi Langkah Konfigurasi NGINX di VPS

Bagian ini menjelaskan bagaimana berkas konfigurasi utama Nginx berkomunikasi dengan berkas konfigurasi situs LIMS, serta langkah-langkah detail implementasinya di VPS Anda.

☞ Konsep & Hubungan Komunikasi Berkas Nginx:

1. **/etc/nginx/nginx.conf** adalah berkas konfigurasi **induk/utama** yang dibaca pertama kali oleh Nginx saat startup.
2. Di dalam blok `http { ... }` pada `nginx.conf` bawaan Nginx resmi, terdapat perintah `include /etc/nginx/conf.d/*.conf;`.
3. Perintah `include` tersebut menginstruksikan Nginx untuk otomatis **membaca dan menggabungkan** seluruh berkas konfigurasi yang berakhiran `.conf` di dalam direktori `/etc/nginx/conf.d/`.
4. **Solusi VPS Anda:** Karena Nginx di VPS Anda tidak memiliki folder `sites-available` / `sites-enabled`, Anda **tidak perlu** membuat folder tersebut atau menggunakan tautan simbolis (*symlink*). Anda cukup membuat satu berkas bernama **lims.conf** langsung di dalam folder **/etc/nginx/conf.d/**. Berkas ini akan otomatis dimuat oleh `nginx.conf`.

📁 Langkah 1: Modifikasi Berkas Utama `/etc/nginx/nginx.conf`

Anda perlu mendaftarkan format log monitoring (`upstream_monitoring`) dan pemetaan IP asli agar Nginx dapat meneruskannya ke backend Go LIMS.

1. Buka berkas konfigurasi utama menggunakan editor teks nano:
2. `sudo nano /etc/nginx/nginx.conf`
3. Cari blok **http { ... }** di dalam berkas tersebut. Tempelkan (*paste*) baris kode berikut di bagian dalam blok `http {` (taruh tepat di atas baris `include /etc/nginx/conf.d/*.conf;`):
4. *# Pemetaan IP Client Asli jika melewati proxy*
5. `map $http_x_forwarded_for $real_client_ip {`
6. `"" $remote_addr;`
7. `default $http_x_forwarded_for;`
8. `}`
- 9.
10. *# Deklarasi format log untuk monitoring LIMS & GoAccess*
11. `log_format upstream_monitoring '$remote_addr - ClientIP: $real_client_ip -`
`[$time_local] '`
12. `"$request" $status $body_bytes_sent '`
13. `'to_server=$upstream_addr status=$upstream_status '`
14. `'resp_time=$upstream_response_time '`
15. `'agent="$http_user_agent" '`
16. `'app_ver="$http_x_app_version"`
`app_plat="$http_x_app_platform";`
17. Simpan berkas (Ctrl + O, lalu Enter untuk menyimpan, dan Ctrl + X untuk keluar).

📁 Langkah 2: Buat Berkas Konfigurasi Situs `/etc/nginx/conf.d/lims.conf`

Buat berkas konfigurasi reverse proxy khusus untuk situs LIMS agar permintaan eksternal ke IP VPS diteruskan ke backend Go dan PM2.

1. Jalankan perintah berikut untuk membuat berkas konfigurasi LIMS yang baru:
2. `sudo nano /etc/nginx/conf.d/lims.conf`
3. Salin seluruh kode blok konfigurasi di bawah ini dan tempelkan (*paste*) ke dalam berkas tersebut:

```
#
=====
=====
# FILE CONFIGURATION: LIMS NGINX PROXY CONFIG
#
=====
=====

# --- CLUSTER LOAD BALANCER BACKEND ---
# Mendistribusikan request API ke dua instance Go backend
upstream lims_backend_cluster {
    server 127.0.0.1:8081 max_fails=3 fail_timeout=10s;
    server 127.0.0.1:8091 max_fails=3 fail_timeout=10s;
}

# --- CLUSTER LOAD BALANCER FRONTEND ---
# Mendistribusikan request Frontend ke dua instance PM2 Node.js
upstream lims_frontend_cluster {
    server 127.0.0.1:3000 max_fails=3 fail_timeout=10s;
    server 127.0.0.1:3001 max_fails=3 fail_timeout=10s;
}

#
=====
=====
# BLOK SERVER: HTTP PORT 80 (Akses Publik Eksternal VPS)
#
=====
=====
server {
    listen 80;
    server_name 212.85.24.33; # Ganti dengan Domain jika ada

    # Log Akses Khusus untuk Monitoring Upstream & Client IP
    access_log /var/log/nginx/lims_access.log;
    error_log /var/log/nginx/lims_error.log;

    # Batas Maksimal Upload Lampiran Dokumen LIMS
    client_max_body_size 20M;

    # --- 1. PROXY FRONTEND REACT (PM2) ---
    location / {
```

```

proxy_pass http://lims_frontend_cluster;
proxy_http_version 1.1;
proxy_set_header Upgrade $http_upgrade;
proxy_set_header Connection 'upgrade';
proxy_set_header Host $host;
proxy_cache_bypass $http_upgrade;

# Meneruskan Client IP asli ke PM2
proxy_set_header X-Real-IP $remote_addr;
proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
}

# --- 2. PROXY BACKEND API GO ---
location /api/ {
    proxy_pass http://lims_backend_cluster;
    proxy_http_version 1.1;
    proxy_set_header Upgrade $http_upgrade;
    proxy_set_header Connection 'upgrade';
    proxy_set_header Host $host;
    proxy_cache_bypass $http_upgrade;

    # Header Penting untuk Keamanan & IP Deteksi
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto $scheme;
}

# --- 3. SERVING MEDIA / UPLOAD FILES (Shared Storage Link) ---
# File lampiran SPD, Reimbursement, dll dibaca langsung dari folder bersama
location /uploads/ {
    alias /home/lims/shared_uploads/;
    expires 30d;
    add_header Cache-Control "public, no-transform";
    access_log off;
}

# --- 4. SERVING LAPORAN ANALITIK (Shared Reports Link) ---
location /report.html {
    alias /home/lims/shared_reports/report.html;
    expires 5m;
    add_header Cache-Control "public, no-transform";
}
}

Setelah menyalin berkas Nginx di atas, ikuti 3 nomor urut berikut untuk mengaktifkan konfigurasi situs di VPS:

```

1. Nonaktifkan Konfigurasi Bawaan (Default) Nginx

Agar Nginx tidak mengarah ke halaman sambutan default "Welcome to nginx!", nonaktifkan berkas default.conf:

```
sudo mv /etc/nginx/conf.d/default.conf /etc/nginx/conf.d/default.conf.disabled
```

2. Verifikasi Sintaks Nginx

```
sudo nginx -t
```

3. Restart Layanan Nginx

```
sudo systemctl restart nginx
```

4. Cara Mengakses Web Aplikasi LIMS di VPS

Setelah seluruh kluster frontend, backend, database, dan proxy Nginx berhasil diaktifkan di VPS, gunakan rincian berikut untuk mengaksesnya:

- **URL Akses Utama LIMS (Web Browser):** → **<https://lims-d4551821.nip.io:8082>**

[!TIP] **Penggunaan Domain Dinamis & SSL Valid (Status: Terpasang & Berjalan Normal):** Untuk bypass peringatan "Not Secure" pada browser dan HP Android, sistem telah sukses dihubungkan menggunakan domain dinamis gratis **lims-d4551821.nip.io** yang secara otomatis menerjemahkan hex d4551821 menjadi IP VPS Anda (212.85.24.33). Sertifikat SSL tepercaya telah di-generate menggunakan Let's Encrypt (fullchain.pem dan privkey.pem) sehingga koneksi HTTPS LIMS kini terverifikasi gembok hijau di web browser dan aplikasi mobile Android tanpa kendala keamanan.

- **Kredensial Login Utama:**
 - **Username:** nur
 - **Password:** *Gunakan password yang sama dengan akun 'nur' pada database lokal Anda yang telah di-restore ke VPS.*

Jika Anda ingin me-running server development Vite di laptop lokal Anda, namun datanya langsung sinkron menembak ke backend VPS, edit berkas **frontend/.env** di laptop lokal Anda:

```
# Ganti IP target proxy lokal ke alamat backend VPS Anda
```

```
VITE_PROXY_TARGET=http://212.85.24.33:8081
```

🔒 Opsi Tambahan: Menaruh Berkas Sertifikat SSL (fullchain.pem) di VPS

Jika Anda ingin menggunakan berkas sertifikat SSL lokal yang sudah Anda miliki di VPS, berikut adalah langkah penempatan foldernya:

1. Buat Folder /etc/nginx/ssl/ di VPS

Jalankan perintah ini di terminal VPS Anda untuk membuat direktori penyimpanan sertifikat:

```
sudo mkdir -p /etc/nginx/ssl
```

2. Letakkan Berkas Sertifikat di VPS

Salin atau buat berkas di VPS dan letakkan ke dalam folder tersebut dengan nama berkas yang sama persis:

- **Path Sertifikat:** /etc/nginx/ssl/fullchain.pem
- **Path Kunci Privat:** /etc/nginx/ssl/privkey.pem

Opsi A: Membuat Berkas Secara Manual via Editor nano di VPS:

```
sudo nano /etc/nginx/ssl/fullchain.pem
```

```
sudo nano /etc/nginx/ssl/privkey.pem
```

Opsi B: Transfer Berkas Secara Langsung Menggunakan scp dari Terminal Ubuntu Lokal: (Catatan: Karena pengguna login SSH biasa lims tidak diizinkan menulis berkas secara langsung ke folder /etc/nginx/ssl/ milik root, berkas dikirim terlebih dahulu ke folder /tmp/ di VPS, baru kemudian dipindahkan menggunakan hak akses sudo mv).

1. Jalankan perintah transfer ini **di terminal Ubuntu Lokal**
Anda (ganti /path/to/ dengan lokasi folder sertifikat Anda):
2. `scp /path/to/fullchain.pem lims@212.85.24.33:/tmp/`
3. `scp /path/to/privkey.pem lims@212.85.24.33:/tmp/`
4. Jalankan perintah ini **di terminal VPS Anda** untuk memindahkan berkas ke folder tujuan Nginx SSL:
5. `sudo mv /tmp/fullchain.pem /etc/nginx/ssl/`
6. `sudo mv /tmp/privkey.pem /etc/nginx/ssl/`

3. Amankan Hak Akses Kunci Privat

Batasi izin akses agar kunci privat aman:

```
sudo chmod 600 /etc/nginx/ssl/privkey.pem
```

```
sudo chmod 644 /etc/nginx/ssl/fullchain.pem
```

PROPOSAL TEKNIS (REVISI 4): INTEGRASI KEAMANAN & HARDENING LIMS

Dokumen ini menambahkan aspek keamanan tingkat tinggi untuk mencegah kebocoran token, penyalahgunaan token oleh pihak ketiga (Postman/hacker), serta pengamanan fitur Login Overwrite dengan skrip SQL berbasis skema lims.

TANGGAPAN TERHADAP CATATAN KEAMANAN & PRIVASI

1. Pencegahan Penggunaan Token Bersamaan (Anti-Token Sharing / Hijacking)

Catatan: "Seharusnya jika session di database active maka token tidak boleh digunakan berbarengan. Bagaimana kontrolnya supaya orang lain tidak bisa akses LIMS menggunakan token orang lain?"

Solusi Teknis: Untuk mencegah seseorang menyalin token JWT orang lain dan menaruhnya di Postman atau perangkat berbeda, kita menerapkan **Validasi Sidik Jari Client (Client Fingerprinting)** pada middleware.

1. **IP Address & User-Agent Binding (Pengikatan Perangkat):**
 - o Saat user sukses melakukan login, backend mencatat IP Address dan User-Agent (informasi browser/aplikasi) ke dalam database tabel `lims.user_sessions`.
 - o Pada setiap request API, AuthMiddleware akan mencocokkan IP Address dan User-Agent pengirim request dengan data yang terdaftar di database untuk token tersebut.
 - o Jika tidak cocok (misalnya token asli dari Chrome PC, lalu dicoba diakses dari Postman/PC lain dengan IP/User-Agent berbeda), backend langsung mengembalikan **401 Unauthorized**, **menghapus sesi tersebut dari database**, dan memaksa logout di kedua sisi.

Catatan Optimasi: Pada jaringan seluler yang IP-nya dinamis, kita bisa menggombinasikan dengan **Device Fingerprint** unik yang dikirimkan via Header khusus (misal X-Device-ID).

2. Perlindungan Token via Secure HttpOnly Cookie:

- Token JWT **tidak boleh disimpan di localStorage** karena rentan dicuri lewat serangan XSS (Cross-Site Scripting).
- Token harus disimpan secara eksklusif dalam cookie dengan atribut HttpOnly, Secure, dan SameSite=Strict.
- Atribut HttpOnly menjamin bahwa kode JavaScript (termasuk script berbahaya) **tidak dapat membaca token** lewat browser. Hanya browser yang secara otomatis menempelkan cookie tersebut saat melakukan request ke backend.

2. Mengamankan LIMS dari Hacker (Hardening Login & Takeover)

Catatan: *"Bagaimana membuat LIMS aman dari hacker?"*

Untuk memastikan fitur pengambilalihan sesi (force_login: true) tidak dieksploitasi oleh hacker yang mencoba melakukan Brute Force (tebak password) atau pembajakan sesi, kita menerapkan langkah pengamanan berikut:

1. Rate Limiting & Account Lockout (Anti Brute-Force):

- Membatasi percobaan login maksimal 5 kali dalam 15 menit per akun/IP.
- Jika melebihi batas, akun akan dikunci otomatis selama 15 menit, dan IP tersebut akan diblokir sementara dari akses endpoint /login. Hal ini mencegah hacker menggunakan bot untuk menebak password.

2. Notifikasi Pengambilalihan Sesi (Real-time Alert):

- Ketika sesi pengguna diambil alih (force_login bernilai true), backend secara otomatis mengirimkan notifikasi instan kepada pemilik akun melalui channel komunikasi yang terhubung (WhatsApp / Telegram Bot LIMS).
- **Pesan Notifikasi:**

"⚠ Peringatan Keamanan LIMS: Akun Anda baru saja login dari perangkat lain (IP: 192.168.10.12). Sesi lama Anda telah dinonaktifkan. Jika ini bukan Anda, segera ganti password Anda."

3. Multi-Factor Authentication (MFA) / OTP:

- Untuk akun dengan hak akses krusial (seperti Administrator atau Supervisor), saat mendeteksi adanya sesi aktif di perangkat lain, proses force_login tidak langsung masuk, melainkan meminta kode OTP 6-digit yang dikirimkan ke Telegram/WhatsApp terdaftar pengguna. Hacker tidak akan bisa masuk meskipun mengetahui password, karena tidak memiliki akses fisik ke Telegram/WhatsApp pengguna.

4. Enkripsi Lalu Lintas Data (HTTPS & SSL Pinning):

- Seluruh API LIMS wajib berjalan di atas protokol HTTPS (SSL/TLS versi terbaru). Hal ini mencegah hacker melakukan penyadapan data di jaringan (Man-in-the-Middle Attack).
- Untuk aplikasi LIMS berbasis mobile Android, terapkan **SSL Pinning** agar koneksi aplikasi tidak bisa di-intercept oleh tools seperti Fiddler atau Charles Proxy.

DETETKSI BINDING PADA KODE BACKEND (GO)

A. Penyimpanan Sidik Jari Sesi saat Login (controllers/auth_controller.go)

```
// Menangkap IP dan User-Agent saat membuat sesi baru
session := models.UserSession{
    UserID:      user.ID,
    Token:       tokenString,
    ExpiresAt:   expiresAt,
    IPAddress:   strings.TrimPrefix(c.ClientIP(), "::ffff:"),
    UserAgent:   c.GetHeader("User-Agent"), // Kolom Baru: Menyimpan sidik jari
    browser/klien
    ClientVersion: c.GetHeader("X-App-Version"),
    ClientPlatform: c.GetHeader("X-App-Platform"),
    LastActivityAt: time.Now(),
}
}
```

B. Validasi Sidik Jari pada Middleware (middleware/auth.go)

```
func AuthMiddleware() gin.HandlerFunc {
    return func(c *gin.Context) {
        // ... pembacaan token ...

        var session models.UserSession
        result := database.DB.Where("token = ? AND user_id = ?", tokenString,
        userID).First(&session)
        if result.Error != nil {
            c.JSON(http.StatusUnauthorized, gin.H{"error": "Session tidak
            ditemukan"})
            c.Abort()
            return
        }

        // 1. VALIDASI SIDIK JARI (Anti-Token Sharing / Hijacking)
        currentIP := strings.TrimPrefix(c.ClientIP(), "::ffff:")
        currentUserAgent := c.GetHeader("User-Agent")

        if session.IPAddress != currentIP || session.UserAgent !=
        currentUserAgent {
            // Deteksi akses ilegal: Hapus sesi dari DB segera
            database.DB.Delete(&session)

            // Catat ke log keamanan
            utils.WriteSecurityLog(userID, currentIP, currentUserAgent,
            "TOKEN_HIJACKING_ATTEMPT")

            c.JSON(http.StatusUnauthorized, gin.H{
                "error": "Akses Ditolak. Deteksi perubahan perangkat atau
                IP ilegal. Sesi Anda telah ditutup.",
            })
            c.Abort()
            return
        }
    }
}
```



```

    }

    // ... Validasi status user aktif & idle timeout ...
    c.Next()
}
}

```

SKRIP DATABASE UPDATE (DDL MANUWAL)

Untuk mengimplementasikan fitur keamanan di atas, jalankan skrip DDL berikut pada PostgreSQL secara manual. Pastikan untuk menyertakan prefix skema lims. pada setiap nama tabel agar tidak terjadi kesalahan relation does not exist:

-- 1. Tambah kolom *user_agent* dan *last_activity_at* di tabel *lims.user_sessions*

```
ALTER TABLE lims.user_sessions ADD COLUMN IF NOT EXISTS user_agent
VARCHAR(255) DEFAULT NULL;
```

```
ALTER TABLE lims.user_sessions ADD COLUMN IF NOT EXISTS last_activity_at
TIMESTAMP WITHOUT TIME ZONE DEFAULT NOW();
```

-- 2. Tambah kolom *is_active* dan *idle_timeout_minutes* di tabel *lims.users* & *lims.hist_users*

```
ALTER TABLE lims.users ADD COLUMN IF NOT EXISTS is_active BOOLEAN
NOT NULL DEFAULT TRUE;
```

```
ALTER TABLE lims.users ADD COLUMN IF NOT EXISTS idle_timeout_minutes INT
DEFAULT NULL;
```

```
ALTER TABLE lims.hist_users ADD COLUMN IF NOT EXISTS is_active BOOLEAN
NOT NULL DEFAULT TRUE;
```

```
ALTER TABLE lims.hist_users ADD COLUMN IF NOT EXISTS idle_timeout_minutes
INT DEFAULT NULL;
```

-- 3. Membuat tabel penyimpanan OTP sementara di skema *lims*

```
CREATE TABLE IF NOT EXISTS lims.otp_codes (
    id SERIAL PRIMARY KEY,
    user_id INT NOT NULL REFERENCES lims.users(id) ON DELETE CASCADE,
    code VARCHAR(10) NOT NULL,
    expires_at TIMESTAMP WITHOUT TIME ZONE NOT NULL,
    created_at TIMESTAMP WITHOUT TIME ZONE DEFAULT NOW()
);
```

-- Index untuk mempercepat query pencocokan OTP

```
CREATE INDEX IF NOT EXISTS idx_otp_codes_user_code ON
lims.otp_codes(user_id, code);
```

-- 4. Pengisian Parameter Global (DML)

-- A. Parameter untuk mengaktifkan Single-Session (true/false)

```
INSERT INTO lims.global_parameters (param_key, param_value, description,
created_at, updated_at, created_user, updated_user)
```

```
VALUES (
    'SINGLE_SESSION_MODE',
    'false',
    'Menentukan apakah user hanya boleh memiliki satu sesi aktif (true) atau multi sesi (false).',
    NOW(), NOW(), 'SYSTEM', 'SYSTEM'
) ON CONFLICT (param_key) DO UPDATE SET description =
EXCLUDED.description;
```

-- B. Parameter untuk mengizinkan pengambilalihan sesi/force login (true/false)

```
INSERT INTO lims.global_parameters (param_key, param_value, description,
created_at, updated_at, created_user, updated_user)
VALUES (
    'ALLOW_SESSION_TAKEOVER',
    'true',
    'Mengizinkan user mengambil alih sesi aktif lainnya saat login kembali jika bernilai true.',
    NOW(), NOW(), 'SYSTEM', 'SYSTEM'
) ON CONFLICT (param_key) DO UPDATE SET description =
EXCLUDED.description;
```

-- C. Parameter untuk waktu idle global default (dalam menit)

```
INSERT INTO lims.global_parameters (param_key, param_value, description,
created_at, updated_at, created_user, updated_user)
VALUES (
    'DEFAULT_IDLE_TIMEOUT_MINUTES',
    '30',
    'Batas waktu idle sistem default dalam menit sebelum sesi ditutup otomatis.',
    NOW(), NOW(), 'SYSTEM', 'SYSTEM'
) ON CONFLICT (param_key) DO UPDATE SET description =
EXCLUDED.description;
```

-- D. Parameter Batas Maksimal Percobaan Login Salah

```
INSERT INTO lims.global_parameters (param_key, param_value, description,
created_at, updated_at, created_user, updated_user)
VALUES (
    'LOGIN_MAX_ATTEMPTS',
    '5',
    'Jumlah maksimal percobaan login salah sebelum akun dikunci sementara.',
    NOW(), NOW(), 'SYSTEM', 'SYSTEM'
) ON CONFLICT (param_key) DO UPDATE SET description =
EXCLUDED.description;
```

-- E. Parameter Durasi Waktu Akun Dikunci (Menit)

```
INSERT INTO lims.global_parameters (param_key, param_value, description,
created_at, updated_at, created_user, updated_user)
```

```
VALUES (
    'LOGIN_LOCKOUT_MINUTES',
    '15',
    'Durasi waktu (dalam menit) akun dikunci akibat salah password berturut-turut.',
    NOW(), NOW(), 'SYSTEM', 'SYSTEM'
) ON CONFLICT (param_key) DO UPDATE SET description =
EXCLUDED.description;
```

Q. Monitoring Infrastruktur LIMS (Grafana & Prometheus)

Monitoring LIMS menggunakan *stack* Prometheus dan Grafana adalah standar industri untuk memantau performa dan ketersediaan layanan secara *real-time*.

1. Arsitektur Monitoring & Penjelasan Fungsi

Monitoring LIMS dirancang dengan memisahkan tugas pengumpulan data (Prometheus) dan visualisasi (Grafana):

- **Fungsi Prometheus:** Bertindak sebagai *Time-Series Database* (TSDB) sekaligus mesin penarik (*scraper*). Prometheus tidak menunggu data dikirim kepadanya, melainkan secara aktif mengambil (pull) data berupa angka-angka (*metrics*) dari berbagai titik aplikasi (*exporter*) secara berkala.
- **Fungsi Grafana:** Bertindak sebagai "wajah" (UI) dari sistem monitoring. Grafana tidak menyimpan data, melainkan membaca data dari Prometheus lalu menerjemahkannya ke dalam bentuk grafik, diagram, dan *dashboard* yang interaktif. Grafana juga berfungsi mengirimkan *Alert* (peringatan via Telegram/Email) jika ada parameter yang melebihi batas (misal: CPU > 90%).
- **Node Exporter:** Agen yang membaca beban CPU, RAM, Disk I/O, dan Network dari sistem operasi server host.
- **Postgres Exporter:** Mengekspos status *connection*, transaksi, dan lambatnya *query* dari database LIMS.
- **NGINX Exporter:** Membaca metrik trafik web (*stub_status*) NGINX.

Mengapa Menggunakan Docker untuk Instalasi? Pendekatan menggunakan Docker sangat disarankan karena:

1. **Isolasi (Sandboxing):** Ekosistem monitoring (Prometheus + Grafana + Exporter) tidak akan mengotori OS Linux server Anda dengan *library* atau dependensi yang aneh-aneh. Semuanya terbungkus rapi di dalam *container*.
2. **Keamanan Port:** Anda dapat menghubungkan Prometheus ke *exporter* menggunakan jaringan internal Docker tanpa harus mengekspos port-port tersebut ke dunia luar (Internet).
3. **Kemudahan Manajemen:** Cukup satu perintah `docker-compose up` untuk menyalakan seluruh sistem monitoring, dan `docker-compose down` untuk mematikannya tanpa meninggalkan sampah sistem.

2. Persiapan File Konfigurasi (Prometheus)

Buat folder khusus untuk menyimpan konfigurasi monitoring:

```
mkdir -p /home/lims/monitoring/prometheus
```

```
nano /home/lims/monitoring/prometheus/prometheus.yml
```

Isi file `prometheus.yml`: (Catatan: Anda bebas mengubah `scrape_interval` dari 15s (15 detik) menjadi 1m (1 menit) jika Anda ingin menghemat penyimpanan data atau jika trafik tidak terlalu padat).

global:

scrape_interval: 1m # *Interval waktu Prometheus menarik data (bisa diatur 15s, 30s, atau 1m)*

scrape_configs:

- job_name: 'prometheus'

 - static_configs:

 - targets: ['localhost:9090']

- job_name: 'node_exporter'

 - static_configs:

 - targets: ['node-exporter:9100']

- job_name: 'postgres_exporter'

 - static_configs:

 - targets: ['postgres-exporter:9187']

- job_name: 'nginx_exporter'

 - static_configs:

 - targets: ['nginx-exporter:9113']

- job_name: 'pm2_exporter'

 - fallback_scrape_protocol: PrometheusText0.0.4

 - static_configs:

 - targets: ['host.docker.internal:9209']

3. Instalasi via Docker Compose

Prasyarat: Instalasi Docker (Jika Belum Ada) Jika server VPS Anda belum ter-install Docker dan plugin Compose, jalankan perintah berikut, lalu masukkan user lims ke dalam grup docker:

```
sudo apt update
```

```
sudo apt install docker.io docker-compose-plugin -y
```

```
sudo usermod -aG docker lims
```

```
newgrp docker
```

Buat file docker-compose.yml di folder monitoring:

```
nano /home/lims/monitoring/docker-compose.yml
```

Isi dengan skrip berikut:

services:

prometheus:

- image: prom/prometheus:latest

- container_name: prometheus

- volumes:

 - ./prometheus/prometheus.yml:/etc/prometheus/prometheus.yml

 - prometheus_data:/prometheus

- extra_hosts:

 - "host.docker.internal:host-gateway"

- ports:

- "127.0.0.1:9090:9090" # *HANYA bisa diakses lokal demi keamanan*
restart: always

grafana:

image: grafana/grafana:latest

container_name: grafana

environment:

- GF_SECURITY_ADMIN_PASSWORD=PasswordGrafanaKuat123

volumes:

- grafana_data:/var/lib/grafana

ports:

- "127.0.0.1:3010:3000" # *Diekspos di port 3010 untuk menghindari konflik dengan*

Frontend

restart: always

depends_on:

- prometheus

node-exporter:

image: prom/node-exporter:latest

container_name: node_exporter

volumes:

- /proc:/host/proc:ro

- /sys:/host/sys:ro

- /:/rootfs:ro

command:

- '--path.procfs=/host/proc'

- '--path.rootfs=/rootfs'

- '--path.sysfs=/host/sys'

- '--collector.filesystem.mount-points-exclude=^/(sys|proc|dev|host|etc)(\$\$|/)'

restart: always

postgres-exporter:

image: prometheuscommunity/postgres-exporter

container_name: postgres_exporter

extra_hosts:

- "host.docker.internal:host-gateway"

environment:

-

"DATA_SOURCE_NAME=postgres://lims_grafana@host.docker.internal:5432/lims_p
rod_db?sslmode=disable"

- "PGPASSFILE=/root/.pgpass"

volumes:

- /.pgpass:/root/.pgpass:ro

restart: always

nginx-exporter:

```
image: nginx/nginx-prometheus-exporter:latest
container_name: nginx_exporter
extra_hosts:
  - "host.docker.internal:host-gateway"
command:
  - "--nginx.scrape-uri=http://host.docker.internal:8084/stub_status"
restart: always
```

```
loki:
  image: grafana/loki:latest
  container_name: loki
  ports:
    - "127.0.0.1:3100:3100"
  restart: always
```

```
promtail:
  image: grafana/promtail:latest
  container_name: promtail
  volumes:
    - /var/log/nginx:/var/log/nginx:ro
    - ./promtail-config.yml:/etc/promtail/config.yml
  command: -config.file=/etc/promtail/config.yml
  restart: always
```

```
volumes:
  prometheus_data:
  grafana_data:
```

4. Konfigurasi Hak Akses Monitoring di PostgreSQL

Agar *PostgreSQL Exporter* bisa membaca statistik beban dan aktivitas *database*, user yang dipakai (*lims_grafana*) wajib diberi peran khusus bernama *pg_monitor*. Jalankan perintah berikut di terminal server:

```
sudo -u postgres psql
Lalu eksekusi kueri SQL ini:
GRANT pg_monitor TO lims_grafana;
```

```
\q
```

(Catatan: Jika user lims_grafana belum ada, buat terlebih dahulu dengan CREATE USER lims_grafana WITH PASSWORD 'password_rahasia'; lalu sesuaikan isi file .pgpass).

5. Menjalankan Monitoring Stack

Pindah ke direktori tersebut dan jalankan *container* di latar belakang:

```
cd /home/lims/monitoring
```

```
docker compose up -d
```

(Catatan: Jika server Anda menggunakan versi Docker lama, gunakan perintah docker-compose up -d (menggunakan tanda hubung/strip)).

6. Ekspos Grafana via NGINX (Reverse Proxy)

Agar Grafana bisa diakses dengan aman melalui internet, Anda perlu menambahkan blok server ke konfigurasi NGINX VPS Anda.

Buat file konfigurasi Nginx baru di dalam folder conf.d (contoh menggunakan nama domain lims-d4551821.nip.io):

```
sudo nano /etc/nginx/conf.d/monitoring.conf
```

Isi file tersebut dengan blok berikut (menggunakan SSL/HTTPS):

```
# Endpoint Khusus Nginx Exporter (Port 8084)
```

```
server {  
    listen 8084;  
    listen [::]:8084;  
    server_name _; # Menangkap semua hostname  
  
    location /stub_status {  
        stub_status on;  
        allow all;  
    }  
}
```

```
# Konfigurasi HTTPS (Grafana di Port 8083)
```

```
server {  
    listen 8083 ssl;  
    listen [::]:8083 ssl;  
    http2 on;  
    server_name lims-d4551821.nip.io;
```

```
# Ganti path ini sesuai dengan lokasi sertifikat SSL Anda (misal dari Certbot)
```

```
ssl_certificate /etc/letsencrypt/live/lims-d4551821.nip.io/fullchain.pem;  
ssl_certificate_key /etc/letsencrypt/live/lims-d4551821.nip.io/privkey.pem;
```

```
# Halaman Dashboard Grafana
```

```
location / {  
    proxy_pass http://127.0.0.1:3010;  
    proxy_set_header Host $http_host;  
    proxy_set_header X-Real-IP $remote_addr;  
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;  
    proxy_set_header X-Forwarded-Proto $scheme;  
}  
}
```

Aktifkan konfigurasi tersebut dengan melakukan verifikasi sintaks lalu *reload* Nginx:

```
sudo nginx -t
```

```
sudo systemctl reload nginx
```

(Pastikan Anda melengkapi blok di atas dengan Sertifikat SSL/HTTPS Let's Encrypt menggunakan certbot).

7. Konfigurasi Awal di Dashboard Grafana

1. Buka browser dan akses Grafana Anda.

- **Jika via Nginx (Internet):** Buka `https://lims-d4551821.nip.io:8083` (Pastikan port 8083 terbuka di Firewall).
 - **Jika via SSH Tunnel (Lokal):** Buka `http://localhost:3010`.
- 2. Halaman Login Grafana akan muncul. Masukkan kredensial berikut (sesuai yang kita atur di `docker-compose.yml`):
 - **Username:** admin
 - **Password:** PasswordGrafanaKuat123
- 3. Di Menu **Connections** →→ **Data Sources**, tambahkan **Prometheus**.
- 4. Masukkan URL: `http://prometheus:9090` lalu klik **Save & Test**.
- 5. **Mengimpor Dashboard:** Di menu sebelah kiri, buka **Dashboards** →→ klik tombol **New** (di pojok kanan atas) →→ pilih **Import**. Masukkan ID Template berikut lalu klik **Load**:
 - **1860:** Node Exporter Full (Metrik Server OS).
 - **9628:** PostgreSQL Database.
 - **12708:** NGINX Exporter. *(Catatan: Saat Anda mengklik Load, Anda akan diminta mengisi dropdown **DS_PROMETHEUS**. Silakan klik dan pilih **Prometheus**. Jika pilihannya kosong, artinya Anda belum melakukan Langkah 3 & 4 di atas!)*
- 6. **Manajemen Pengguna (Ganti Password & User Baru):**
 - **Ganti Password Sendiri:** Klik ikon Profil Anda (pojok kiri bawah) →→ **Profile** →→ **Change Password**.
 - **Buat User Baru:** Buka menu **Administration** →→ **Users and access** →→ **Users** →→ klik tombol **New user**.

8. Monitoring Frontend (Node.js/React) via PM2

Untuk melihat detail pemakaian CPU dan RAM dari masing-masing aplikasi yang berjalan di PM2, kita menggunakan modul resmi `pm2-prometheus-exporter`.

1. **Instalasi Exporter di Server:** Jalankan perintah ini di terminal server Anda (tanpa `sudo` jika PM2 berjalan di user Anda):
2. `pm2 install pm2-prometheus-exporter`

Modul ini akan membuka port 9209 secara otomatis.

3. **Konfigurasi Prometheus:** Pastikan Anda sudah menambahkan blok `pm2_exporter` ke dalam `prometheus.yml` Anda (seperti yang ditunjukkan di Langkah 2). Penting untuk menggunakan `fallback_scrape_protocol: PrometheusText0.0.4` untuk menghindari error blank Content-Type. Pastikan juga Anda sudah merestart Prometheus: `sudo docker restart prometheus`.
4. **Impor Dasbor PM2 di Grafana:** Di menu Grafana, buka **Dashboards** →→ **New** →→ **Import**. Masukkan ID Dasbor: **12745** (Dasbor PM2 resmi).
5. **Penyesuaian Dasbor (Jika "No Data"):** Terkadang dasbor ID 12745 memfilter data menggunakan variabel bawaan yang tidak cocok dengan Docker. Jika grafik kosong:
 - Buka panel CPU, klik titik tiga →→ **Edit**.
 - Ubah Query rumit di bawah menjadi murni `pm2_cpu`.
 - Lakukan hal yang sama untuk panel Memory, ubah menjadi `pm2_memory`.

- Simpan Dasbor. Grafik Anda akan langsung muncul menampilkan setiap proses aplikasi yang dikelola PM2!

9. Monitoring Trafik Frontend & NGINX Access Logs (Loki + Promtail)

Untuk melihat statistik pengunjung yang nyata (jumlah akses, rute/URL yang paling sering diakses, serta mendeteksi *error* 404/500), kita menggunakan Grafana Loki dan Promtail untuk membaca file access.log Nginx secara real-time.

1. **Membuat File Konfigurasi Promtail** Buat file promtail-config.yml di folder monitoring Anda:
2. `nano /home/lims/monitoring/promtail-config.yml`

Isi dengan skrip berikut:

server:

```
http_listen_port: 9080
grpc_listen_port: 0
```

positions:

```
filename: /tmp/positions.yaml
```

clients:

```
- url: http://loki:3100/loki/api/v1/push
```

scrape_configs:

```
- job_name: nginx
  static_configs:
    - targets:
        - localhost
    labels:
      job: nginx
      host: server-lims
      __path__: /var/log/nginx/json_access.log
```

3. **Konfigurasi Format Log JSON di NGINX (Wajib untuk Dasbor!)** Dasbor Loki membutuhkan log dalam bentuk tabel/JSON agar bisa membuat grafik. Format bawaan Nginx (teks biasa) tidak akan terbaca. Buka file konfigurasi utama Nginx:

4. `sudo nano /etc/nginx/nginx.conf`

Cari blok `http {` dan tambahkan format JSON ini di dalamnya (tepat di bawah baris `include /etc/nginx/mime.types;`):

```
log_format json_analytics escape=json '{'
  "msec": "$msec", '
  "remote_addr": "$remote_addr", '
  "request_uri": "$request_uri", '
  "status": "$status", '
  "body_bytes_sent": "$body_bytes_sent", '
  "request_time": "$request_time", '
  "http_referer": "$http_referer", '
  "http_user_agent": "$http_user_agent"
  }';
```

Lalu, terapkan format tersebut dengan mengubah/menambahkan baris `access_log` menjadi seperti ini:

```
access_log /var/log/nginx/json_access.log json_analytics;
```

Simpan file, lalu *reload* Nginx:

```
sudo nginx -t
```

```
sudo systemctl reload nginx
```

5. **Memastikan Izin File Log** Karena Promtail berjalan di Docker, kita harus memastikan ia memiliki izin membaca log Nginx:
6. `sudo chmod 644 /var/log/nginx/json_access.log`
7. **Menerapkan Perubahan Docker Compose** Pastikan Anda sudah menambahkan blok *service* loki dan promtail ke dalam file `docker-compose.yml` Anda. Tambahkan blok berikut ini tepat di atas baris `volumes`: paling bawah:
8. `loki:`
9. `image: grafana/loki:latest`
10. `container_name: loki`
11. `ports:`
12. `- "127.0.0.1:3100:3100"`
13. `restart: always`
- 14.
15. `promtail:`
16. `image: grafana/promtail:latest`
17. `container_name: promtail`
18. `volumes:`
19. `- /var/log/nginx:/var/log/nginx:ro`
20. `- ./promtail-config.yml:/etc/promtail/config.yml`
21. `command: -config.file=/etc/promtail/config.yml`
22. `restart: always`

(Catatan: Anda juga bisa melihat bentuk dokumen `docker-compose.yml` yang utuh dan lengkap di panduan **Langkah 3** di atas).

Setelah ditambahkan, jalankan ulang:

```
cd /home/lims/monitoring
```

```
sudo docker compose up -d
```

23. Integrasi ke Grafana

- Di Dasbor Grafana, buka **Connections** →→ **Data Sources** →→ **Add data source**.
- Pilih **Loki**.
- Di kolom URL, masukkan: `http://loki:3100`.
- Klik **Save & test** (pastikan muncul tulisan hijau "Data source connected and labels found").

24. Mengimpor Dasbor Log Nginx Buka Dashboards →→ New →→ Import.

Masukkan ID Dasbor: **13740** atau **24306**. Di bagian opsi `datasource` bawah, pastikan memilih **Loki**. Setelah diklik *Import*, Anda kini bisa melihat secara detail trafik HTTP pengunjung Frontend Anda!